



Instituto Federal do Espírito Santo - Campus Serra

Programação Orientada a Objetos Teoria e Prática

Uma abordagem objetiva, incremental e aplicada

Prof. Dr. Hilario Seibel Junior

2026

Última atualização: 17 de março de 2026

Sumário

Prefácio	v
I Teoria	1
1 Classes e Objetos	2
1.1 Paradigmas de Programação	2
1.2 Classes	3
1.3 Objetos	4
1.4 Exercícios	5
2 Métodos	6
2.1 Métodos de uma classe	6
2.2 Adicionando funcionalidades	8
2.3 Exercício: Juros do cheque especial	10
3 Construtores	12
3.1 Construtor Padrão	12
3.2 Rotinas de Inicialização	13
3.3 E como funcionava até agora?	16
3.4 Exercícios	16
4 Herança	18
4.1 Classes derivadas	18
4.2 E o construtor?	19
4.3 Exercícios	21
5 Reescrita, Sobrecarga e I/O	24
5.1 Reescrita de Métodos	24
5.2 Entrada de dados	27

5.3	Sobrecarga de métodos	29
5.4	Exercícios	30
6	Encapsulamento	32
6.1	Modificadores de Acesso	32
6.2	Métodos getters e setters	34
6.3	Visibilidade dos Modificadores de Acesso	34
6.4	Exercícios	35
7	Array e ArrayList	36
7.1	Arrays	36
7.2	ArrayLists	37
7.3	Arrays Vs. ArrayLists	37
7.4	Exercícios e Métodos Estáticos	38
8	Exercício de Revisão	41
8.1	Empresa de RH	41
8.2	Testando o sistema	43
9	Tratamento de Erros e Exceções	44
9.1	Tratando uma exceção com try-catch	44
9.2	O comando finally	46
9.3	Outras exceções comuns	46
9.4	Lançando uma exceção	48
9.5	Definindo nossas próprias exceções	49
9.6	Exercícios	50
10	Persistência em Arquivos	51
10.1	Salvando dados em arquivos textos	51
10.2	Lendo dados de arquivos textos	52
10.3	Exercícios	52
11	Polimorfismo, Classes Abstratas e Interfaces	53
11.1	Classes Abstratas	54
11.2	Interfaces	55
11.3	Classes Abstratas Vs. Interfaces	57
11.4	Polimorfismo	58

11.5	Exercícios	58
12	Coleções e Ordenação	59
12.1	Coleções	59
12.2	Ordenação	62
12.3	Exercícios	64
II	Prática	66
13	Classes e Objetos	67
13.1	Primeiros passos	67
13.2	Criando novas classes	69
13.3	Testando o programa	70
14	Métodos - Prática	72
14.1	Crie um novo projeto	72
14.2	Testando o código até aqui	74
14.3	Juros do cheque especial	75
15	Construtores - Prática	77
15.1	Retomando o projeto Banco	77
15.2	Criando os construtores	77
15.3	Testando os construtores	78
15.4	Exercícios da Aula 3	79
16	Herança - Prática	81
16.1	Retomando o projeto Banco	81
16.2	Adicionando herança às classes	81
17	Reescrita de métodos, Sobrecarga de Métodos, E/S - Prática	85
18	Encapsulamento - Prática	90
18.1	Retomando o projeto Banco	90
19	Array e ArrayList - Prática	94
19.1	Classe Sistema	94
19.2	Classe Entrada	95
19.3	Menu interativo no projeto do Banco	96

19.4	Métodos para cadastros	97
19.5	Métodos para movimentações financeiras	100
19.6	Testando o menu	102
20	Revisão - Prática	104
20.1	Por onde começar?	104
20.2	Classe Data	105
20.3	Classe Funcionario	105
20.4	Classe Gerente	106
20.5	Classe Empresa	107
20.6	Classe Entrada	110
20.7	Classe principal	114
III	Extra	115
A	Orientação a Objetos em Python	116
A.1	Classes, objetos e construtores	116
A.2	Métodos	117
A.3	Herança	118
A.4	Encapsulamento e modificadores de acesso	119
A.5	Exercício	120
B	Pygame	121
B.1	Primeiro Exemplo	121
B.2	Adicionando um objeto em movimento	122
B.3	Criando mais de um objeto	123
B.4	Criando uma classe para os objetos	124
B.5	Lidando com vários objetos	125
B.6	Exercício	126
B.7	Segundo Exemplo: Ping	126
B.8	Colisões	129
B.9	Ping Pong	130
B.10	Exercício 2	131

Prefácio

Este material foi elaborado para apoiar o estudo de **Programação Orientada a Objetos** de forma **clara**, **progressiva** e **aplicada**. A proposta é apresentar os conceitos fundamentais em uma abordagem acessível, com exemplos práticos e foco na construção de uma base sólida para o desenvolvimento de software.

O material está dividido em aulas. Ao longo delas, os tópicos evoluem de maneira incremental: começamos com classes e objetos, avançamos para métodos, construtores, herança, encapsulamento, polimorfismo, etc, e assim os códigos vão incorporando as boas práticas de programação orientada a objetos a cada novo conceito introduzido.

Embora os exemplos utilizem a linguagem Java, os princípios discutidos são amplamente aplicáveis a outras linguagens orientadas a objetos. Recomenda-se que o leitor acompanhe cada aula implementando os exemplos e propondo pequenas variações, pois a prática constante é essencial para consolidar o aprendizado.

O material é voltado principalmente para o leitor que já possui algum contato com programação. A primeira parte apresenta os principais conceitos da programação orientada a objetos, com alguns exemplos práticos em Java. A Parte II foca em resolver os exercícios apresentados ao longo da Parte I.

Parte I

Teoria

Aula 1

Classes e Objetos

Esta primeira aula apresenta uma introdução ao paradigma orientado a objetos e aos conceitos de **classes** e **objetos**, que vão te nortear por todo o restante do aprendizado.

1.1 Paradigmas de Programação

Qual a diferença entre **programa**, **linguagem de programação**, **código fonte** e **algoritmo**?



Para o computador, um **programa** é apenas uma sequência de instruções (código de máquina) composto por 0's e 1's. Máquinas não entendem linguagens humanas (português, inglês, etc) e humanos, geralmente, não entendem código de máquina. Uma **linguagem de programação** é um meio termo entre humanos e máquinas. Possui um conjunto de regras para descrever o funcionamento de um programa. Um **código-fonte** contém **algoritmos** escritos em alguma linguagem de programação. É preciso haver uma **tradução** de código-fonte escrito em uma certa linguagem de programação para que ele seja transformado em um programa executável por um computador.

Um **paradigma** de programação define o “estilo” de uma linguagem de programação. Existem diversos paradigmas de programação diferentes. Exemplos: *paradigma estruturado*, *paradigma funcional*, *paradigma orientado a objetos*. Python é multiparadigma: é possível escrever código em Python usando o paradigma estruturado, funcional e OO.

No paradigma orientado a objetos, existe:

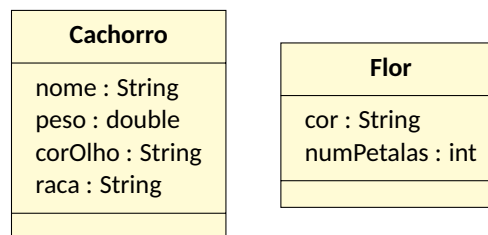
- Abstração e simulação do mundo real;

- Composição e interação entre objetos;
- Reaproveitamento de códigos.

A programação orientada a objetos é especialmente útil quando os programas começam a ficar mais complexos. Um exemplo de linguagem OO é **Java**. Um código orientado a objetos é composto por **classes**, **objetos** e pelo **relacionamentos** entre eles.

1.2 Classes

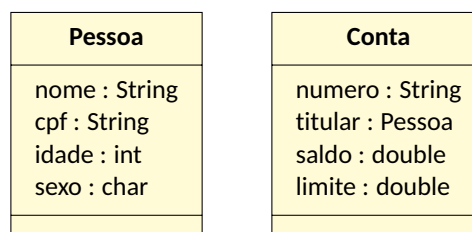
Uma **classe** representa um conjunto de objetos com características semelhantes. Os **atributos** da classe definem o estado de seus objetos. Exemplos:



No modelo de classe acima, os atributos da classe **Cachorro** são nome, peso, cor do olho e raça. Ao lado do nome de cada atributo, mostramos o tipo de cada um. Cada objeto desta classe vai conter algum valor para cada um destes atributos.

O modelo não está escrito em nenhuma linguagem de programação específica: é um diagrama de classe UML, que modela de forma clara a estrutura de um sistema orientado a objetos, exibindo suas classes, atributos, métodos e relacionamentos (que veremos nos próximos capítulos deste material). Ele ajuda a visualizar a estrutura do sistema antes da implementação, e é composto por retângulos divididos em três partes: nome da classe, atributos e operações.

Ao longo deste material, vamos tentar modelar e implementar o sistema de uma agência bancária. Inicialmente, vamos modelar a classe **Pessoa**, para armazenar informações dos clientes, e a classe **Conta**. Para cada pessoa, o sistema deve armazenar seu nome, cpf, sexo e idade. No mundo real, uma pessoa possui diversos atributos além desses, mas uma classe deve armazenar apenas as informações que forem relevantes ao sistema que está sendo implementado. Já uma conta precisa armazenar seu número, titular, saldo e limite:



Note que o atributo **titular** da classe **Conta** é do tipo **Pessoa**, uma outra classe do sistema. Já o atributo **limite** de uma conta indica o valor máximo que o banco permite que o cliente retire de sua conta caso seu saldo não seja positivo. Um limite de R\$ 200,00, por exemplo, indica que o cliente pode continuar retirando dinheiro da conta até que seu saldo seja R\$ -200,00.

Em Java, cada classe deve ser implementada num arquivo separado com o nome da classe. A classe **Pessoa**, por exemplo, é implementada no arquivo **Pessoa.java**:

```
1 class Pessoa {
2     String nome;
3     char sexo;
4     int idade;
5     String cpf;
6 }
```

Ja a classe Conta deve ser implementada no arquivo Conta.java:

```
1 class Conta {
2     String numero;
3     Pessoa titular;
4     double saldo, limite;
5 }
```

1.3 Objetos

Um **objeto** é uma **instância** de uma classe. Cada objeto armazena seus dados através de diferentes valores para os atributos da classe. Para a classe Pessoa, por exemplo, podemos criar objetos com os seguintes valores:

Objetos da classe Pessoa:

"Enzo", 123.456.789-00, 20, 'M'

"Valentina", 234.567.890-00, 18, 'F'

Para criarmos nossos primeiros objetos em Java, vamos criar uma classe **Banco** contendo o método principal do sistema, chamado **main**. É a partir dele que o programa começará a ser executado, e é nele que criamos os objetos das classes que foram definidas. Métodos serão detalhados mais profundamente no Capítulo 2.

```
1 class Banco {
2     public static void main(String[] args) {
3
4         Pessoa p1 = new Pessoa();
5         p1.nome = "Hilario";
6         p1.cpf = "123.456.789-00";
7         p1.idade = 18;
8         p1.sexo = 'M';
9
10        Pessoa p2 = new Pessoa();
11        p2.nome = "Alessandra";
12        p2.cpf = "234.567.890-00";
13        p2.idade = 21;
14        p2.sexo = 'F';
15
16        Conta c1 = new Conta();
17        c1.numero = "1234-5";
18        c1.titular = p1; // Conta pertencente ao Hilario
19        c1.saldo = 100.0;
```

```
20         c1.limite = 200.0;
21
22         Conta c2 = new Conta();
23         c2.numero = "2345-6";
24         c2.titular = p2; // Conta da Alessandra
25         c2.saldo = 150.0;
26         c2.limite = 200.0;
27
28         System.out.println("A conta " + c1.numero + "pertence a " +
29                             c1.titular.nome);
30     }
```

Na linha 2, existem três palavras reservadas que serão explicadas nos próximos capítulos: **public** (Capítulo 6), **static** (Capítulo 7) e **void** (Capítulo 2).

As linhas 4, 10, 16 e 22 criam novos objetos das classes **Pessoa** e **Conta**. A palavra reservada **new** é responsável por instanciar o objeto, reservando espaço para ele na memória, e será explicada no Capítulo 3. As linhas seguintes atribuem valores aos atributos de cada um desses objetos.

A saída impressa pela linha 27 do código será:

```
A conta 1234-5 pertence a Hilario
```

O objeto **c1** é da classe **Conta**. Ao acessarmos o atributo **c1.numero**, será impresso o valor que foi armazenado neste atributo na linha 17 (**c1.numero** = "1234-5");. Já o atributo **c1.titular** é uma **Pessoa**, cujo valor foi associado na linha 18 (**c1.titular** = **p1**;). Portanto, ao acessarmos **p1.titular.nome**, estamos acessando o atributo **nome** do objeto **p1**, que por sua vez foi definido na linha 5 (**p1.nome** = "Hilario";).

1.4 Exercícios

Escolha uma IDE para testar sua primeira aplicação. Implemente as classes **Pessoa**, **Conta** e o método principal na classe **Banco**. Dicas iniciais:

1. Não copie e cole. Vá digitando linha por linha, para se familiarizar com o código em Java. Dependendo da IDE escolhida, ela pode ir auto-completando algumas coisas para facilitar seu trabalho.
2. Altere os valores dos atributos no seu código (por exemplo, o nome das pessoas e o saldo das conta).
3. Altere a impressão no método principal para imprimir algo semelhante a:

```
*** EXTRATO DA CONTA ***
Conta: 1234-5
Titular: Hilario (CPF: 123.456.789-00)
Saldo disponivel para saque: R$ 300.0
```

Note que o valor disponível para saque não é o saldo da conta, e sim a soma do saldo com o limite.

O capítulo 13 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 2

Métodos

Uma classe contendo apenas atributos não tem muita utilidade. Nesta aula, adicionamos algumas funcionalidades às classes do sistema.

2.1 Métodos de uma classe

Métodos definem o comportamento dos objetos de uma classe, e como eles reagem a mensagens enviadas a ele. Usamos os métodos para acessar ou modificar os atributos de uma classe. O modelo abaixo adiciona três métodos à classe Pessoa:

Pessoa
nome : String idade : int sexo : char cpf : String
aniversario() : void alteraCPF(c : String) : void consultaldade() : int

Note que os métodos acima parecem não ter muita utilidade para o sistema, mas servem aqui para mostrar que métodos podem ou não ter parâmetros e/ou retorno. De acordo com o modelo acima, o método **aniversario** não recebe nenhum parâmetro, e não possui retorno (o retorno é *void*, que significa vazio). O método **alteraCPF** recebe uma String **c** como parâmetro, e também não possui retorno. Por fim, o método **consultaldade** não possui parâmetro, e retorna algum valor do tipo inteiro.

Agora, a classe Pessoa no arquivo Pessoa.java passa a conter também a implementação dos métodos:

```
1 public class Pessoa {  
2     String nome;  
3     int idade;  
4     char sexo;  
5     String cpf;  
6  
7     void aniversario() {  
8         /* Adiciona um ano na idade da pessoa */  
9         this.idade = this.idade+1;  
10    }
```

```
10     }
11
12     void atribuiCPF(String c) {
13         /* Altera o valor do atributo CPF */
14         this.cpf = c;
15     }
16
17     int getIdade() {
18         /* Retorna o valor da idade da pessoa */
19         return this.idade;
20     }
21
22 }
```

Os métodos são funcionalidade dos objetos de uma classe. Por isso, para que um método seja invocado, é preciso que algum objeto já instanciado seja associado a ele. Para que o método **aniversario** seja executado no main, por exemplo, faríamos:

```
1 Pessoa p1 = new Pessoa();
2 p1.nome = "Hilario";
3 p1.cpf = "123.456.789-00";
4 p1.idade = 18;
5 p1.sexo = 'M';
6
7 Pessoa p2 = new Pessoa();
8 p2.nome = "Alessandra";
9 p2.cpf = "234.567.890-00";
10 p2.idade = 21;
11 p2.sexo = 'F';
12
13 p1.aniversario();
14 p2.aniversario();
```

O método **não** pode ser invocado sozinho. Na linha 13 do código acima, o método aniversário é executado exclusivamente para o objeto *p1*, e na linha 14 para objeto *p2*.

Voltando ao código do método aniversário:

```
1 void aniversario() {
2     this.idade = this.idade+1;
3 }
```

O objetivo deste método é incrementar a idade do objeto em uma unidade. Entretanto, não existe uma variável **idade** no sistema. O que existe é um atributo **idade** dentro dos objetos da classe **Pessoa**. Seria preciso acessar **p1.idade** ou **p2.idade**. Entretanto, quando estamos implementando o método, não sabemos previamente qual será o nome do objeto (*p1* ou *p2*, nesse caso). Além disso, o mesmo método precisa funcionar para qualquer objeto da classe. Para isso, existe a palavra reservada **this**. Quando você executar **p1.aniversario()**, **this** será equivalente ao objeto *p1*. Assim, neste momento, acessar **this.idade** será equivalente a acessar o valor do atributo **idade** do objeto *p1*.

2.2 Adicionando funcionalidades

Agora, já podemos adicionar funcionalidades a uma Conta no sistema bancário:

Conta
numero : String titular : Pessoa saldo : double limite : double
disponivel() : double depositar(valor : double) : void sacar(valor : double) : boolean transferir(valor : double, destino : Conta) : boolean extrato() : void

O método **disponível** calcula e retorna a quantidade de dinheiro que o cliente pode retirar de sua conta (o banco permite que o cliente continue retirando dinheiro da conta mesmo que o saldo esteja negativo, até atingir o valor do limite). Lembre-se que cada método deve ser implementado dentro da classe, logo após a declaração dos atributos.

```

1 double disponivel() {
2     return this.saldo + this.limite;
3 }

```

O método **depositar** permite que o cliente adicione um valor à sua conta.

```

1 void depositar(double valor) {
2     this.saldo += valor;
3     System.out.println("Deposito de " + valor + " realizado com
4         sucesso.");
5     System.out.println("Novo saldo: " + this.saldo);
6 }

```

O método **sacar** permite que o cliente retire um valor de sua conta desde que haja dinheiro disponível.

```

1 boolean sacar(double valor) {
2     if (valor <= this.disponivel()) {
3         this.saldo -= valor;
4         System.out.println("Saque de " + valor + " realizado com
5             sucesso.");
6         System.out.println("Novo saldo: " + this.saldo);
7         return true;
8     }
9     else {
10        System.out.println("Erro: nao foi possivel sacar " + valor);
11        System.out.println("Valor disponivel para saque: " + this.
12            disponivel());
13        return false;
14    }
15 }

```

O método **transferir** retira um valor de uma conta e deposita esse valor em outra conta.

```

1  boolean transferir(double valor, Conta destino) {
2      if (this.sacar(valor)) { // Se foi possível sacar
3          destino.depositar(valor);
4          System.out.println("Transferencia de " + valor + " realizado
                               com sucesso.");
5          System.out.println("Origem: " + this.numero + " - Destino: "
                               + destino.numero);
6          return true;
7      }
8      else {
9          System.out.println("Erro: nao foi possível transferir " +
                               valor);
10         System.out.println("Valor disponível para transferencia: " +
                               this.disponivel());
11         return false;
12     }
13 }

```

Por fim, o método **extrato** apenas imprime as principais informações da conta, como número, titular e valor disponível.

```

1  void extrato() {
2      System.out.println("*** EXTRATO DA CONTA ***");
3      System.out.println("Conta: " + this.numero);
4      System.out.println("Titular: " + this.titular.cpf);
5      System.out.println("Valor disponível para saque: R$" + this.
                               disponivel() + "\n");
6  }

```

Agora, vamos alterar o método principal para utilizar os métodos acima:

```

1  public static void main(String[] args) {
2
3      Pessoa p1 = new Pessoa();
4      p1.nome = "Hilario";
5      p1.idade = 18;
6      p1.sexo = 'M';
7      p1.cpf = "123.456.789-00";
8
9      Pessoa p2 = new Pessoa();
10     p2.nome = "Alessandra";
11     p2.idade = 21;
12     p2.sexo = 'F';
13     p2.cpf = "234.567.890-00";
14
15     Conta c1 = new Conta();
16     c1.numero = "1234-5";
17     c1.titular = p1;
18     c1.saldo = 100.0;
19     c1.limite = 200.0;
20
21     Conta c2 = new Conta();

```

```
22     c2.numero = "2345-6";
23     c2.titular = p2;
24     c2.saldo = 150.0;
25     c2.limite = 200.0;
26
27     c1.extrato();
28     c2.extrato();
29
30     c1.sacar(150);
31     c1.transferir(100, c2);
32     c1.sacar(100);
33     c1.depositar(100);
34
35 }
```

A saída exata do código acima, de acordo com os métodos que implementamos na classe Conta, será:

```
*** EXTRATO DA CONTA ***
Conta: 1234-5
Titular: 123.456.789-00
Valor disponível para saque: 300.0

*** EXTRATO DA CONTA ***
Conta: 2345-6
Titular: 234.567.890-00
Valor disponível para saque: 350.0

Saque de R$150.0 realizado com sucesso.
Novo saldo: R$-50.0
Saque de R$100.0 realizado com sucesso.
Novo saldo: R$-150.0
Deposito de R$100.0 realizado com sucesso.
Novo saldo: R$250.0
Transferencia de R$100.0 realizado com sucesso.
Origem: 1234-5 - Destino: 2345-6
Erro: nao foi possivel sacar R$100.0
Valor disponivel para saque: R$50.0
Deposito de R$100.0 realizado com sucesso.
Novo saldo: R$-50.0
```

2.3 Exercício: Juros do cheque especial

O cheque especial é um tipo de crédito que o banco oferece ao correntista. É como se fosse um empréstimo pré-aprovado que o banco deixa ali disponível pra você usar a qualquer momento. O valor do cheque especial de cada um muda conforme a avaliação que cada banco faz do cliente. Com o cheque especial, o **limite** da conta fica disponível para que o cliente usar sempre que precisar.

O cheque especial é caro porque o banco cede esse crédito sem pedir nenhuma garantia, ao contrário do que acontece com o empréstimo pessoal. É fácil e prático. Mas esta simplicidade custa caro, e é por isso

que muita gente comete o erro de usar o limite do cheque especial como se fosse uma extensão da conta corrente.

Como a instituição está se arriscando mais, os juros desse tipo de crédito são **muito** mais caros. Enquanto a cobrança de juros no empréstimo consignado chega a uma média de 42,8% ao ano, no cheque especial os juros chegam a 16% ao mês (em torno de 0.5% por dia).

Implemente e teste um método **chequeEspecial** que recebe uma taxa de juros (por exemplo, *juro* = 0.5 indica que a taxa do banco é de 0.5% por dia). O método deve verificar se o saldo está negativo e, caso esteja, aplicar o juro ao saldo devedor.

Para testar seu método, adicione o seguinte código ao final do método main:

```
1 for(int i = 1; i <= 30; i++) {  
2     c1.chequeEspecial(0.5);  
3     System.out.println("Saldo apos " + i + " dia: " + c1.saldo);  
4 }
```

O método acima simula o saldo diário na conta após 30 dias no cheque especial. Após testá-lo, simule também o saldo da conta após 4 meses.

O capítulo 14 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 3

Construtores

Esta aula adiciona rotinas de inicialização às classes do sistema, permitindo que objetos sejam instanciados de forma mais fácil e consistente, e ainda melhorando a legibilidade e reuso do código.

3.1 Construtor Padrão

Nos materiais anteriores, vimos os primeiros conceitos de orientação a objetos: **classes**, **objetos** e **métodos**. O diagrama da classe Conta mostrado a seguir, por exemplo, contém algumas dessas informações:

Conta
numero : String titular : Pessoa saldo : double limite : double
disponivel() : double extrato() depositar(valor : double) sacar(valor : double) : boolean transferir(valor : double, destino : Conta) : boolean chequeEspecial(juro : double)

Até agora, sempre que precisássemos criar um objeto desta ou de outra classe, utilizávamos a palavra reservada **new** e depois atribuíamos manualmente o valor de seus atributos, como no exemplo a seguir:

```
1 public static void main(String[] args) {  
2     Pessoa p1 = new Pessoa();  
3     p1.nome = "Hilario";  
4     p1.idade = 18;  
5     p1.sexo = 'M';  
6     p1.cpf = "123.456.789-00";  
7  
8     Conta c1 = new Conta();  
9     c1.numero = "1234-5";  
10    c1.titular = p1;  
11    c1.saldo = 100.0;  
12    c1.limite = 200.0;
```

```
13
14     c1.extrato();
15     c1.sacar(150);
16     c1.sacar(100);
17     c1.depositar(100);
18
19 }
```

Note que estamos criando novos objetos nas linhas 2 e 8, e que os valores dos atributos destes objetos são definidos nas linhas 3, 4, 5, 6, 9, 10, 11 e 12. A palavra reservada **new** é responsável por “construir” um novo objeto da classe desejada. Sempre que isso é feito, o construtor da classe é executado.

O **construtor** define as rotinas de inicialização de um objeto da classe. Caso não seja definido, o Java implementa um construtor padrão (que utilizamos até agora), que não possui parâmetros, e que simplesmente cria o objeto.

Porém, podemos redefinir o construtor da maneira que quisermos. Por exemplo, podemos criar um construtor para a classe **Pessoa** que simplesmente exibe uma mensagem confirmando que uma nova pessoa foi criada:

```
1  public class Pessoa {
2      String nome;
3      int idade;
4      char sexo;
5      String cpf;
6
7      Pessoa() {
8          System.out.println("Nova pessoa criada no sistema.");
9      }
10
11     /* Demais metodos da classe Pessoa... */
12 }
```

A partir de agora, além de o Java reservar o espaço na memória para armazenar o objeto, a mensagem será exibida na tela a cada vez que um objeto da classe **Pessoa** for criado.

O construtor da classe é um bloco declarado com o mesmo nome que a classe, logo após os atributos e antes dos métodos. Apesar de **bem semelhante**, ele **não é** um método.

3.2 Rotinas de Inicialização

Também podemos criar mais de um construtor na mesma classe. O Java vai diferenciá-los pelos parâmetros que cada um espera receber:

```
1  public class Pessoa {
2      String nome;
3      int idade;
4      char sexo;
5      String cpf;
6
7      Pessoa() {
8          System.out.println("Nova pessoa criada no sistema.");
9      }
10
11     /* Demais metodos da classe Pessoa... */
12 }
```

```

9      }
10
11      Pessoa(String n, int i, char s, String c) {
12          System.out.println("Nova pessoa criada no sistema.");
13          this.nome = n;
14          this.idade = i;
15          this.sexo = s;
16          this.cpf = c;
17      }
18
19      // Demais metodos da classe Pessoa
20  }

```

No segundo construtor acima, além de exibir a mensagem na tela, os valores iniciais de seus atributos já serão automaticamente inicializados, através dos parâmetros recebidos pelo construtor. É como uma **rotina de inicialização** que é chamada sempre que um novo objeto é criado.

O trecho de código abaixo mostra a diferença na chamada de cada um dos construtores quando os objetos são criados:

```

1  public static void main(String[] args) {
2
3      Pessoa p1 = new Pessoa();
4      p1.nome = "Maria";
5      p1.idade = 18;
6      p1.sexo = 'F';
7      p1.cpd = "123.456.789-00";
8
9      Pessoa p2 = new Pessoa("Joao", 21, 'M', "234.567.890-00");
10 }

```

Repare que a primeira chamada do `new` irá executar o primeiro construtor, que não possui parâmetros. Já a segunda chamada, irá executar o outro construtor, que recebe os valores dos atributos como parâmetros e já os salva dentro do objeto, deixando o código principal mais enxuto. Não é possível criar dois construtores em uma mesma classe que possuam a mesma quantidade de parâmetros e que todos eles possuam os mesmos tipos. É preciso que seja possível diferenciar qual dos dois será invocado.

Para a classe **Conta**, vamos criar um construtor que recebe apenas o número da conta e o titular. O saldo e limite vão assumir valores padrões. Haverá também outro construtor que recebe também o limite. Este construtor começa chamando o construtor de dois parâmetros, e em seguida altera o limite de acordo com o valor recebido:

```

1  public class Conta {
2      String numero;
3      Pessoa titular;
4      double saldo, limite;
5
6      Conta(String n, Pessoa t) {
7          this.numero = n;
8          this.titular = t;
9          this.saldo = 0;
10         this.limite = 200;
11     }

```

```
12
13     Conta(String n, Pessoa t, double l) {
14         this(n, t); /* Executa o outro construtor desta classe, que
15                     recebe dois valores como parametros */
16         this.limite = l;
17     }
18
19     // Demais metodos da classe Conta
20 }
```

Após definir estes dois construtores, o código do nosso método principal do material passado pode ficar bem mais enxuto:

```
1 Pessoa p1 = new Pessoa("Maria", 18, 'F', "123.456.789-00");
2 Pessoa p2 = new Pessoa("Joao", 22, 'M', "234.567.890-00");
3
4 Conta c1 = new Conta("1234-5", p1);
5 Conta c2 = new Conta("2345-6", p2);
6
7 c1.extrato();
8 c2.extrato();
9
10 c1.sacar(150);
11 c1.transferir(100, c2);
12 c1.sacar(100);
13 c1.depositar(100);
14 c1.transferir(200, c2);
```

Assim, não precisamos mais atribuir os valores a esses parâmetros após instanciarmos os objetos p1 e c1. Ao invés disso, passamos os valores como parâmetro para o construtor de suas classes no momento em que criamos os objetos, deixando o código mais enxuto. Se a mesma rotina precisa ser executada toda vez que um objeto é instanciado, ela deve estar no construtor de sua classe. O **reuso de código** é uma das principais características da orientação a objetos.

A partir de agora, **não é mais possível** criar um novo objeto da classe *Conta* sem passar os parâmetros necessários para o construtor. Uma vez que um ou mais construtores são definidos, o construtor padrão não pode mais ser invocado. Assim, além do reuso de código, o construtor pode deixar o sistema mais **consistente**. Se o sistema não permite que haja uma *Conta* sem titular, então basta que os construtores desta classe exijam que este atributo seja inicializado.

De forma geral, os construtores permitem uma inicialização mais fácil e mais consistente de objetos, reuso de código das rotinas de inicialização e melhor legibilidade.

Os construtores também podem ser representados no diagrama de classe:

Conta
numero : String titular : Pessoa saldo : double limite : double
Conta(n : String, t : Pessoa) disponivel() : double extrato() depositar(valor : double) sacar(valor : double) : boolean transferir(valor : double, destino : Conta) : boolean chequeEspecial(juro : double)

3.3 E como funcionava até agora?

Até agora, as nossas classes não possuíam nenhum construtor. Mas, se o construtor da classe é sempre executado quando um novo objeto é criado, o que acontecia quando criávamos nossas classes até então?

Quando você não declara algum construtor na sua classe, o Java cria um para você. Esse construtor é o **construtor padrão**, que não recebe nenhum argumento e cujo corpo é vazio. A partir do momento em que você declara um construtor, o construtor padrão deixa de existir.

E se tudo estava funcionando até agora, para que utilizamos um construtor? A ideia é bem simples: se toda conta precisa de um usuário titular, como obrigar todos os objetos que forem criados a ter um valor desse tipo? Basta exigir que o construtor que receba essa informação como parâmetro. Assim, não é possível criar uma Conta que não esteja vinculada a uma Pessoa.

O construtor se resume a isto: dar possibilidades ou obrigar o usuário de uma classe a passar argumentos para o objeto durante o processo de criação do mesmo. Por exemplo: não podemos abrir um arquivo para leitura sem dizer qual é o nome do arquivo que desejamos ler. Portanto, nada mais natural que passar uma String representando o nome de um arquivo na hora de criar um objeto do tipo de leitura de arquivo, e que isso seja obrigatório. Além disso, se criamos um construtor com vários parâmetros, o usuário não precisará criar várias sentenças ao longo do programa para atribuir os valores de cada um desses parâmetros.

3.4 Exercícios

1. Qual a saída exata a ser impressa com a execução do trecho de código a seguir?

```

1 Pessoa p1 = new Pessoa("Maria", 18, 'F', "123.456.789-00");
2 Pessoa p2 = new Pessoa("Joao", 22, 'M', "234.567.890-00");
3
4 Conta c1 = new Conta("1234-5", p1);
5 Conta c2 = new Conta("2345-6", p2, 0);
6
7 c1.sacar(30);
8 c2.sacar(100);
9 c2.depositar(50);
10 c1.transferir(200, c2);

```

11

```
12 System.out.println(c2.saldo)
```

2. Implemente no seu projeto uma nova classe chamada **Data**. A classe deve conter três atributos: **dia**, **mês** e **ano** (todos números inteiros).
3. Implemente um construtor para a classe, que vai receber os três atributos como parâmetro.
4. Implemente e teste um método na classe **Data** chamado **imprimir** que imprima a data no formato D/M/A. Exemplo: "23/10/2018".
5. Altere a classe **Pessoa**, para que o objeto possua um atributo **dtNasc** com sua data de nascimento, ao invés da idade.
6. Faça um método na classe **Pessoa** que receba a data de hoje como parâmetro, e calcule e retorne a idade desta pessoa.

O capítulo 15 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 4

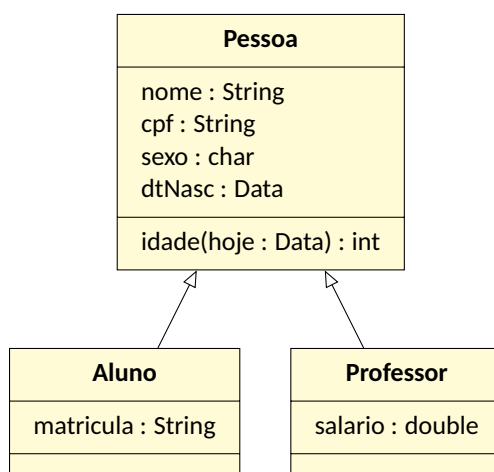
Herança

Esta aula apresenta um dos principais conceitos da programação orientada a objetos: a possibilidade de uma classe herdar todos os atributos e funcionalidades de outra classe.

4.1 Classes derivadas

Herança em POO permite que uma classe possa ser derivada de outra classe. Um Aluno, por exemplo, **é uma** Pessoa. Dessa forma, um objeto da classe Aluno deve **herdar** todos os atributos e métodos da classe Pessoa, mas também pode ter seus atributos próprios (ex: número de matrícula). Da mesma forma, um Professor também **é uma** Pessoa. Além dos atributos da classe Pessoa, um Professor também possui seus próprios atributos (ex: salário).

O diagrama a seguir representa a **superclasse** Pessoa (classe base) e as **subclasses** Aluno e Professor (classes derivadas):

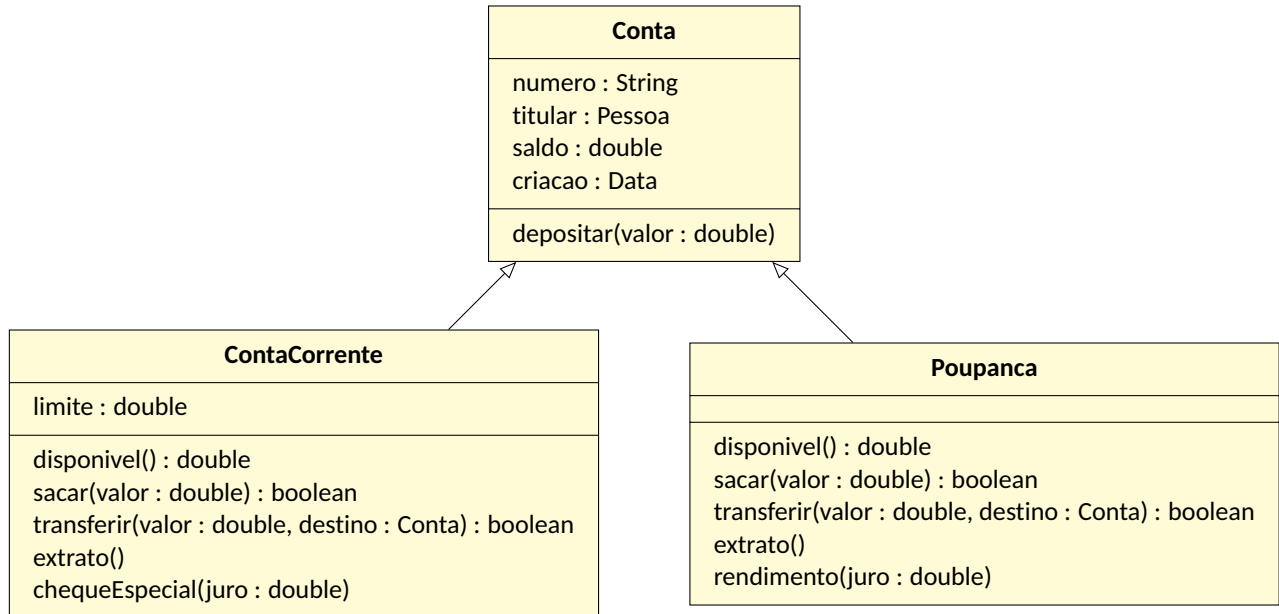


A classe Aluno pode ser vista como uma **extensão** da classe Pessoa. Em Java, isso é feito através da palavra **extends**:

```
1 class Aluno extends Pessoa {
2     String matricula;
3
4     /* Metodos... */
5 }
```


Cada vez que criarmos um objeto da classe **Aluno**, ele irá possuir todos os atributos de **Pessoa**, pois agora um **Aluno** **é uma** **Pessoa**. Uma grande vantagem da herança é o **reuso**. Quando usada corretamente, reduz o número de linhas de código, facilita a programação, a verificação de erros e futuras correções.

O sistema bancário implementado neste material precisa permitir a criação de dois tipos de contas: uma conta-corrente e uma poupança. Ambas possuem atributos em comum: número, titular, saldo e data de criação. Só a conta-corrente, entretanto, possui limite. Uma poupança nunca pode ter saldo negativo:



O método **depositar** é idêntico em ambas as contas, e por isso pode ser implementado na superclasse. Já o método **disponivel** é diferente, pois a classe **ContaCorrente** possui o atributo **limite** e a classe **Poupanca** não. Portanto, com o que já aprendemos sobre orientação a objetos até aqui, é preciso deixar que cada subclasse implemente sua própria versão deste método. Consequentemente, sem o método **disponivel** na classe **Conta**, também será necessário implementar os métodos **sacar**, **transferir** e **extrato** nas duas subclasses. Já o método **chequeEspecial** é exclusivo da conta-corrente, e a poupança possui um método chamado **rendimento**.

4.2 E o construtor?

Se uma superclasse não possui um construtor implementado, não é necessário implementá-lo nas subclasses. Neste caso, também será utilizado o construtor padrão (sem parâmetros) para as classes derivadas. Se o construtor da superclasse for implementado, torna-se obrigatória a implementação de um construtor em cada subclasse, que devem chamar o construtor da superclasse passando os parâmetros adequados. O construtor da superclasse é chamado através da palavra **super**.

Considere que o construtor da classe **Conta** recebe como parâmetros o número da conta, o titular e a data de criação da conta:

```

1 public class Conta {
2     String numero;
3     Pessoa titular;
4     double saldo;
5     Data criacao;
6

```

```

7  public Conta(String numero, Pessoa titular, Data c) {
8      this.numero = numero;
9      this.titular = titular;
10     this.saldo = 0;
11     this.criacao = c;
12 }
13
14 void depositar(double valor) {
15     this.saldo += valor;
16     System.out.println("Deposito na conta " + this.numero + "
17         realizado com sucesso. Novo saldo: " + this.saldo);
18 }

```

Isso significa que não é possível criar uma Conta no sistema sem informar o número, titular e data de criação. E como uma ContaCorrente **é uma** Conta, também é impossível criá-la sem informar ao menos esses três atributos. Em outras palavras, como o construtor da ContaCorrente deve chamar o construtor da classe Conta, ele deve receber pelo menos os parâmetros que o construtor da superclasse exige:

```

1  public class ContaCorrente extends Conta{
2      double limite;
3
4      ContaCorrente(String numero, Pessoa titular, Data c) {
5          super(numero, titular, c);
6          this.limite = 200;
7      }
8
9      double disponivel() {
10         return this.saldo + this.limite;
11     }
12
13     boolean sacar(double valor) {
14         if (valor <= this.disponivel()) {
15             this.saldo -= valor;
16             System.out.println("Saque na conta " + this.numero + "
17                 realizado com sucesso. Novo saldo: " + this.saldo);
18             return true;
19         }
20         else {
21             System.out.println("ERRO: Saque na conta " + this.numero + "
22                 nao foi realizado. Valor disponivel: " + this.disponivel());
23             ;
24             return false;
25         }
26     }
27
28     /* Demais metodos... */
29 }

```

Repare que os atributos **saldo** e **limite** são tratados da mesma forma (linhas 7 e 11), independente de terem sido herdados ou não. O mesmo vale para a classe Poupanca:

```
1 public class Poupanca extends Conta{
2
3     Poupanca(String numero, Pessoa titular, Data c) {
4         super(numero, titular, c);
5     }
6
7     double disponivel() {
8         return this.saldo;
9     }
10
11     // Demais metodos
12 }
```

Por fim, o método principal irá distinguir entre ContaCorrente e Poupanca no momento da criação, mas a chamada dos métodos independente deles terem sido implementados na superclasse ou na subclasse:

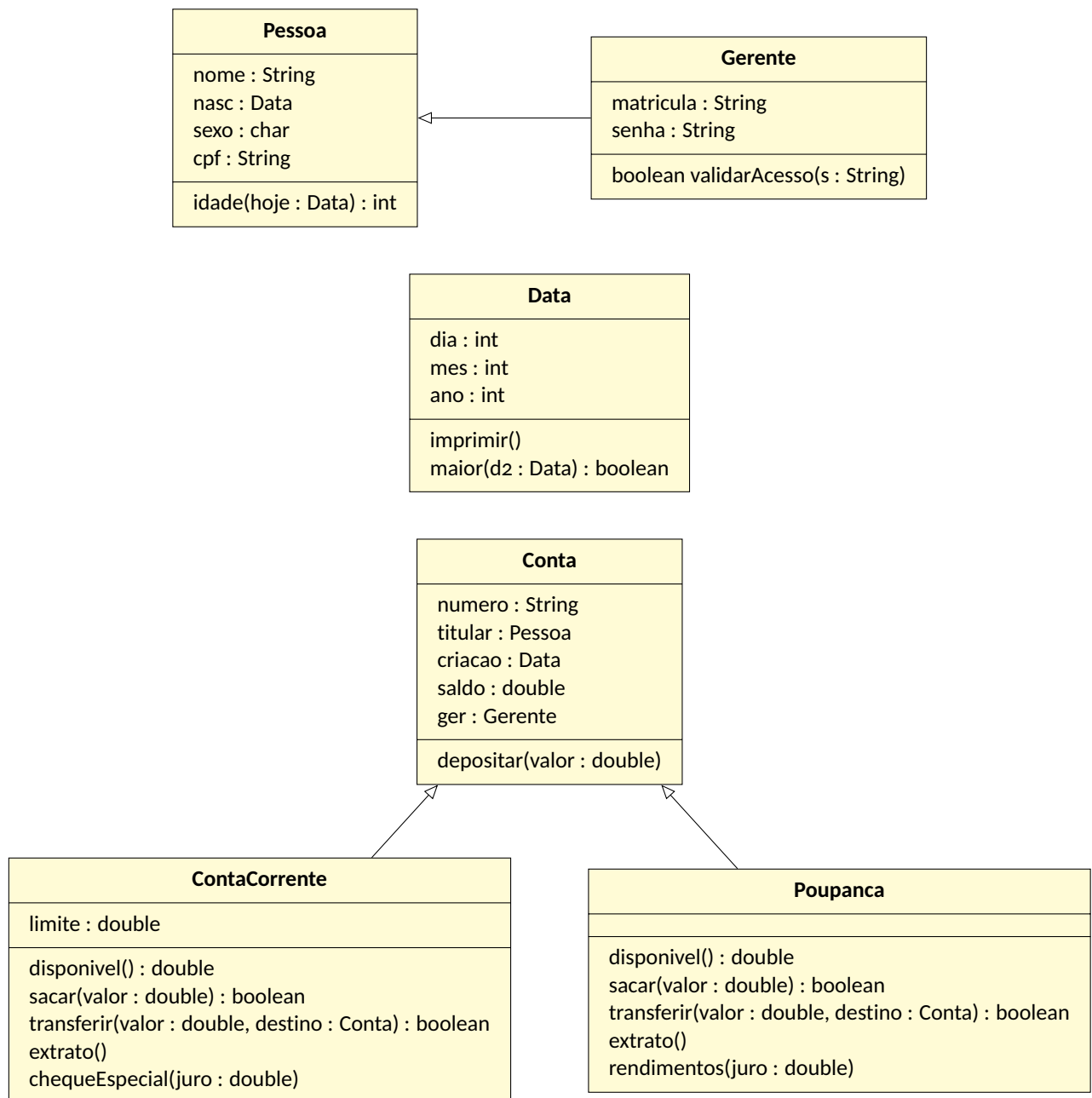
```
1 public static void main(String[] args) {
2
3     Pessoa p1 = new Pessoa("Maria", 18, 'F', "123.456.789-00");
4     Pessoa p2 = new Pessoa("Joao", 22, 'M', "234.567.890-00");
5
6     ContaCorrente c1 = new ContaCorrente("1234-5", p1, new Data(12, 3,
7         2019));
8     ContaCorrente c2 = new ContaCorrente("2345-6", p2, new Data(13, 3,
9         2019));
10
11     c1.sacar(150);
12     c1.sacar(100);
13     c1.depositar(100);
14 }
```

4.3 Exercícios

Utilize herança para criar três novas classes:

1. **ContaCorrente:** Subclasse de Conta. Apenas uma conta corrente possui limite, além do método chequeEspecial.
2. **Poupanca:** Subclasse de Conta. Apenas uma poupança possui rendimentos.
3. **Gerente:** Subclasse de Pessoa. Um gerente é uma pessoa e também possui um número de matrícula e uma senha.

Suas classes a partir de agora devem refletir **exatamente** o diagrama a seguir:



Os construtores das classes devem receber os seguintes parâmetros:

- Data: dia, mês e ano.
- Pessoa: nome, data de nascimento, sexo e cpf.
- Gerente: nome, data de nascimento, sexo, cpf, número de matrícula e senha.
- Conta: número, titular, gerente e data de criação. O saldo inicia zerado.
- ContaCorrente: número, titular, gerente e data de criação. O limite a princípio é 200.0.
- Poupanca: número, titular, gerente e data de criação.

A seguir, uma breve explicação sobre alguns métodos a serem implementados:

- O método **idade** da classe Pessoa deve receber a data de hoje como parâmetro e retornar a idade da pessoa.
- O método **validarAcesso** da classe Gerente deve receber uma senha como parâmetro e verificar se ela está correta.
- O método **maior** da classe Data deve receber outra data d2 como parâmetro de verificar se a data **this** é maior que a data d2 (ou seja, se ela aconteceu após a data d2).
- O método **rendimentos** da classe Poupanca recebe uma taxa de juros como parâmetro e adiciona essa porcentagem ao saldo da poupança.

Certifique-se de que os construtores e os métodos exibam mensagens na tela indicando ao usuário se foram executados com sucesso. Crie ao menos dois objetos de cada classe e teste todos os métodos de todas as classes no método principal.

O capítulo 16 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 5

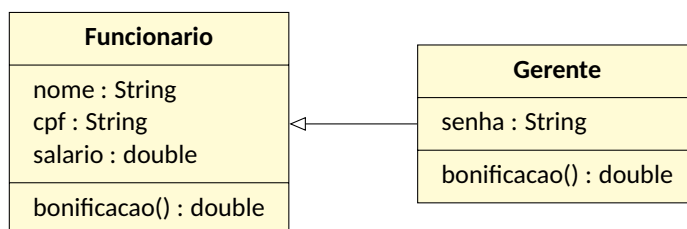
Reescrita, Sobrecarga e I/O

Esta aula introduz dois novos conceitos fundamentais da orientação a objetos: reescrita de métodos e sobrecarga de métodos. Além disso, mais detalhes sobre a entrada e saída padrão de dados no Java, para que os códigos deste materiais possam se tornar mais interativos.

5.1 Reescrita de Métodos

Reescrita de métodos permite reimplementar um método em uma subclasse que tenha o comportamento diferente do método na sua superclasse.

Suponha que uma empresa possui funcionários e gerentes. Um funcionário possui nome, cpf e salário. Um gerente possui todos os atributos e métodos de um funcionário, mas também possui uma senha. No final do ano, um funcionário comum recebe 10% de bonificação em seu salário, enquanto um gerente recebe 15%. Se a classe Gerente for uma subclasse de Funcionario, que já possui o método **bonificacao**, o mesmo pode ser **reescrito** na classe Gerente:



```
1 class Funcionario {
2     String nome, cpf;
3     double salario;
4
5     double bonificacao() {
6         return this.salario*0.1;
7     }
8 }

1 class Gerente extends Funcionario {
2     String senha;
3
4     double bonificacao() {
```

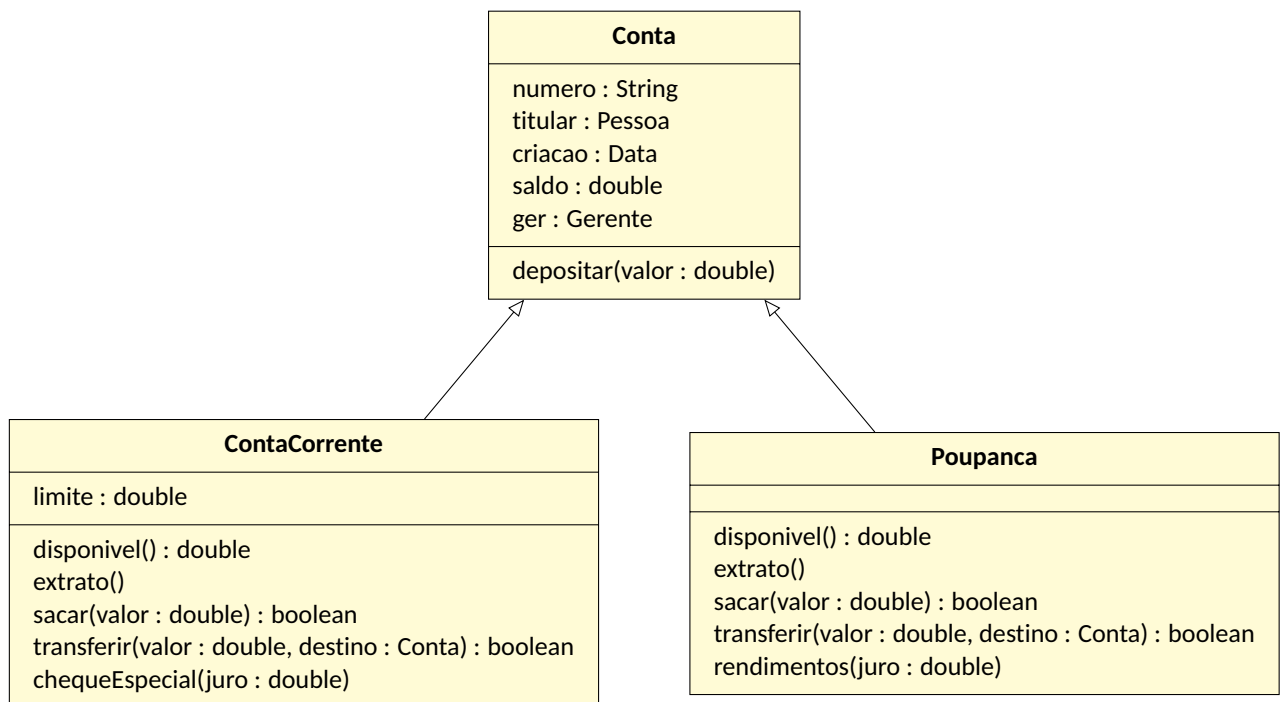
```

5         return this.salario*0.15;
6     }
7 }

```

Dessa forma, um objeto da classe Gerente deixa de utilizar o método `bonificacao` que foi implementado na superclasse, e passa a utilizar o método reescrito. Se for necessário que uma subclasse acesse especificamente o método implementado na superclasse que foi reescrito, usamos `super.bonificacao()`. A palavra `super` sempre se refere à superclasse herdada.

A reescrita pode ser usada no nosso projeto do Sistema Bancário para evitar que os métodos “sacar”, “depositar”, “extrato” e “transferir” sejam repetidos nas classes **ContaCorrente** e **Poupanca**. Atualmente, temos o diagrama a seguir:



Com a reescrita, esses quatro métodos ficarão na classe **Conta**, que deverá possuir um método “disponível” que utiliza apenas o saldo da conta, e não o limite (pois não há limite numa **Conta**):

```

1 public class Conta {
2     String numero;
3     Pessoa titular;
4     Gerente gerente;
5     double saldo;
6     Data criacao;
7
8     /* Construtor da classe Conta */
9
10    public double disponivel() {
11        return this.saldo;
12    }
13
14    /* Demais metodos: extrato, depositar, sacar, transferir */
15 }

```

Os objetos da classe **Poupanca** irão utilizar o método “disponível” herdado da superclasse **Conta**, enquanto os objetos da classe **ContaCorrente** irão utilizar o método “disponível” que será **reescrito**:

```

1 public class ContaCorrente extends Conta{
2     double limite;
3
4     ContaCorrente(String n, Pessoa t, Gerente g, Data c) {
5         super(n, t, g, c);
6         this.limite = 200.0;
7     }
8
9     double disponivel() {
10        return this.saldo + this.limite;
11    }
12
13    void extrato() {
14        System.out.println("EXTRATO DA CONTA-CORRENTE");
15        super.extrato();
16    }
17
18    void chequeEspecial ( double juro ) {
19        if ( this.saldo < 0) {
20            this.saldo = this.saldo + ( juro*this.saldo ) /100;
21        }
22    }
23 }

```

Repare que o método “extrato” também foi reescrito. Ele irá imprimir um cabeçalho próprio da conta corrente, e em seguida chamará o método extrato da superclasse, através do comando “super.extrato()”.

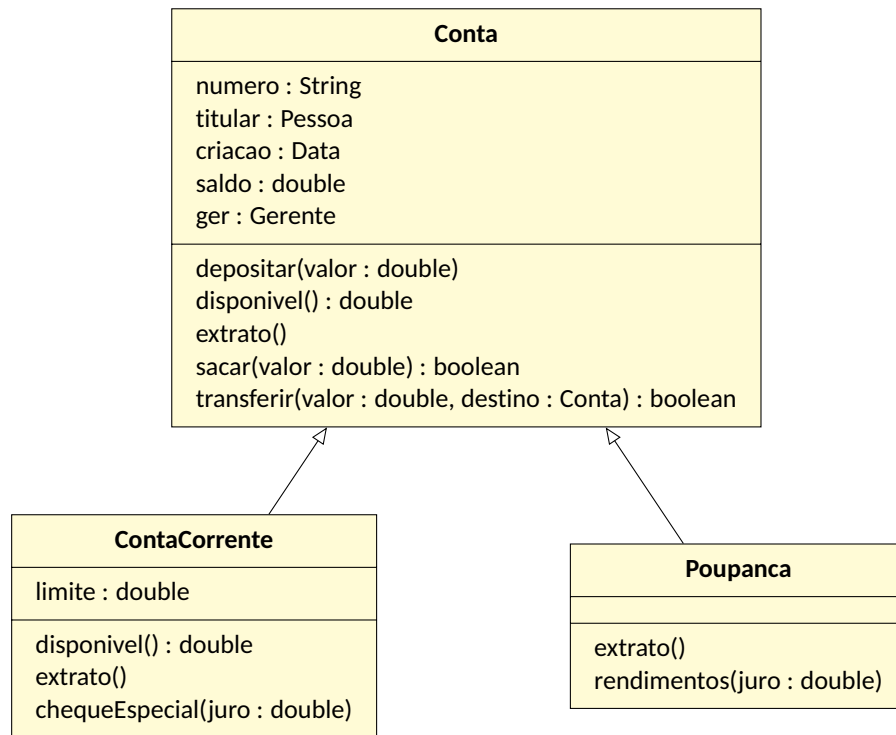
Na classe Poupanca, não é necessário reescrever o método disponível, apenas o “extrato”:

```

1 public class Poupanca extends Conta{
2     Poupanca(String n, Pessoa t, Gerente g, Data c) {
3         super(n, t, g, c);
4     }
5
6     void extrato() {
7         System.out.println("EXTRATO DA POUPANCA");
8         super.extrato();
9     }
10
11    void rendimentos ( double juro ) {
12        this.saldo = this.saldo + (juro*this.saldo ) /100;
13    }
14 }

```

Com isso, temos o novo diagrama a seguir:



5.2 Entrada de dados

A saída de dados padrão no Java (ou seja, a impressão de dados na tela) costuma ser feita através do método `System.out.println()`. Existem diversas formas de se fazer a **entrada de dados** da entrada padrão (ou seja, do teclado). A partir do Java 5.1, isso pode ser feito de forma mais fácil através da biblioteca **Scanner**. A primeira coisa a se fazer é importar a biblioteca no início do arquivo onde ela for utilizada:

```

1 import java.util.Scanner;
2
3 class Pessoa {
4     // ...
5 }

```

Agora, vamos criar um novo construtor para as classes **Data** e **Pessoa**, para que os valores de seus atributos sejam digitados pelo usuário, ao invés de serem recebidos como parâmetro:

```

1 import java.util.Scanner;
2
3 class Data {
4     int dia, mes, ano;
5
6     Data() {
7         Scanner s = new Scanner(System.in);
8
9         System.out.printf("Digite o dia: ");
10        this.dia = s.nextInt();
11
12        System.out.printf("Digite o mes: ");
13        this.mes = s.nextInt();

```

```

14
15     System.out.printf("Digite o ano: ");
16     this.ano = s.nextInt();
17 }
18
19 Data(int d, int m, int a) {
20     this.dia = d;
21     this.mes = m;
22     this.ano = a;
23 }
24 }

```

A instrução `Scanner s = new Scanner(System.in)` cria o objeto que chamamos de `s` e será responsável pela leitura dos dados. O método `s.nextInt()` faz a leitura de um número inteiro.

```

1  import java.util.Scanner;
2
3  class Pessoa {
4      String nome, cpf
5      Data nasc;
6      char sexo;
7
8      Pessoa() {
9          Scanner s = new Scanner(System.in);
10
11          System.out.printf("Digite o nome: ");
12          this.nome = s.nextLine();
13
14          System.out.printf("Digite o cpf: ");
15          this.cpf = s.nextLine();
16
17          System.out.printf("Data de nascimento: ");
18          this.nasc = new Data();
19
20          System.out.printf("Informe o sexo: ");
21          this.sexo = s.nextLine().charAt(0);
22      }
23
24      Pessoa(String n, Data d, char s, String c) {
25          this.nome = n;
26          this.nasc = d;
27          this.sexo = s;
28          this.cpf = c;
29      }
30 }

```

O método `s.next()` faz a leitura de uma única palavra no formato `String`. Se você digitar “Maria da Silva”, por exemplo, ele lerá apenas “Maria”. Já o método `s.nextLine()` faz a leitura de uma linha inteira (ou seja, podemos utilizá-la para a leitura de um nome completo).

Não existe um método para a leitura de um único caractere. Por isso, para a leitura do sexo do usuário, fizemos a leitura de uma linha inteira e pegamos o primeiro caractere da linha (`s.nextLine().charAt`

(0)).

5.3 Sobrecarga de métodos

Foi possível criar dois construtores para as classes **Data** e **Pessoa** (um que faz a leitura dos atributos pelo teclado, e outro que recebe seus valores como parâmetro) porque a orientação a objetos permite a **sobrecarga de construtores**.

Agora, se você criar um objeto da classe **Pessoa** passando parâmetros pra ele, será utilizado o construtor com parâmetros. Ex:

```
1 Pessoa p1 = new Pessoa("Hilario", new Data(4,12,1981), 'M', "
    123.456.789-00");
```

Se você criar um objeto da classe **Pessoa** sem passar parâmetros pra ele, será utilizado o construtor sem parâmetros. Neste caso, os atributos terão que ser informados pelo usuário no momento em que o construtor for executado:

```
1 Pessoa p3 = new Pessoa();
```

Repare que o construtor sem parâmetros da classe **Pessoa** está chamando o construtor sem parâmetros da classe **Data** (afinal, o usuário terá que informar o dia, mês e ano de nascimento, já que eles não foram passados por parâmetro).

Da mesma forma, é possível implementar a **sobrecarga de métodos**, permitindo que existam vários métodos com o mesmo nome em uma mesma classe. Esses métodos homônimos se diferem pela **quantidade** e/ou **tipo** dos parâmetros que cada um recebe. Em outras palavras, eles devem possuir uma assinatura **diferente**.

A classe **Gerente**, por exemplo, possui um método chamado “validarAcesso”, que recebe uma senha como parâmetro e verifica se ela é a mesma senha cadastrada para este gerente. Podemos criar um novo método homônimo que faz a leitura da senha pelo teclado, caso ela não seja passada como parâmetro:

```
1 import java.util.Scanner;
2
3 public class Gerente extends Pessoa {
4     String mat;
5     String senha;
6
7     Gerente(String nome, String cpf, Data d, char s, String mat,
8         String p) {
9         super(nome, cpf, d, s);
10        this.mat = mat;
11        this.senha = p;
12    }
13
14    boolean validarAcesso(String pwd) {
15        return pwd.equals(this.senha);
16    }
17
18    boolean validarAcesso() {
19        Scanner s = new Scanner(System.in);
```

```

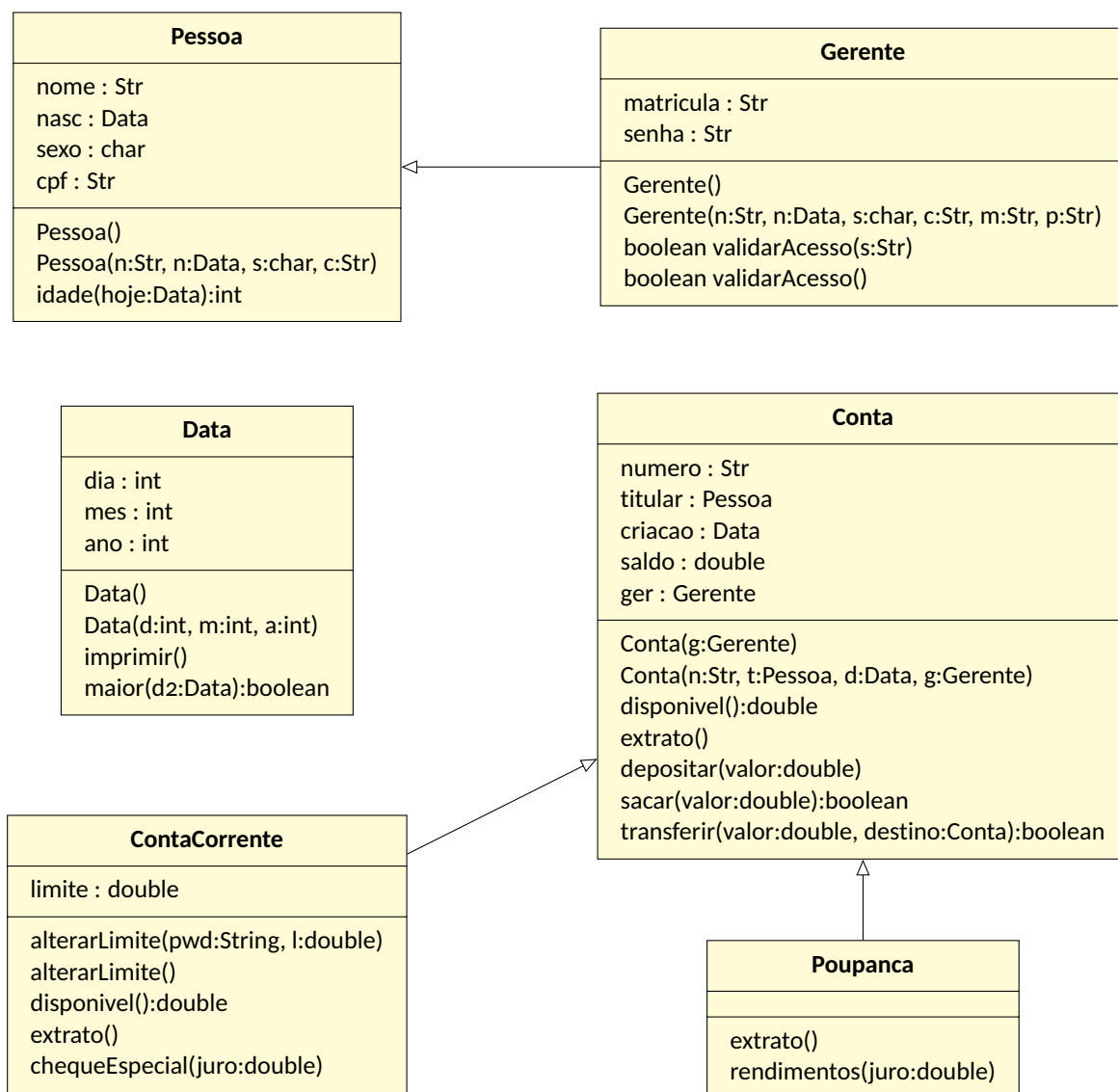
19
20     System.out.printf("Digite a senha: ");
21     String pwd = s.nextLine();
22
23     return this.validarAcesso(pwd);
24 }
25 }

```

Repare que, após fazer a leitura da senha, o segundo método “validarAcesso” fez a chamada do método homônimo que recebe a String como parâmetro. Caso se opte por alterar a lógica da validação no futuro, isso só precisará ser feito uma vez no código do programa.

5.4 Exercícios

Incorpore os conceitos de reescrita de métodos, sobrecarga de métodos e entrada de dados ao projeto do Sistema Bancário. O novo sistema deve refletir o diagrama a seguir:



Observações:

1. Os construtores sem parâmetros das classes Pessoa e Data devem solicitar que o usuário informe **todos** os seus atributos.
2. O construtor sem parâmetros da classe Gerente deve solicitar que o usuário informe todos os seus atributos, exceto a senha (que deve assumir um valor padrão – exemplo: “123456”).
3. Na classe Gerente, o método **validarAcesso()** precisa comparar duas Strings. Em Java, para verificar se duas Strings `s1` e `s2` são iguais, usamos `s1.equals(s2)`. Não é correto utilizar `s1 == s2`. O método **validaraAcesso()** da classe Gerente que não recebe a senha como parâmetro deve solicitar que o usuário digite-a.
4. O construtor da classe Conta que recebe apenas o Gerente como parâmetro deve solicitar que o usuário informe os outros atributos e criar um novo objeto para armazenar o titular da conta. Apenas o saldo não deve ser digitado, pois deve começar sempre zerado. Não é possível criar uma Conta sem um Gerente associado a ela.
5. Com a reescrita, os métodos **disponivel()**, **extrato()**, **sacar()** e **transferir()** ficam apenas na superclasse Conta, evitando códigos duplicados. O que fará com que os comportamentos deles sejam diferentes numa ContaCorrente é o fato de reescrevermos o método **disponivel()** nesta subclasse.
6. O método **alterarLimite()** de uma ContaCorrente só deve executar a ação após validar o acesso do gerente, utilizando a senha recebida como parâmetro. O método homônimo que não recebe parâmetros deve fazer a leitura tanto da senha como do novo limite.
7. Repare que o método **extrato()** deverá ser reescrito tanto na classe ContaCorrente quanto na Poupanca. A ideia é a seguinte: o método da suuperclasse imprime as informações necessárias sobre titular, número da conta e saldo disponível. Os métodos reescrito nas subclasses imprimem um cabeçalho indicando o tipo de conta (se é um extrato de conta corrente ou de uma poupança), e em seguida executa o método da superclasse para imprimir as demais informações. Note que **não devemos repetir o código** que imprime titular, número e saldo da conta. Para que um método reescrito execute o método homônimo da superclasse, utilizamos a palavra reservada **super**. Por exemplo, para que o método **extrato()** na ContaCorrente execute o método homônimo que foi reescrito, utilizamos o comando **super.extrato()**. Com isso, os extratos serão diferenciados da seguinte forma:

```
*** EXTRATO DE CONTA-CORRENTE ***
Titular: Joao da Silva (CPF: 123.456.789-00)
Número da Conta: 1234-5
Saldo disponível para saque: R$ 250.00

*** EXTRATO DE POUPANCA ***
Titular: Maria Joaquina (CPF: 234.567.890-00)
Número da Conta: 2345-6
Saldo disponível para saque: R$ 50.00
```

8. Por fim, seu método principal deve criar quatro objetos da classe Pessoa, dois da classe Gerente, dois da classe ContaCorrente e dois da classe Poupanca. Execute saques, depósitos, transferências e extratos em todas as contas, e verifique se o resultado está correto.

Repare ainda que os construtores de ContaCorrete e Poupanca foram omitidos, mas você deve implementá-los.

O capítulo 17 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 6

Encapsulamento

O sistema de informações de um banco possui um ponto crítico que precisa ser observado com atenção: controlar os saques e transferências, para que não seja possível que a conta possua um saldo abaixo do limite disponível. Mesmo que o método **sacar()** faça essa verificação, temos que garantir que o usuário da classe Conta irá sempre utilizar esse método, e nunca alterar diretamente o saldo da conta. Por exemplo:

```
1 Gerente g = new Gerente();
2 ContaCorrente c1 = new ContaCorrente(g);
3 // Saldo: R$ 0,00
4 // Limite: R$ 200,00
5
6 c1.sacar(150);
7 // Saldo: R$ -150,00
8 // Disponível: R$ 50,00
9
10 c1.sacar(100);
11 // Erro!
12
13 c1.saldo = -400;
14 // Saldo: R$ -400,00
15 // Menor que o limite!!
```

O sistema ficou inconsistente pois o saldo foi modificado na linha 11 sem que o método **sacar()** fosse utilizado, e é ele que verifica corretamente se há saldo disponível para saque. Precisamos **obrigar** o usuário da classe Conta a utilizar o método **sacar()** sempre que ele quiser diminuir o saldo em conta e **não permitir o acesso direto** ao atributo **saldo**.

Isso pode ser feito através do **encapsulamento** dos dados, que será detalhado nesta aula.

6.1 Modificadores de Acesso

Em Java, podemos restringir o acesso a um atributo através de um **modificador de acesso** chamado **private**:

```
1 class Conta {
2     private double saldo;
3     // Demais atributos...
4 }
```

```

5      // Construtores...
6
7      // Metodos...
8  }

```

A partir de agora, o atributo saldo só poderá ser alterado dentro dos métodos da classe Conta. Com isso, o código anterior que alterava o saldo manualmente nem iria compilar.

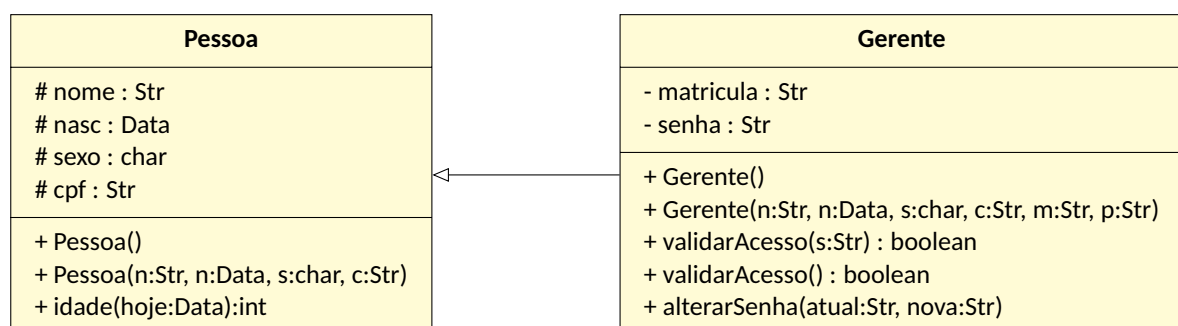
Em orientação a objetos, os atributos quase sempre serão privados, pois as classes são responsáveis por controlar seus atributos, julgando se aquele novo valor é válido ou não. Esta validação não deve ser controlada por quem está usando a classe e sim por ela mesma, centralizando essa responsabilidade e facilitando futuras mudanças no sistema. Muitas vezes, o usuário da classe não deveria saber nem quais atributos existem na classe, conhecendo apenas as funcionalidades que a classe disponibiliza (ex: sacar, depositar, tirar um extrato, transferir, etc). E o usuário do sistema que invocar o método sacar não precisa saber nem a forma com a qual o sistema verifica o valor disponível. Ele precisa saber **apenas** se a operação foi ou não realizada com sucesso. Ou seja, é preciso saber **O QUE** o método faz, e não exatamente **COMO** ele faz. Se em algum momento for preciso alterar essa verificação, poderemos ter certeza de que isso só precisará ser feito dentro do método da classe, pois agora teremos CERTEZA que em nenhuma outra parte do código o saldo está sendo alterados diretamente.

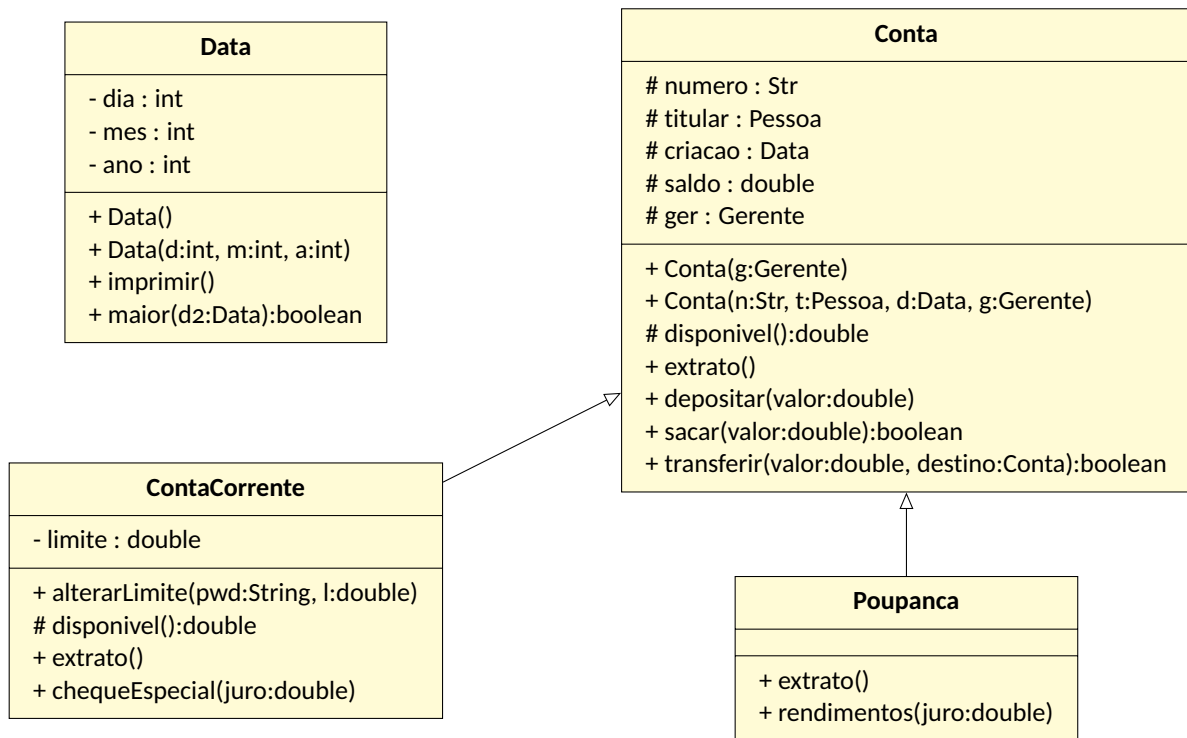
Em geral, utilizamos três tipos de modificadores de acesso:

- **public**: indica que uma classe, variável ou método pode ser usado mesmo fora do pacote em que foram definidos. No diagrama de classes, é representado por um +.
- **private**: indica que uma variável ou método só pode ser acessado dentro da classe em que foi declarado. No diagrama, é representado por um -.
- **protected**: indica que uma variável ou método só pode ser acessado na própria classe ou em suas subclasses. No diagrama, é representado por um #.

Se não houver modificador de acesso, o Java utiliza um quarto modificador chamado *friendly*, no qual só é possível acessar uma classe, atributo ou método dentro do mesmo pacote em que foram definidos (mais restrito que o *public*). Geralmente, os métodos são públicos e os atributos são privados ou protegidos (caso tenham sido definidos em uma superclasse). Com isso, estamos realizando o **encapsulamento** do sistema.

Não entendeu? É simples: você não precisa, por exemplo, conhecer os detalhes dos circuitos internos do seu computador para utilizá-lo. A carcaça do computador encapsula esses detalhes, provendo a você uma interface mais amigável (monitor, teclado, mouse etc) para utilizá-lo. Exemplo:





6.2 Métodos getters e setters

Caso seja útil que o usuário acesse ou modifique o valor de um atributo da classe, o ideal é criarmos métodos para isso ao invés de deixá-los públicos. Ex:

```

1 public class ContaCorrente extends Conta{
2     private double limite;
3
4     public double getLimite() {
5         return limite;
6     }
7
8     public void setLimite(String passwd, double l) {
9         if (this.gerente.validarAcesso(passwd)) {
10             this.limite = l;
11         }
12     }
13
14     // outros metodos...
15 }
  
```

6.3 Visibilidade dos Modificadores de Acesso

A tabela a seguir resume as situações em que os atributos de uma classe podem ser acessados para cada modificador de acesso:

	public	protected	friendly	private
mesma classe	SIM	SIM	SIM	SIM
subclasse do mesmo pacote	SIM	SIM	SIM	NÃO
outras classes no mesmo pacote	SIM	SIM*	SIM	NÃO
subclasse em outro pacote	SIM	SIM	NÃO	NÃO
outras classes em outro pacote	SIM	NÃO	NÃO	NÃO

* Em Java, membros **protected** também são acessíveis por classes do mesmo pacote.

6.4 Exercícios

Altere pela última vez as classes Data, Pessoa, Gerente, Conta, ContaCorrente e Poupanca, encapsulando seus dados de acordo com o diagrama de classes mostrado no final da seção 6.1.

Note que alguns erros podem surgir no código após encapsular os dados. O método **idade(Data hoje)** da classe Pessoa, por exemplo, recebe a data de hoje como parâmetro e tenta acessar o atributo **hoje.dia**. Isso não será mais possível, já que os atributos de Data agora serão privados. Por isso, é necessário inserir um método para permitir na classe Data que retorne este valor:

```
1 public int getDia() {  
2     return dia;  
3 }
```

Assim, quando o método **idade(Data hoje)** precisar acessar o dia de hoje, ele o fará através do comando **hoje.getDia()** ao invés de **hoje.dia**.

O capítulo 18 da Parte II contém um guia para auxiliar na realização deste exercício.

Aula 7

Array e ArrayList

Toda linguagem de programação fornece ao menos alguma estrutura de dados para armazenar um conjunto de elementos. Apesar de não serem conceitos específicos da orientação a objetos, vamos abordar aqui duas formas de agrupar elementos em Java, para que o sistema bancário possa lidar com um conjunto maior de dados.

7.1 Arrays

Um **array** possui um tamanho fixo e pode armazenar apenas elementos de um mesmo tipo. Uma vez criado, seu tamanho não muda mais:

```
1 public static void main(String[] args) {
2
3     int[] arr = new int[3];
4     arr[0] = 4;
5     arr[1] = 7;
6     arr[2] = 9;
7
8     System.out.println(arr[0] + arr[1] + arr[2]);
9 }
```

No exemplo acima, foi criado um array de inteiros chamado **arr**, com espaço para armazenar três números inteiros. Nas linhas seguintes, é armazenado um valor em cada posição desse array. O array é indexado a partir da posição zero.

Podemos também criar um array da forma a seguir:

```
1 public void imprimirExtenso() {
2     String[] meses = {"janeiro", "fevereiro", "marco", "abril", "
3         maio", "junho", "julho", "agosto", "setembro", "outubro", "
4         novembro", "dezembro"};
5     System.out.println(this.dia + " de " + meses[this.mes-1] + " de
6         " + this.ano);
7 }
```

No exemplo acima, o array armazena 12 Strings com os meses do ano, para que uma data possa ser impressa por extenso. Uma vez criado, não é possível alterar a quantidade de elementos armazenados em um array.

7.2 ArrayLists

Array é uma funcionalidade fornecida pelo Java. Já o **ArrayList** precisa ser importado explicitamente:

```
1 import java.util.ArrayList;
```

Ao contrário do array, o tamanho de um ArrayList pode crescer:

```
1 import java.util.ArrayList;
2
3 public static void main(String[] args) {
4     ArrayList<Pessoa> p = new ArrayList<>();
5
6     for (int i = 0; i < 4; i++) {
7         p.add( new Pessoa() );
8     }
9
10    System.out.println(p.get(0).getNome());
11 }
```

A linha 4 mostra como o ArrayList é criado (inicialmente sem nenhum elemento). Note que é preciso informar o tipo dos dados que serão armazenados. O método **add** na linha 7 adiciona novos elementos no final do ArrayList.

ArrayLists ocupam mais espaço na memória que arrays. Por isso, devem ser usados apenas quando realmente necessários. Por não ser uma funcionalidade nativa do Java, não é possível acessar um elemento do ArrayList através de `p[0]`. Ao invés disso, utilizamos o método `p.get(0)`.

7.3 Arrays Vs. ArrayLists

Para criar um novo array com 10 objetos da classe ContaCorrente:

```
1 ContaCorrente[] arr = new ContaCorrente[10];
```

Note que, por enquanto, foi apenas reservado o espaço suficiente para que 10 contas correntes sejam armazenadas. Os objetos ainda precisam ser explicitamente instanciados.

Para criar um ArrayList de contas correntes:

```
1 ArrayList<ContaCorrente> listaContas = new ArrayList<>();
```

Para adicionar itens no array, temos que especificar a posição em que o elemento será armazenado:

```
1 arr[0] = new ContaCorrente();
2 arr[1] = new ContaCorrente();
3 // ...
4 arr[2] = new ContaCorrente();
```

Já no ArrayList, novos elementos sempre são adicionado no final.

```
1 listaContas.add( new ContaCorrente() );
2 listaContas.add( new ContaCorrente() );
3 // ...
4 listaContas.add( new ContaCorrente() );
```

Para acessar uma posição específica do array:

```
1 arr[0].extrato();
```

O mesmo é feito no ArrayList através do método **get**, que recebe como parâmetro a posição desejada:

```
1 listaContas.get(0).extrato();
```

Para percorrer todos os elementos de um array:

```
1 for(int i=0; i<arr.length; i++) {  
2     arr[i].extrato();  
3 }
```

Os elementos de um ArrayList podem ser percorridos de duas formas:

```
1 for(int i=0; i<listaContas.size(); i++) {  
2     listaContas.get(i).extrato();  
3 }  
4  
5 for(ContaCorrente c : listaContas) {  
6     c.extrato();  
7 }
```

Cuidado: Às vezes, quando alocamos um array, sabemos apenas o número o máximo de elementos que irão ocupá-lo. Portanto, não necessariamente ele estará todo preenchido no momento em que ele for percorrido. O atributo `length` retorna sempre a quantidade de elementos que o array possui, e não quantos foram realmente preenchidos. O aconselhável, nesse caso, é mantermos uma variável auxiliar que vai acumulando a quantidade de itens que foram realmente adicionados ao array.

A partir do Java 5.0, também é possível percorrer um array da mesma forma que o ArrayList:

```
1 for(ContaCorrente c : arr) {  
2     c.extrato();  
3 }
```

7.4 Exercícios e Métodos Estáticos

Implemente um menu para seu sistema bancário. Ao entrar, o usuário terá as seguintes opções:

- 1) Cadastro
 - 2) Movimentações financeiras

Ao escolher a opção 1, o usuário terá as seguintes opções:

- a) Adicionar pessoa
 - b) Adicionar gerente
 - c) Criar conta corrente
 - d) Criar poupança

Haverá 4 ArrayLists na memória: um para as pessoas, um para os gerentes, um para as contas correntes e outro para as poupanças.

Por fim, ao escolher a opção 2, o usuário terá as seguintes opções:

- a) Verificar extrato
- b) Depositar
- c) Sacar
- d) Transferir

Crie códigos para implementar o menu. Será necessário criar métodos auxiliares para impressões do menu na tela. Em geral, isso é feito através de métodos estáticos.

Até agora, todos os métodos que criamos precisam ser invocados através de algum objeto instanciado previamente. Agora, queremos apenas um método auxiliar contendo uma rotina de entradas e saídas. Para isso, existe a palavra reservada `static`. Métodos estáticos são representados no diagrama de classes com um sublinhado.

Repare que já utilizamos a palavra `static` no método `main`. Isso significa que o método pode ser executado mesmo que nenhum objeto de nenhuma classe tenha sido criado. Exemplo:

```
1 public class Banco {
2     public static int menuInicial() {
3         Scanner s = new Scanner(System.in);
4
5         System.out.println("Bem vindo! Escolha uma opcao.\n");
6         System.out.println("1) Cadastro");
7         System.out.println("2) Movimentacao Financeira");
8         System.out.println("0) Sair");
9
10        return s.nextInt();
11    }
```

Neste exemplo, ele está definido na mesma classe do método principal. Portanto, ele pode ser chamado no `main` sem precisar ser associado a algum objeto, como fazíamos até agora:

```
1 int op = menuInicial();
2 while (op != 0) {
3     if (op == 1) {
4         menuCadastro(p, g, c);
5     }
6     if (op == 2) {
7         menuMovimentacoes(p, g, c);
8     }
9     op = menuInicial();
10 }
11
12 System.out.println("\nBye! :-)");
```

Caso o método estático esteja em outra classe do mesmo pacote, devemos chamá-lo indicando o nome da classe ao invés de um objeto. Se ele estiver, por exemplo, em uma classe chamada **Entrada** apenas com rotinas de entrada e saída de dados, podemos chamar o método no **main** da seguinte forma:

```
1 int op = Entrada.menuInicial();
```

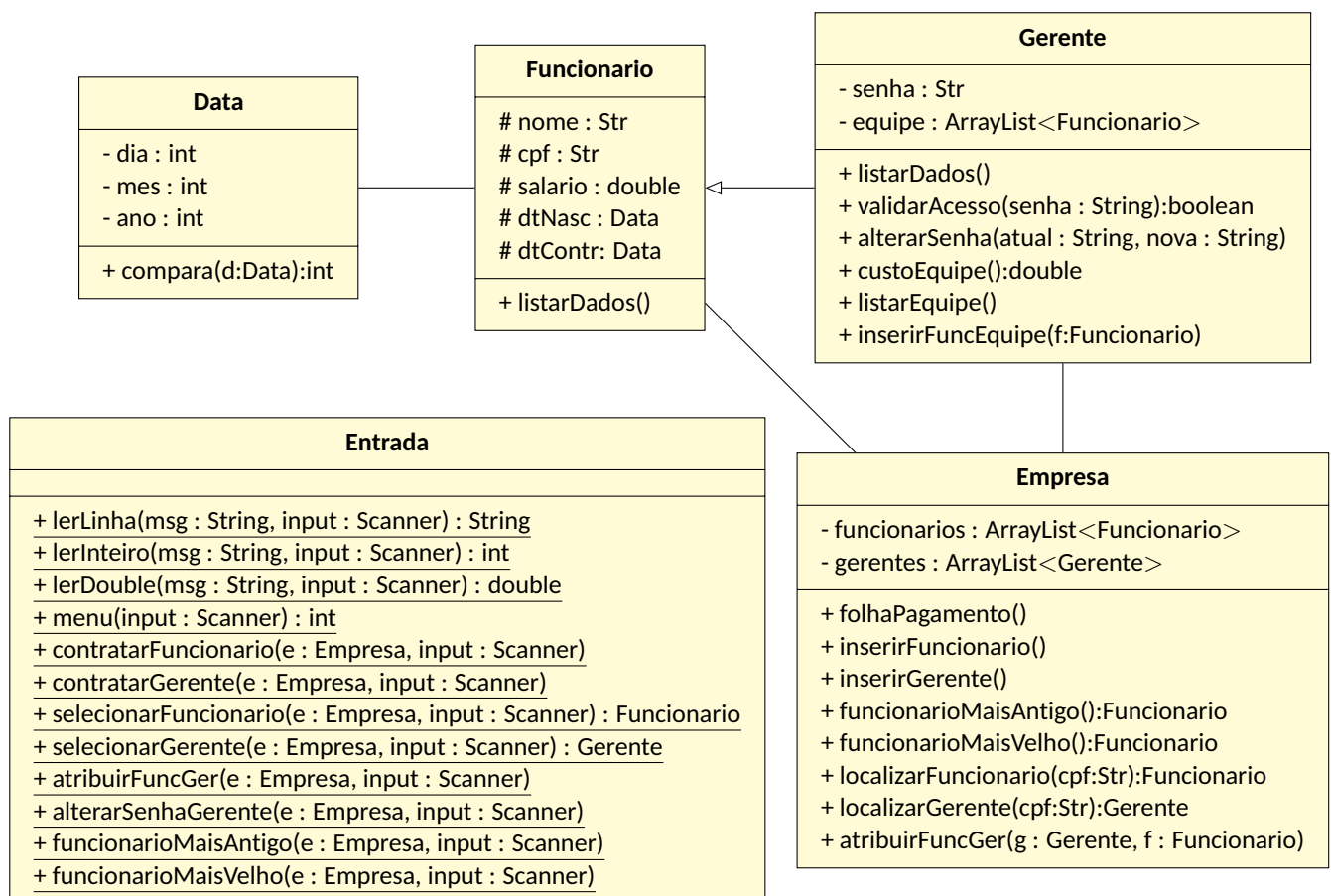
O capítulo 19 da Parte II contém um extenso guia para auxiliar na realização deste exercício.

Aula 8

Exercício de Revisão

8.1 Empresa de RH

O chefe da empresa confiou em você para desenvolver o novo sistema para gerenciar a parte de Recursos Humanos. O sistema permite cadastrar funcionários, gerentes e equipes. A figura abaixo mostra o projeto das classes do sistema:



Como podemos ver no diagrama, a classe **Data** possui 3 inteiros (dia, mês e ano). O método **compara** recebe uma outra data **d2** como parâmetro e retorna:

- -1, caso esta data seja anterior a Data **d2**

- 1, caso esta data seja maior que Data d2
- 0, caso sejam a mesma data

A classe `Funcionario` possui 5 atributos: nome, cpf, salario, data de nascimento e data de contratação. O método **listarDados** deve imprimir nome, cpf e salário do funcionário.

A classe `Gerente` herda todos os atributos e métodos que um `Funcionario` possui. Além disso, possui outros 2 atributos:

1. Senha (usada para o gerente logar no sistema)
2. Equipe (ArrayList com os funcionários da empresa que são liderados por este gerente)

A classe `Gerente` ainda possui métodos para:

- Validar o acesso, verificando se a senha está correta
- Alterar a senha, que deve validar a senha atual antes de modificá-la para a nova senha
- Calcular o custo total da equipe, incluindo o salário do gerente e de todos os funcionários da equipe
- Listar o nome de todos os membros da equipe, incluindo o gerente
- Inserir um funcionário à sua equipe (adicionando-o no array de funcionários do gerente).
- Listar dados, informando que ele é um gerente

A classe `Empresa` possui um ArrayList de `Funcionarios` e um de `Gerentes`. Os métodos desta classe devem:

1. Mostrar a folha de pagamento mensal, que lista o nome e salário de todos os funcionários da empresa. No caso dos gerentes, deve-se também imprimir o custo total da equipe gerenciada por ele. Por fim, o método também imprime o custo mensal da empresa, somando o salário de todos os funcionários e gerentes.
2. Contratar um funcionário, adicionando-o ao array de funcionários.
3. Contratar um gerente, adicionando-o ao array de funcionários.
4. Retornar o funcionário mais antigo (tempo de casa) da empresa.
5. Retornar o funcionário mais velho (idade) da empresa.
6. Dado um CPF, localizar o funcionário que possui aquele CPF.
7. Dado um CPF, localizar o gerente que possui aquele CPF.
8. Atribuir um certo funcionário à equipe de um certo gerente.

Por fim, a classe `Entrada` vai concentrar a leitura dos dados. Há métodos para leitura de Strings, inteiros e doubles. Além disso, há métodos estáticos para:

1. Exibir o menu do sistema, e retornar a opção digitada pelo usuário.

2. Contratar um funcionário, que vai fazer a leitura de todos os dados e depois inseri-lo na empresa.
3. Contratar um gerente, que vai fazer a leitura de todos os dados e depois inseri-lo na empresa.
4. Selecionar um funcionário, que exibe os funcionários cadastrados e retorna o funcionário escolhido pelo usuário.
5. Selecionar um gerente, que exibe os gerentes cadastrados e retorna o gerente escolhido pelo usuário.
6. Usuário seleciona um funcionário e um gerente, e faz a inserção do funcionário na equipe do gerente.
7. Selecionar um gerente e alterar sua senha com os dados digitados pelo usuário.
8. Imprimir os dados do funcionário mais antigo (tempo de casa) da empresa.
9. Imprimir os dados do funcionário mais velho (idade) da empresa.

Lembre-se de criar os construtores para todas as classes, encapsulando-os devidamente. Os modificadores de acesso devem ser idênticos aos do diagrama de classes. Seu programa principal deverá criar um objeto da classe Empresa.

8.2 Testando o sistema

Para testar o sistema, você deve:

- Contratar 5 funcionários (invente os dados de cada um deles)
- Contratar 1 gerente
- Atribuir cada funcionário a este gerente
- Contratar outros 5 funcionários
- Contratar 1 novo gerente
- Atribuir os novos funcionários a este novo gerente
- Alterar a senha de um dos gerentes
- Imprimir os dados do funcionário mais antigo da empresa
- Imprimir os dados do funcionário mais velho da empresa
- Imprimir a folha de pagamento mensal.

A correção do exercício está no Capítulo 20.

Aula 9

Tratamento de Erros e Exceções

Quando o usuário executa um programa que nós criamos, é possível que ocorram erros imprevistos. Isso pode ocorrer, por exemplo, se ele escolher o nome de um arquivo de entrada a ser lido pelo programa e o arquivo não existir. Se o programa precisa de um arquivo de entrada, o normal seria que ele escolhesse um arquivo que existisse. Portanto, o erro seria uma **exceção** ao comportamento normal que o programa deveria executar.

O uso de exceções permite separar a detecção da ocorrência de uma situação excepcional e o seu tratamento. As exceções podem ser provenientes de erros na lógica do programa ou por acesso a dispositivos ou arquivos externos inválidos. Exemplos de exceções comuns:

- Tentar abrir um arquivo que não existe.
- Tentar escrever algo em um arquivo sobre o qual não se tem permissão de escrita.
- Tentar fazer consulta a um banco de dados que não está disponível.
- Tentar conectar em servidor inexistente.
- Tentar manipular um objeto que não foi instanciado (ou seja, declaramos o objeto mas não usamos o `new` para criá-lo).
- Dividir um número por zero.
- Tentar manipular um tipo de dado como se fosse outro.

9.1 Tratando uma exceção com try-catch

Se detectamos um trecho de código no qual potencialmente pode ocorrer uma ou mais exceções, podemos usar os comandos `try` e `catch` para separar o código normal a ser executado, e o código a ser executado quando a exceção acontecer. A sintaxe é a seguinte:

```
1 try {  
2     // código normal a ser executado  
3 }  
4 catch ( <excecao1> ) {  
5     // código de tratamento do primeiro tipo de erro  
6 }
```

```

7  catch ( <excecao2> ) {
8      // código de tratamento do segundo tipo de erro
9  }
10 catch ( <excecao3> ) {
11     // código de tratamento do terceiro tipo de erro
12 }

```

Como exemplo, vamos supor que foi pedido que o usuário digitasse um número. Se o usuário digitar apenas algarismos (ex: 1024), a leitura irá funcionar. Entretanto, se o usuário digitar algum caractere inválido (ex: 1024, com a letra 'o' ao invés do algarismo zero), o programa iria abortar a execução. Para que o programa não aborte, trataremos a exceção:

```

1  Scanner s = new Scanner(System.in);
2
3  try {
4      System.out.println("Digite um numero:");
5      int x = s.nextInt();
6  }
7  catch (InputMismatchException e)
8  {
9      System.out.println("Erro: numero invalido.");
10 }

```

No trecho de código acima, o bloco **try** indica que o programa irá **tentar** executar as linhas 4 e 5. Se alguma houver algum erro e uma exceção for lançada pelo Java, o programa não irá abortar caso ela seja corretamente **capturada** no bloco **catch**. Para que isso ocorra, temos que “prever” corretamente o tipo de erro que pode ocorrer em cada trecho de código. Existem vários tipos clássicos de exceções. Por enquanto, estamos prevendo a exceção do tipo “InputMismatchException” (quando a entrada digitada pelo usuário não bate com o tipo de dados esperado). Veremos outros tipos de exceções mais adiante.

Agora, o programa não irá mais abortar quando o erro acontecer. Ao invés disso, exibiremos uma mensagem de erro na tela avisando que o número não pôde ser lido. Entretanto, talvez não faça sentido que o programa continue a executar normalmente sem que usuário digite corretamente um número. Assim, podemos repetir a leitura do dado **até que** ele digite um número válido.

```

1  Scanner s = new Scanner(System.in);
2  int x;
3  boolean continua = true;
4
5  while (continua){
6      try{
7          System.out.println("Digite um numero:");
8          x = s.nextInt();
9          continua = false;
10     }
11     catch (InputMismatchException e) {
12         System.err.println("Numero invalido.");
13         s.nextLine(); //descarta a entrada incorreta
14     }
15 }

```

No trecho de código acima, criamos uma variável auxiliar para indicar que a leitura deve ser realizada novamente. Enquanto essa variável for verdadeira, o laço irá se repetir.

O bloco **try** nas linhas 6 a 10 tenta fazer a leitura do número. Se a linha 8 lançar uma exceção (ou seja, o usuário digitar um número inválido), a linha 9 nem será executada e a variável **continua** continuará com o valor **true**. No caso do erro, a exceção será capturada e o código será imediatamente desviado para o bloco **catch**.

O bloco **catch** na linhas 11 a 14 faz o tratamento da exceção do tipo **InputMismatchException** (que é lançada pelo Java quando a entrada digitada pelo usuário não foi a esperada). Quando isso acontecer, imprimimos uma mensagem avisando ao usuário que o número estava incorreto (linha 12) e em seguida forçamos a leitura do valor incorreto (linha 13). Note que o número incorreto não chegou a ser lido, pois foi lançada a exceção. Assim, usamos o método **nextLine** para ler e descartar a string digitada (o método **nextLine** não vai dar erro caso o número seja inválido).

9.2 O comando finally

O comando **finally** pode ser combinado com os comando **try** e **catch**:

```
1 try {
2     // Executa ate linha onde ocorrer excecao.
3 } catch (TipoExcecao1 e) {
4     // executa somente se ocorrer TipoExcecao1
5 } catch (TipoExcecao2 e) {
6     // executa somente se ocorrer TipoExcecao2
7 } finally {
8     // executa sempre, mesmo que nenhuma
9     // excecao seja lancada ou mesmo que
10    // o programa aborte!
11 }
```

Tipicamente, um bloco **finally** costuma conter comandos de liberação de recursos alocados no bloco **try** (abertura de arquivos, de banco de dados, etc), fechar um janela da interface gráfica, etc. Se esses comandos ficassem no final do bloco **try**, poderiam nunca ser executados em caso de lançamento de exceção.

O bloco **finally** será **sempre** executado após o bloco **try** terminar normalmente, ou após algum bloco **catch** executar, mesmo que a saída desses blocos seja causada pelo lançamento de outra exceção não tratada, ou por comando **return**. O bloco **finally** somente não será executado se o programa for explicitamente terminado (utilizando o método **System.exit(o)**).

Os blocos **catch** e **finally** são opcionais, mas não é permitido haver apenas o bloco **try** sem pelo menos um bloco **catch** ou um bloco **finally** associado.

9.3 Outras exceções comuns

Alguns tipos comuns de exceção previstos no Java são:

- **InputMismatchException**: Quando você está fazendo a leitura de dados (ex: `scanner.nextInt()`) e o valor encontrado não corresponde ao tipo que era esperado.
- **NullPointerException**: Uma das exceções mais conhecidas, pois é a que ocorre com maior frequência. Ocorre quando há uma tentativa de acesso a um atributo ou a um método de um objeto, sendo que este

objeto foi declarado mas não foi instanciado através do comando **new**. Se você declarar, por exemplo, um objeto da classe `ContaCorrente`:

```
1  ContaCorrente c;
```

mas **não** instanciar este objeto:

```
1  c = new ContaCorrente();
```

o objeto terá o valor **null** (nulo, ou inexistente em português). Ao executar **c.sacar()**, será disparada a exceção `NullPointerException`, pois um objeto nulo não possui o método **sacar()**.

- *NumberFormatException*: Quando uma String tem um formato inválido e não pode ser convertida em um número.
- *ArrayIndexOutOfBoundsException*: Tentativa de acesso a uma posição inválida de um array (através de um índice negativo, ou um índice maior ou igual ao tamanho do array).
- *ArithmeticException*: Tentativa de executar alguma operação aritmética inválida, como é o caso da divisão por zero.
- *IOException*: Classe genérica para erros de entrada/saída (input/output). Por exemplo, durante a leitura de um arquivo em disco.
 - *FileNotFoundException*: Uma das subclasses de *IOException*, indica uma exceção um pouco mais específica (tentativa de acessar um arquivo que não existe).

Veja o exemplo a seguir:

```
1  Scanner s1 = null, s2 = null;
2  try {
3      s1 = new Scanner(new FileReader("arq.txt")); /* Ao inves de
4          fazer a leitura da entrada padrao, vamos ler do arquivo 'arq.
5          txt'. Entretanto, existe o risco de FileNotFoundException
6          caso o arquivo nao exista no HD. */
7      int numerador = s1.nextInt(); // Risco de InputMismatchException
8      s2 = new Scanner(System.in);
9      System.out.print("Digite o denominador: ");
10     int denominator = s2.nextInt(); // Risco de
11         InputMismatchException
12
13     System.out.println(numerador / denominator); // Risco de
14         ArithmeticException (divisao por zero)
15 }
16 catch (FileNotFoundException e) {
17     System.out.println("Arquivo nao existe.");
18 }
19 catch (InputMismatchException e) {
20     System.out.println("Numero invalido.");
21 }
22 catch (ArithmeticException e) {
23     System.out.println("Numero invalido.");
24 }
```

```

21 finally {
22     if (s1 != null)
23         s1.close(); /* Caso o arquivo tenha sido aberto mas nao foi
                        fechado por causa de alguma excecao, vamos nos certificar
                        de que ele sera fechado */
24 }

```

Quando o bloco **try** finalmente executar completamente sem que nenhuma exceção seja lançada, o bloco **finally** é executado. Ele é opcional.

9.4 Lançando uma exceção

Podemos querer tratar erros de lógica da nossa aplicação. Para isso, vamos retomar o método de controle bancário que controla os débitos em uma conta corrente. Um cenário possível é o cliente tentar fazer um saque no caixa, acima do valor do seu saldo. Até agora, esse “problema” havia sido tratado da seguinte forma:

```

1 boolean sacar(double quantidade) {
2     if (quantidade > (this.saldo + this.limite)) {
3         return false;
4     } else {
5         this.saldo = this.saldo - quantidade;
6         return true;
7     }
8 }

```

Uma forma mais adequada de tratarmos essa situação é utilizando exceções. Quando nosso método identificar o cenário acima, vamos lançar a exceção `IllegalArgumentException`, que significa que o argumento passado para o método não era o que ele esperava. Veja no trecho de código a seguir:

```

1 void sacar(double quantidade) throws IllegalArgumentException {
2     if (quantidade > (this.saldo + this.limite)) {
3         // Aqui lancamos uma excecao:
4         throw new IllegalArgumentException();
5     } else {
6         this.saldo = this.saldo - quantidade;
7     }
8 }

```

A diretiva **throw** lança uma exceção, para que um bloco **try-catch** a capture e faça o tratamento necessário. Já a diretiva **throws** (na primeira linha do código) apenas avisa da possibilidade de um determinado método lançar uma exceção (diferente de **throw**, que efetivamente lança uma exceção).

Note que o método não precisa mais retornar verdadeiro ou falso. Agora, podemos tratar o erro num nível mais alto, ou seja, onde o método de **sacar()** foi chamado. Para isso, não usaremos mais um **if-else**, e sim um **try-catch** (o que faz mais sentido, já que a falta de saldo é uma exceção ao funcionamento normal do programa).

```

1 ContaCorrente c = new ContaCorrente();
2 try {
3     c.sacar(100);
4 }

```

```
5 catch (IllegalArgumentException e) {  
6     System.out.println("Saldo Insuficiente.");  
7 }
```

Para melhorar ainda mais nosso tratamento de erro, podemos fazer com que o método **sacar** passe uma mensagem junto com a exceção, explicando o motivo do erro:

```
1 void sacar(double quantidade) throws IllegalArgumentException {  
2     if (quantidade > (this.saldo + this.limite)) {  
3         throw new IllegalArgumentException(  
4             "Saldo Insuficiente.");  
5     } else {  
6         this.saldo = this.saldo - quantidade;  
7     }  
8 }
```

Para termos acesso a essa mensagem no trecho de código que utilizar o método **sacar()**, utilizamos o método **getMessage()**:

```
1 ContaCorrente c = new ContaCorrente();  
2 try {  
3     c.sacar(100);  
4     System.out.println("Saque realizado com sucesso.");  
5 }  
6 catch (IllegalArgumentException e) {  
7     System.out.println(e.getMessage());  
8 }
```

Se a exceção lançada for capturada, o restante do código dentro do try não será executado. Ou seja, a mensagem “Saque realizado com sucesso.” só seria impressa se o método **sacar()** não retornar uma exceção.

9.5 Definindo nossas próprias exceções

Por fim, se não quisermos utilizar umas das exceções já existentes (no exemplo, escolhemos a exceção **IllegalArgumentException**), podemos criar nossa própria classe de exceção, chamada **SaldoInsuficienteException**, que será uma subclasse de **Exception**. Veja o trecho de código a seguir:

```
1 public class SaldoInsuficienteException extends Exception{  
2     SaldoInsuficienteException(String mensagem) {  
3         super(mensagem);  
4     }  
5 }
```

Agora, ao invés de lançarmos a exceção **IllegalArgumentException** no método **sacar**, podemos lançar a exceção do tipo **SaldoInsuficienteException** que acabamos de criar.

9.6 Exercícios

Retome o projeto do Banco, identifique onde podem ser tratadas as exceções e onde o bloco finally é necessário. Depois, confira se é possível criar e lançar suas próprias exceções.

Aula 10

Persistência em Arquivos

É comum que um projeto manipule uma grande quantidade de informações que não devem ser perdidas quando o programa for encerrado. Quando não temos um banco de dados a nossa disposição para salvar as informações, podemos armazená-las em arquivos textos no disco. Dessa forma, as informações salvas em execuções anteriores podem ser carregadas automaticamente na memória a cada nova execução do programa.

10.1 Salvando dados em arquivos textos

No projeto do nosso Banco, por exemplo, a leitura e escrita em arquivos pode ser muito útil para o usuário não ter que recadastrar toda informação sobre os clientes e contas a cada vez que o programa for inicializado.

Vamos começar salvando num arquivo as informações sobre cada cliente que for cadastrado. A classe **FileWriter** é responsável pela escrita em arquivos, e o construtor recebe como parâmetro o nome do arquivo que será criado para escrita. Exemplos:

```
1 FileWriter f = new FileWriter("arquivo.txt");
```

É possível passar um segundo parâmetro para o construtor indicando se queremos adicionar informações ao arquivo, caso ele já exista (ao invés de sobrescrever o arquivo):

```
1 FileWriter f = new FileWriter("arquivo.txt", true);
```

Já a classe **BufferedWriter** cria um buffer (semelhante a um Array na memória) onde é armazenada a sequência de caracteres a ser escrita no arquivo, e recebe como parâmetro um objeto da classe **FileWriter**:

```
1 BufferedWriter buff = new BufferedWriter(f);
```

A cada nova sequência de caracteres a ser inserida, usamos o método **write**:

```
1 buff.write("bla bla bla");  
2 buff.write("\n");  
3 buff.write("Ola!");
```

As informações vão sendo armazenadas no buffer, e só serão efetivamente salvas no arquivo quando o arquivo for fechado:

```
1 buff.close();
```

10.2 Lendo dados de arquivos textos

De forma análoga, a leitura dos dados é feita através das classes `FileReader` e `BufferedReader`:

```
1 FileReader f = new FileReader("arquivo.txt");
2 BufferedReader buff = new BufferedReader(f);
```

Cada linha do arquivo é lida através do comando **readline**. O resultado é sempre uma `String`, e deve ser transformada apropriadamente do acordo com o tipo de dado que for lido:

```
1 String nome = buff.readline();
2 int tam = Integer.parseInt( buff.readline() );
```

Assim como na escrita, o buffer deve ser fechado:

```
1 buff.close();
```

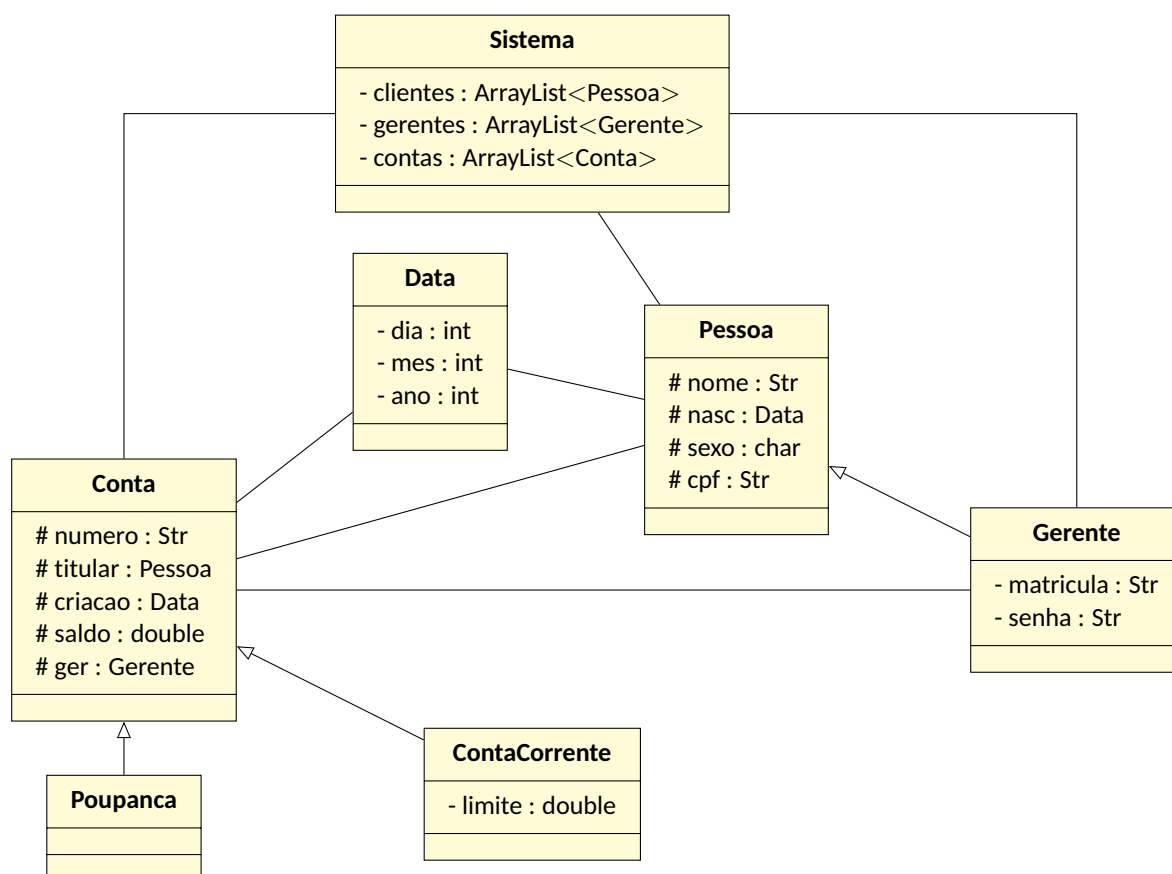
10.3 Exercícios

Retome o projeto do Banco, e identifique os principais dados a serem salvos em arquivo. Salve os dados a cada atualização do cadastro e leia os dados toda vez que o sistema seja inicializado. Certifique-se de utilizar as boas prática de Orientação a Objetos.

Aula 11

Polimorfismo, Classes Abstratas e Interfaces

Até agora, o diagrama de classes do nosso Banco está da seguinte forma:



Dessa forma, uma conta no banco pode ser de dois tipos: Conta Corrente ou Poupança. No método principal, chegamos a criar um único ArrayList contendo todas as contas, e nos referimos a elas de forma genérica, utilizando um ArrayList <Conta>:

```
1 ArrayList<Conta> contas = new ArrayList<>();
2
3 contas.add( new ContaCorrente() );
4 contas.add( new Poupanca() );
```

Porém, ainda é possível criar um objeto da classe **Conta** que não seja de nenhuma dos dois tipos:

```
1 Conta c = new Conta();
```

Isso deveria ser possível?

11.1 Classes Abstratas

Uma **classe abstrata** é uma classe especial que não pode ser instanciada, apenas estendida. Sua finalidade, portanto, é ser superclasse em uma hierarquia. Se transformarmos a classe Conta numa classe abstrata, não será mais possível criar um objeto desta classe, apenas de suas subclasses.

Em Java, uma classe é definida como abstrata através do modificador **abstract**. Uma classe abstrata pode contar ou não métodos que também sejam abstratos. Um método abstrato é um método que é apenas declarado, mas não implementado efetivamente. Ele indica apenas que todas as classes filhas (que forem concretas, ou seja, que não forem abstratas) são obrigadas a reescrever esse método:

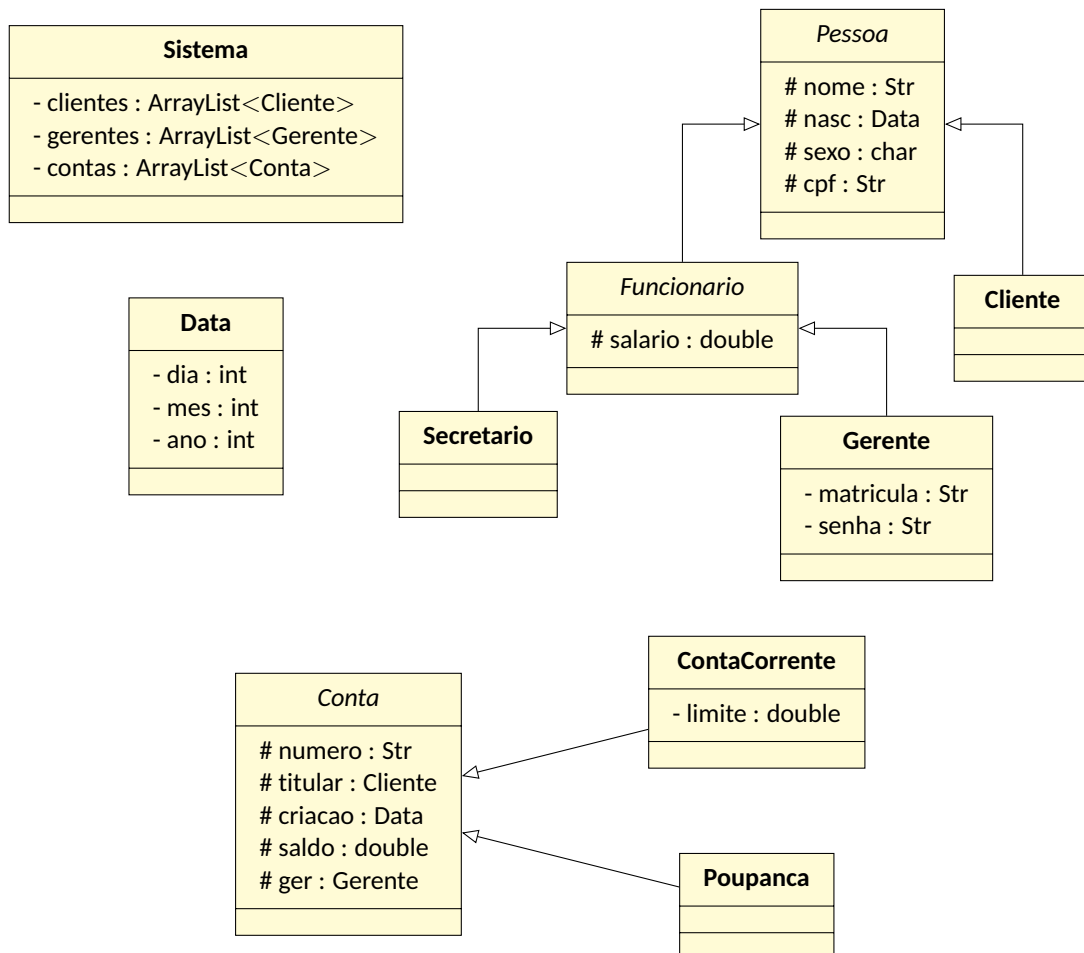
```
1 public abstract class Conta {
2     protected String numero;
3     protected Pessoa titular;
4     protected double saldo;
5     protected Data criacao;
6     protected Gerente gerente;
7
8     /* Construtores podem permanecer. Inicializam os atributos da
9        classe e são chamados pelas subclasses. */
10
11    // Metodo abstrato:
12    public abstract double disponivel();
13
14    // Metodos concretos:
15    public boolean sacar(double valor) {
16        if (valor <= this.disponivel()) {
17            this.setSaldo(this.getSaldo() - valor);
18            return true;
19        }
20        else {
21            return false;
22        }
23    }
24
25    // Demais metodos...
```

Para declarar um **método abstrato** (linha 13 do código acima), basta escrever **abstract** na assinatura do mesmo e colocar um ponto e vírgula em vez de abri e fechar chaves. O método **disponível** é diferente em cada tipo de conta. Por isso, pode parecer não fazer sentido implementá-lo na superclasse. Entretanto, toda subclasse terá a obrigação de implementá-lo. Já o método **sacar**, mesmo utilizando o método abstrato **disponível**, pode continuar na superclasse, pois temos a certeza que cada conta terá uma forma de calcular seu valor disponível.

Por não permitir a instanciação de novos objetos com base em uma classe abstrata, este tipo de classe é utilizada para implementar o conceito de herança na orientação a objetos. Ou seja, as classes abstratas irão armazenar atributos e métodos comuns às classes que a irão herdar, permitindo um maior reaproveitamento de código.

Podemos também transformar a classe **Pessoa** em abstrata, sendo estendida pelas subclasses **Cliente** e **Funcionario**. Esta última possui o atributo salário, e também é uma classe abstrata (pois pode haver mais de um tipo de funcionário no banco). Por enquanto, a classe **Funcionario** será estendida pelas subclasses **Gerente** (com senha e matrícula) e **Secretario**.

No diagrama de classes, uma classe abstrata é diferenciada pelo seu nome em *itálico* (as associações simples entre as classes foram omitidas):



11.2 Interfaces

Já uma **interface** serve para definir um conjunto de métodos que um grupo de classes deverá, obrigatoriamente, implementar. Ou seja, define um “contrato” a ser seguido por outras classes, com os comportamentos que este grupo de classes deverá seguir.

Vamos supor que os dados do nosso sistema serão salvos em um único arquivo texto. Ao sair do programa, queremos iterar sobre todos os objetos “*salváveis*” do sistema para, efetivamente, salvá-los no disco. Podemos definir a interface **Salvável**, que vai obrigar um certo conjunto de classes a implementar o método **salvarArq**:

```

1 public interface Salvavel {
2
3     public abstract void salvarArq(BufferedWriter b) throws
      IOException;
4 }

```

As classes **Cliente**, **Gerente**, **Secretario**, **ContaCorrente** e **Poupanca** irão implementar a interface **Salvavel**, pois todas elas possuem informações a serem gravadas em disco:

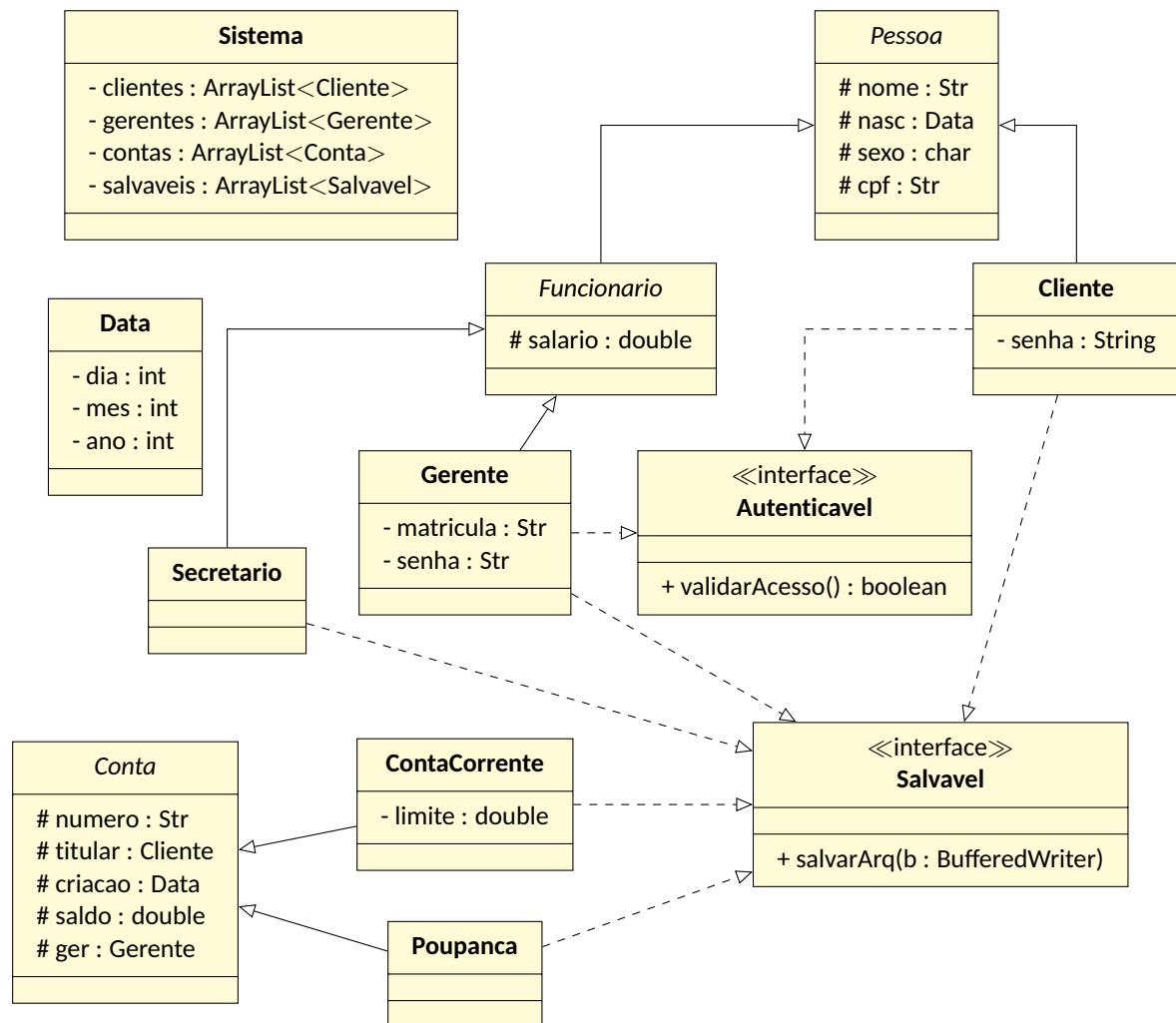
```
1 public class ContaCorrente extends Conta implements Salvavel {  
2  
3     // Restante da classe  
4  
5 }
```

Em todas elas, deve obrigatoriamente ser implementado o método **salvarArq**, que foi definido na interface.

Podemos criar também uma interface chamada **Autenticavel**, com os tipos de objetos que poderão se logar no sistema: gerentes e clientes. Note que um gerente é um funcionário, mas nem todo tipo de funcionário terá acesso ao sistema. O cliente irá acessá-lo para movimentar suas contas, e os gerentes para realizar os cadastros e demais manutenções no sistema.

```
1 public interface Autenticavel {  
2  
3     public boolean validarAcesso(String pwd);  
4 }  
  
1 public class Gerente extends Funcionario implements Salvavel,  
    Autenticavel {  
2  
3     // Restante da classe  
4  
5 }
```

O diagrama a seguir introduz as duas interfaces e mostra a relação entre as classes que as implementam:



Os métodos nas interfaces **devem** ser abstratos, pois a interface define apenas o conjunto de métodos que devem ser implementados, e não suas implementações. Além disso, por não ser considerada uma classe, as interfaces não possuem construtores e, por isso, não podem ser instanciadas, como as classes abstratas. A interface não é considerada uma classe, e sim uma **entidade**.

Como a linguagem Java não tem herança múltipla, as interfaces ajudam nessa questão. Uma classe pode herdar de apenas uma superclasse, mas pode implementar inúmeras interfaces.

11.3 Classes Abstratas Vs. Interfaces

As classes abstratas e interfaces possuem algumas características semelhantes: não podem ser instanciadas e possuem métodos abstratos que obrigam as outras classes a implementá-los. Porém, elas não possuem o mesmo propósito.

Quando utilizamos as interfaces, estamos definindo um conjunto de assinatura de métodos que outras classes devem implementar. Com isso, apenas definimos o comportamento base de um conjunto de classes que, por ventura, implementem esta interface.

Já as classes abstratas servem para prover uma base para que as classes que “herdem” desta não precisem se preocupar com o comportamento padrão, apenas com suas características e comportamentos pessoais.

Sendo assim, sempre que precisarmos definir um conjunto de métodos que devem ser implementados

por um grupo de classes, utilizamos as interfaces. Se precisarmos determinar uma classe base para outras classes, que herdarão seus atributos e métodos e esta classe não deva ser instanciada, utilizamos as classes abstratas.

11.4 Polimorfismo

O termo **polimorfismo** vem do grego e significa “muitas formas”. Em orientação a objetos, o polimorfismo ocorre quando duas ou mais classes distintas têm métodos de mesmo nome, de forma que possam responder de formas distintas a um mesmo comando.

O saque em uma conta-corrente, por exemplo, é diferente do saque em uma poupança. O polimorfismo permite códigos como este a seguir:

```
1 for (Conta c : this.contas) {  
2     c.sacar();  
3 }
```

sendo que o `ArrayList` de contas pode possuir tanto contas correntes como poupanças. O polimorfismo pode ser obtido através de heranças, classes abstratas e até interfaces. Neste trecho de código, os objetos das classes **Poupanca** e **ContaCorrente** vão se comportar como objetos da classe **Conta**. Afinal, uma **ContaCorrente** é uma **Conta**. O método **chequeEspecial**, por exemplo, não pode ser invocado neste contexto. Apesar de uma **ContaCorrente** possuir esse método, neste contexto os objetos só podem invocar métodos comuns a todos os objetos da classe **Conta**.

É importante observar que, quando polimorfismo está sendo utilizado, o comportamento que será adotado por um método só será definido durante a execução. Embora em geral esse seja um mecanismo que facilite o desenvolvimento e a compreensão do código orientado a objetos, há algumas situações onde o resultado da execução pode ser não-intuitivo.

11.5 Exercícios

Retome o projeto do Banco, e identifique onde é possível inserir classes abstratas e interfaces.

Aula 12

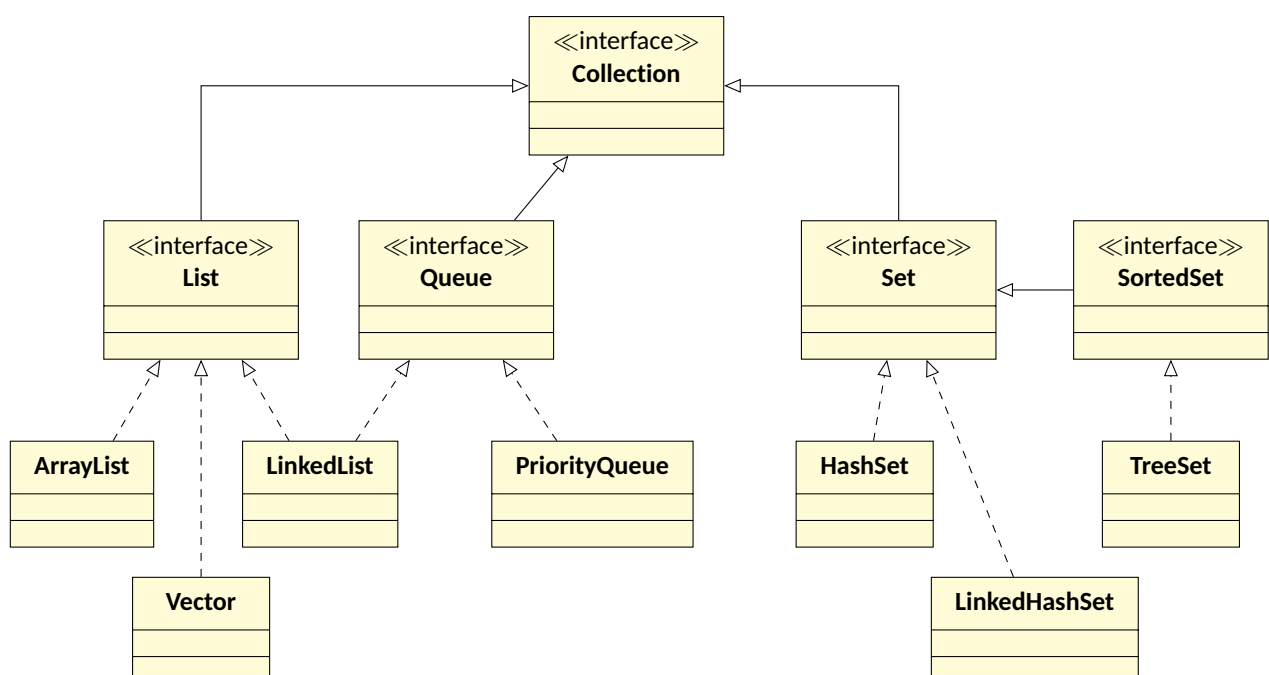
Coleções e Ordenação

12.1 Coleções

Uma **coleção** é uma estrutura de dados que permite armazenar vários objetos. Você pode adicionar novos elementos a uma coleção, bem como remover, acessar e pesquisar por eles. Existem 3 tipos principais de coleções:

- **Lista:** Coleção de elementos dispostos em ordem linear (cada elemento após o primeiro possui um antecessor, e cada elemento antes do último possui um sucessor). Seus elementos podem ser mantidos em ordem crescente ou não.
- **Conjunto:** Coleção que não possui elementos duplicados. Não há aqui o conceito de “ordem” dos elementos.
- **Mapa:** Armazena pares (chave, valor) chamados itens. Uma chave é utilizada para acessar rapidamente um determinado valor e, portanto, há um “mapeamento” de chaves para valores.

Em cada uma delas, vai variar desempenho e as restrições para cada tipo de funcionalidade a ser utilizada. Java disponibiliza a interface **Collection**, com a hierarquia a seguir:



Repare que um **ArrayList**, que já utilizamos nesta disciplina, é na verdade uma implementação da interface **List**, que por sua vez vem da interface **Collection**.

Collection (coleção, em português) é a interface no topo da hierarquia. Não existe implementação direta dessa interface, mas ela define as operações básicas para as coleções, como adicionar e remover elementos.

List (lista, em português) define uma coleção de elementos dispostos sequencialmente, podendo conter elementos duplicados. Em geral, o usuário tem controle total sobre a posição onde cada elemento é inserido e pode recuperá-los através de seus índices. Esta interface é preferível quando se precisar de acesso aleatório, ou seja, através do índice do elemento.

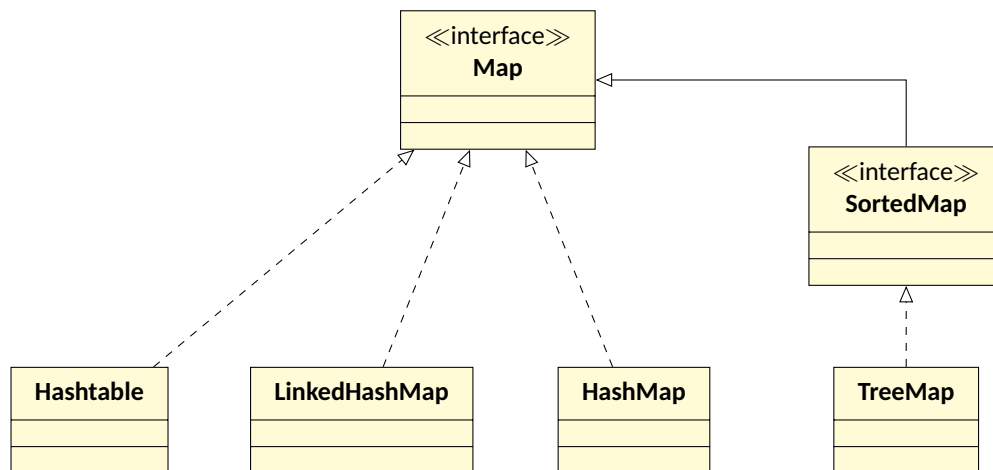
- **ArrayList** é como um array cujo tamanho pode crescer. A busca de um elemento é rápida, mas inserções e exclusões não são. Podemos afirmar que as inserções e exclusões são lineares, o tempo cresce com o aumento do tamanho da estrutura. Esta implementação é preferível quando se deseja acesso mais rápido aos elementos. Por exemplo, se você quiser criar um catálogo com os livros de sua biblioteca pessoal e cada obra inserida receber um número sequencial (que será usado para acesso) a partir de zero. É preferível quando há mais consultas do que inserções.
- **LinkedList** implementa uma lista ligada, ou seja, cada nó contém o dado e uma referência para o próximo nó. Ao contrário do **ArrayList**, a busca é linear e inserções e exclusões são rápidas. Portanto, prefira **LinkedList** quando a aplicação exigir grande quantidade de inserções e exclusões. Um pequeno sistema para gerenciar suas compras mensais de supermercado pode ser uma boa aplicação, dada a necessidade de constantes inclusões e exclusões de produtos.

Queue (fila, em português) define um tipo de coleção para manter uma lista de prioridades, onde a ordem dos seus elementos determina essa prioridade. Com a interface fila pode-se criar filas e pilhas.

Set (conjunto, em português) é a interface que define uma coleção que não permite elementos duplicados. A interface **SortedSet**, que estende **Set**, possibilita a classificação natural dos elementos, tal como a ordem alfabética.

- O acesso aos dados em um **HashSet** é mais rápido que em um **TreeSet**, mas nada garante que os dados estarão ordenados. Escolha este conjunto quando a solução exigir elementos únicos e a ordem não for importante.
- Em um **TreeSet** os dados são classificados, mas o acesso é mais lento que em um **HashSet**. Se a necessidade for um conjunto com elementos não duplicados e acesso em ordem natural, prefira o **TreeSet**.
- **LinkedHashSet** mantém uma lista duplamente ligada através de seus itens. Seus elementos são iterados na ordem em que foram inseridos. Opcionalmente é possível criar um **LinkedHashSet** que seja percorrido na ordem em que os elementos foram acessados na última iteração.

Os mapas não são derivados da interface **Collection**, e sua hierarquia é a seguinte:



Map (mapa, em português), mapeia chaves para valores. Cada elemento tem na verdade dois objetos: uma chave e um valor. Valores podem ser duplicados, mas chaves não. **SortedMap** é uma interface que estende **Map**, e permite classificação ascendente das chaves.

- **HashMap** é baseada em tabela de espalhamento, permite chaves e valores null. Não existe garantia que os dados ficarão ordenados. Escolha esta implementação se a ordenação não for importante e desejar uma estrutura onde seja necessário um identificador.
- **TreeMap** implementa a interface **SortedMap**. Há garantia que o mapa estará classificado em ordem ascendente das chaves. Mas existe a opção de especificar uma ordem diferente. Use esta implementação para um mapa ordenado.
- **LinkedHashMap** mantém uma lista duplamente ligada através de seus itens. A ordem de iteração é a ordem em que as chaves são inseridas no mapa. Se for necessário um mapa onde os elementos são iterados na ordem em que foram inseridos, use esta implementação.

De acordo com as boas práticas de orientação a objetos, você deve **programar para interfaces e não para implementações**. Portanto, é recomendado escolher uma implementação para instanciar o objeto e atribuir a nova coleção ao tipo de interface correspondente. Ou ainda, passar o objeto coleção para um método que espera um argumento do tipo interface. Seguindo essas práticas você conseguirá o que chamamos de baixo acoplamento, ou seja, poderá mudar facilmente de implementação sem que isso acarrete alteração no código da aplicação. Desta forma você fica livre para mudar a implementação sempre que questões relacionadas a desempenho ou detalhes de comportamento exigirem a mudança. Exemplo:

```

1 List<String> lista = new ArrayList<String>();
2
3 lista.add("Hilario");
4 lista.add("Maria");
5 lista.add("Joao");

```

Repare que o objeto **lista** foi instanciado como uma **ArrayList** (esta é a implementação que escolhemos). Porém, ele foi declarado como **List**. Assim, podemos nos referir a ele em todo o restante do código como uma lista (pois sabemos que um **ArrayList** é obrigado a implementar todos os métodos de **List**). Com isso, se um dia for necessário alterar a implementação para outro tipo de lista, basta alterar no lugar em que o objeto foi instanciado.

12.2 Ordenação

Os elementos em um `ArrayList`, por exemplo, são dispostos sequencialmente na ordem em que são inseridos. Não há um método na classe `ArrayList` para colocá-los em ordem crescente, por exemplo. Poderíamos utilizar um `Set`, que dispõe os elementos em ordem natural, mas a inserção de novos elementos seria mais lenta. Se quisermos determinar a forma como os elementos são ordenados em um `ArrayList`, podemos utilizar a classe utilitária `Collections` (no plural, diferente da interface `Collection` vista acima). Esta classe dispõe de um método `sort()`:

```
1 List<String> lista = new ArrayList<String>();
2
3 lista.add("Hilario");
4 lista.add("Maria");
5 lista.add("Joao");
6 Collections.sort(lista);
```

É possível ordenar a lista acima porque a classe `String` possui um método chamado `compareTo()` que recebe uma outra `String` como parâmetro, e retorna se ela é “maior”, “menor”, ou “igual” à segunda (semelhante ao que fizemos anteriormente na classe `Data`):

```
1 public class Data {
2     private int dia;
3     private int mes;
4     private int ano;
5
6     // Construtores...
7
8     int compara(Data d) {
9         if (this.ano < d.getAno()) return -1;
10        if (this.ano > d.getAno()) return 1;
11
12        if (this.mes < d.getMes()) return -1;
13        if (this.mes > d.getMes()) return 1;
14
15        if (this.dia < d.getDia()) return -1;
16        if (this.dia > d.getDia()) return 1;
17
18        return 0;
19    }
20
21    // Demais metodos...
22 }
```

O método `compara` retornava `-1` quando a primeira data ocorreu antes da segunda, `1` quando ocorria após a segunda, e `0` quando eram iguais.

Sempre que for necessário utilizar `Collection.sort` em uma classe qualquer, devemos implementar nesta classe a interface `Comparable`. O único método a ser implementado desta interface é o `compareTo()`. Sua implementação deve ser feita de forma a retornar um inteiro negativo, zero ou um inteiro positivo caso o objeto que execute o método seja menor, igual ou maior que o objeto passado como parâmetro. Cabe ao desenvolvedor decidir o critério que será adotado para comparar dois objetos. Nossa classe `Data`, portanto, poderia ficar assim:

```

1 public class Data implements Comparable<Data> {
2     private int dia;
3     private int mes;
4     private int ano;
5
6     // Construtores...
7
8     public int compareTo(Data d) {
9         if (this.ano < d.getAno()) return -1;
10        if (this.ano > d.getAno()) return 1;
11
12        if (this.mes < d.getMes()) return -1;
13        if (this.mes > d.getMes()) return 1;
14
15        if (this.dia < d.getDia()) return -1;
16        if (this.dia > d.getDia()) return 1;
17
18        return 0;
19    }
20
21
22    public String toString() {
23        return this.dia + "/" + this.mes + "/" + this.ano;
24    }
25
26    // Demais metodos...

```

E caso tenhamos um `ArrayList` chamado **datas** com objetos da classe **Data** e quisermos colocá-lo em ordem crescente, basta fazermos:

```

1 System.out.println(datas);
2
3 Collections.sort(datas);
4
5 System.out.println(datas);

```

Repare que o método **toString()** da classe **Data** define a forma como o Java irá imprimir um objeto desta classe. No nosso caso, definimos que uma data será impressa no formato DD/MM/AAAA. Assim, caso sejam adicionadas no `ArrayList` **datas**, nesta ordem, as datas “1/jan/2000”, “1/fev/1999”, “2/fev/1990” e “2/fev/2001”, a saída do trecho de código acima será:

```

[1/1/2000, 1/2/1999, 2/1/1990, 2/2/2001]
[2/1/1990, 1/2/1999, 1/1/2000, 2/2/2001]

```

Se temos uma lista com objetos da classe **Cliente** e queremos colocá-los em ordem alfabética pelo nome do cliente, podemos criar um método **compareTo()** que utiliza o próprio método **compareTo()** já fornecido pela classe `String`:

```

1 public class Cliente extends Pessoa implements Comparable<Cliente>{
2
3     // Atributos da classe...

```

```

4
5 // Construtores...
6
7 public int compareTo(Cliente c) {
8     return this.nome.compareTo(c.getNome());
9 }
10
11 // Demais metodos...

```

Por fim, você pode optar por, eventualmente, ordenar a lista de clientes por um critério diferente (pelo CPF, por exemplo). Neste caso, você não deveria alterar o método **compareTo()**, pois isso afetaria todas as demais ordenações. Ou, ainda, você pode estar utilizando uma classe e não possui permissão para alterá-la. Neste caso, você pode utilizar o segundo parâmetro do método **Collection.sort()**, representado pela interface **Comparator** (cuidado para não confundi-lo com o anterior, **Comparable**). Para isso, você deve criar uma classe que implementa esta interface, definindo como seus objetos devem ser comparados através do método **compare()**. Este método irá receber dois objetos da classe, retornando um inteiro negativo se o primeiro objeto for menor que o segundo, zero se forem iguais, ou um inteiro positivo se o primeiro objeto for maior que o segundo:

```

1 public class ComparaCliente implements Comparator<Cliente> {
2
3     public int compare(Cliente a, Cliente b) {
4         return a.getCPF().compareTo(b.getCPF());
5     }
6 }

```

Para utilizar esta implementação, chamamos o método **Collections.sort()**, passando como parâmetros a lista e um objeto instanciado da classe **ComparaCliente**:

```

1 List<Cliente> clientes = new ArrayList<Cliente>();
2 ComparaCliente cc = new ComparaCliente();
3
4 // Adicionando objetos na lista...
5
6 Collections.sort(lista, cc);

```

12.3 Exercícios

Retome o projeto do Banco, e revise se as coleções utilizadas no projeto são as mais adequadas. Reformule o código buscando baixo acoplamento e utilização da estrutura de dados mais adequada a seu problema. Em seguida, adicione opções no meu para imprimir os dados ordenados por mais de um critério, utilizando **Comparable** e **Comparator**.

Índice Remissivo

Abstract, 54
ArrayLists, 37
Arrays, 36
Atributos, 3

BufferedReader, 52
BufferedWriter, 51

Catch, 44
Classes, 3
Classes Abstratas, 54
Coleções, 59
Collection, 60
Collections, 62
Comparable, 62
Comparator, 64
compare(), 64
compareTo(), 62
Construtores, 13

Encapsulamento, 33
Escrita em Arquivos Textos, 51

FileReader, 52
FileWriter, 51
Finally, 46

Herança, 18

Instância, 4
Interfaces, 55

Leitura em Arquivos Textos, 52
LinkedList, 60
List, 60

Modificadores de Acesso, 32
Métodos, 6
Métodos Abstratos, 54

Objetos, 4

Persistência em Arquivos, 51
Polimorfismo, 58

readline(), 52

Reescrita de Métodos, 24

Sobrecarga de Construtores, 29
Sobrecarga de Métodos, 29
sort(), 62
Subclasses, 18
Superclasses, 18

Throw, 48
Throws, 48
Tratamento de Erros e Exceções, 44
Try, 44

Parte II

Prática

Aula 13

Classes e Objetos

13.1 Primeiros passos

O primeiro passo para iniciar a resolução dos exercícios propostos ao longo da Parte I o IDE em que irá desenvolver seus projetos. Um IDE serve para centralizar e agilizar a criação de softwares, unindo editor de código, compilador, depurador e ferramentas de automação, com recursos para aumentar a produtividade, autocompletar, detecção de erros e gerenciamento de arquivos. Também é possível não utilizar um IDE e criar os códigos em editores de textos simples. Aqui, mostraremos a criação de um projeto inicial utilizando o IDE IntelliJ. Ele pode ser obtido em <https://www.jetbrains.com/pt-br/idea/>, com uma versões gratuitas e para estudantes.

Ao abrir o IDE, você escolhe o nome do projeto. Na imagem, estamos criando um projeto chamado **Banco**.

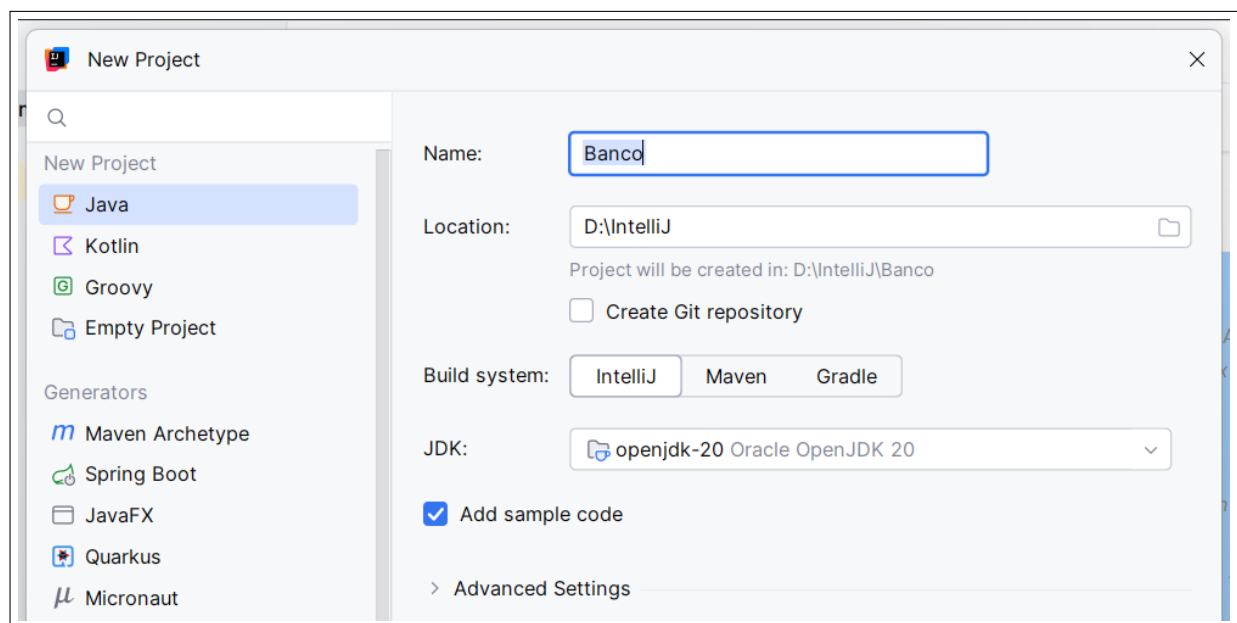


Figura 13.1: Tela inicial do IntelliJ.

Em seguida, será exibida uma tela com os arquivos iniciais do projeto, incluindo uma classe Main com um exemplo de método principal, que imprime uma mensagem de boas vindas na e realiza uma repetição exibindo os números de 1 a 5, um por linha.

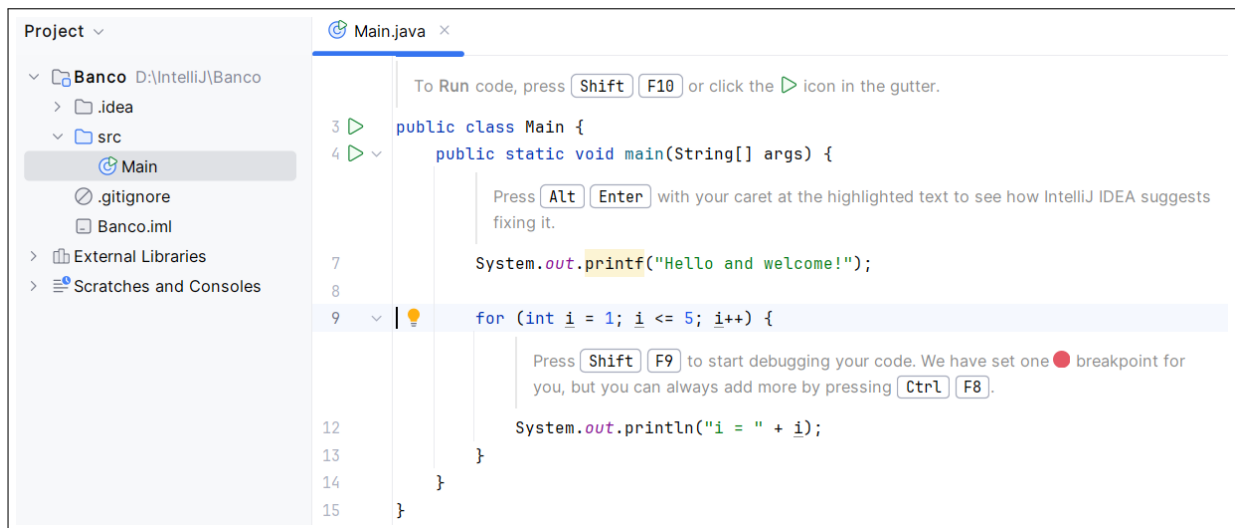


Figura 13.2: Criando um novo projeto.

Não se preocupe ainda com algumas palavras reservadas da linguagem, como **public** e **static**. Por enquanto, o IntelliJ cuidará disso. No canto superior da tela, há um botão para executar o projeto:



Figura 13.3: Executando um projeto.

O projeto também pode ser executado pelas teclas de atalhos SHIFT+F10. Após executar o projeto, a saída do programa provavelmente irá aparecer na parte inferior da tela, como na figura a seguir:

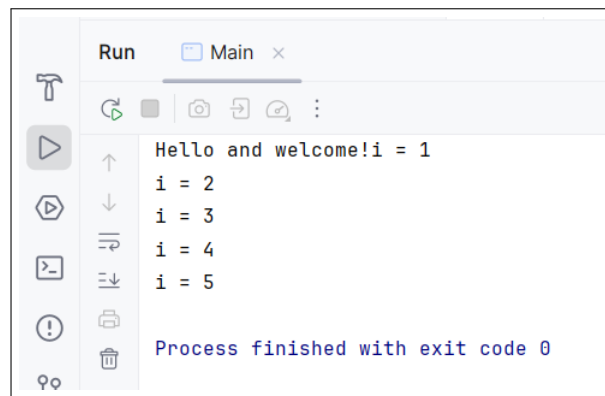


Figura 13.4: Saída do programa.

13.2 Criando novas classes

Agora é possível criar a classe que irá representar as pessoas cadastradas em nosso sistema. Para isso, escolha a opção de adicionar um arquivo e escolha a opção **Java Class**:

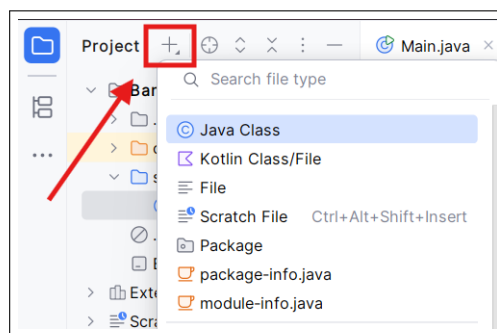


Figura 13.5: Criando uma nova classe.

Em seguida, digite o nome da classe e aperte a tecla enter:

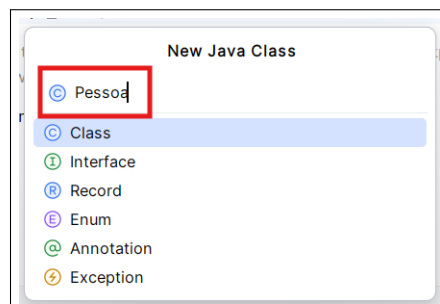


Figura 13.6: Criando a classe Pessoa.

Após finalizar, o IntelliJ irá exibir o código-fonte da nova classe. Vamos alterá-lo para que represente o diagrama a seguir:

Pessoa
nome : String idade : int sexo : char cpf : String

Ou seja, a classe deve conter o seguinte código:

```
1 public class Pessoa {  
2     String nome;  
3     int idade;  
4     char sexo;  
5     String cpf;  
6 }
```

Agora, repita os passos anteriores para criar uma classe **Conta** de acordo com o diagrama a seguir:

Conta
numero : String titular : Pessoa saldo : double limite : double

O tipo **double** em Java representa pontos flutuantes com o dobro da precisão do tipo **float**.

```
1 public class Conta {  
2     String numero;  
3     Pessoa titular;  
4     double saldo, limite;  
5 }
```

13.3 Testando o programa

Por fim, vamos testar essa primeira aplicação. Para isso, volte ao código da classe principal e adicione os códigos a seguir. SUGESTÕES:

1. Não copie e cole. Vá digitando linha por linha, para se familiarizar com o IntelliJ. Repare que ele vai auto-completando algumas coisas para facilitar seu trabalho.
2. Altere os valores dos atributos no seu código (por exemplo, o nome das pessoas e o saldo das conta).

```
1 // Criando um objeto da classe Pessoa  
2 Pessoa p1 = new Pessoa();  
3 p1.nome = "Maria";  
4 p1.idade = 18;  
5 p1.sexo = 'F';
```

```

6  p1.cpf = "123.456.789-00";
7
8  // Criando outro objeto da classe Pessoa
9  Pessoa p2 = new Pessoa();
10 p2.nome = "Joao";
11 p2.idade = 22;
12 p2.sexo = 'M';
13 p2.cpf = "234.567.890-00";
14
15 // Criando um objeto da classe Conta
16 Conta c1 = new Conta();
17 c1.numero = "1234-5";
18 c1.titular = p1; // a Pessoa p1 eh titular desta Conta!
19 c1.saldo = 100.0;
20 c1.limite = 200.0;
21
22 // Criando outro objeto da classe Conta
23 Conta c2 = new Conta();
24 c2.numero = "2345-6"; // esta Conta pertence 'a Pessoa p2
25 c2.titular = p2;
26 c2.saldo = 150.0;
27 c2.limite = 200.0;
28
29 // Imprimindo o extrato
30 System.out.println(c1.saldo);
31 System.out.println(c1.titular.nome);
32 System.out.println("Saldo da conta " + c1.numero + ": " + c1.saldo);

```

Se tudo der certo, a saída será parecida com o trecho a seguir:

```

100.0
Maria
Saldo da conta 1234-5: 100.0

```

Por fim, para terminar o exercício da Aula 1, altere a impressão no método principal, para imprimir algo semelhante a:

```

*** EXTRATO DA CONTA ***
Conta: 1234-5
Titular: Maria (CPF: 123.456.789-00)
Valor disponível para saque: R$ 300.0

```

Note que o valor disponível para saque não é o saldo da conta, e sim a soma do saldo com o limite:

```

1  System.out.println("*** EXTRATO DA CONTA ***");
2  System.out.println("Conta: " + c1.numero);
3  System.out.println("Titular: " + c1.titular.nome + " (CPF: " + c1.
    titular.cpf + ")");
4  System.out.println("Valor disponivel para saque: R$ " + (c1.saldo+c1
    .limite));

```

Aula 14

Métodos - Prática

14.1 Crie um novo projeto

Na aula prática passada, você criou um projeto com as classes Pessoa e Conta. Lembre-se sempre:

- Não utilize acentos em nomes de projetos, classes, variáveis e atributos.
- Nomes de **projetos** e **classes** devem ser escritos com a inicial **maiúscula**.
- Nomes de **atributos** e **variáveis** devem ser escritos com a inicial **minúscula**.

Para começar, certifique-se que suas classes refletem **exatamente** o diagrama a seguir:

Pessoa	Conta
nome : String idade : int sexo : char cpf : String	numero : String titular : Pessoa saldo : double limite : double

Lembre-se que cada classe em um projeto deve ficar num arquivo separado. Além de um arquivo para cada classe, há também um arquivo com a classe principal, que tem o mesmo nome do projeto.

Agora, vamos alterar a classe **Conta**, adicionando os métodos de acordo com o modelo a seguir:

Conta
numero : String titular : Pessoa saldo : double limite : double
disponivel() : double extrato() depositar(valor : double) sacar(valor : double) : boolean transferir(valor : double, destino : Conta) : boolean

Os métodos **extrato** e **depositar** não possuem retorno. O modelo, portanto, omite o retorno destes dois métodos. Entretanto, é importante lembrar que no código Java, o método deve ser declarado como **void**.

Os métodos são implementados dentro do bloco de código da classe, logo após a definição dos atributos. O método **disponivel** deverá calcular e retornar o valor disponível na conta que pode ser gasto pelo titular, ou seja, a soma do seu saldo em conta com o limite disponibilizado pelo banco:

```
1 public class Conta {
2     String numero;
3     Pessoa titular;
4     double saldo, limite;
5
6     double disponivel() {
7         return this.saldo + this.limite;
8     }
9
10    void extrato() {
11        System.out.println("*** EXTRATO DA CONTA ***");
12        System.out.println("Conta: " + this.numero);
13        System.out.println("Titular: " + this.titular.cpf);
14        System.out.println("Saldo disponivel para saque: " + this.
            disponivel());
15    }
16
17    /* Demais metodos irao aparecer aqui... */
18 }
```

Os métodos da classe **Conta** devem ser inseridos dentro da própria classe, logo após a definição de seus atributos.

O método **depositar** deverá receber um valor e adicioná-lo ao saldo da conta:

```
1 void depositar(double valor) {
2     this.saldo += valor;
3 }
```

O método **sacar** deverá retornar um valor booleano (verdadeiro ou falso), que vai indicar se a operação pôde ser realizada com sucesso:

```
1 boolean sacar(double valor) {
2     if (valor <= this.disponivel()) {
3         this.saldo -= valor;
4         System.out.println("Saque na conta " + this.numero + "
            realizado com sucesso. Novo saldo: " + this.saldo);
5         return true;
6     }
7     else {
8         System.out.println("ERRO: Saque na conta " + this.numero + "
            nao foi realizado. Valor disponivel: " + this.disponivel
            ());
9         return false;
10    }
11 }
```

Usando o método **sacar**, vamos implementar o método **transferir** de forma semelhante. A ideia é que o sistema tente realizar um saque seguido de um depósito. Entretanto, o depósito só vai ocorrer caso o saque tenha ocorrido com sucesso (ou seja, retornado verdadeiro). Caso contrário, o depósito não será feito:

```

1 boolean transferir(double valor, Conta destino) {
2     if (this.sacar(valor)) {
3         destino.depositar(valor);
4         System.out.println("Transferencia de " + valor + " da conta
5             " + this.numero + " para a conta " + destino.numero + "
6             foi realizado com sucesso.");
7         return true;
8     }
9     else {
10        System.out.println("ERRO: Nao foi possivel transferir " +
11            valor + " da conta " + this.numero + " para a conta " +
12            destino.numero + ". Valor disponivel: " + this.disponivel
13            ());
14        return false;
15    }
16 }

```

Repare que foi realizado um saque na segunda linha do código acima e que o **if** só será executado caso o saque tenha sido realizado com sucesso (ou seja, retornado verdadeiro). Caso contrário, será executado o trecho de código contido no **else**.

14.2 Testando o código até aqui

Para testar os novos métodos, apague o conteúdo do método **main** no arquivo principal do projeto, e adicione o seguinte:

```

1 Pessoa p1 = new Pessoa();
2 p1.nome = "Maria";
3 p1.idade = 18;
4 p1.sexo = 'F';
5 p1.cpf = "123.456.789-00";
6
7 Pessoa p2 = new Pessoa();
8 p2.nome = "Joao";
9 p2.idade = 22;
10 p2.sexo = 'M';
11 p2.cpf = "234.567.890-00";
12
13 Conta c1 = new Conta();
14 c1.numero = "1234-5";
15 c1.titular = p1;
16 c1.saldo = 100.0;
17 c1.limite = 200.0;
18
19 Conta c2 = new Conta();

```



```

20 c2.numero = "2345-6";
21 c2.titular = p2;
22 c2.saldo = 150.0;
23 c2.limite = 200.0;
24
25 c1.extrato();
26 c2.extrato();
27
28 c1.sacar(150);
29 c1.transferir(100, c2);
30 c1.sacar(100);
31 c1.depositar(100);
32 c1.transferir(200, c2);

```

Entenda cada linha do trecho de código acima, execute o projeto e verifique qual método imprimiu cada saída na tela.

14.3 Juros do cheque especial

Para terminar o exercício da Aula 2, é preciso implementar ainda o método **chequeEspecial** na classe **Conta**. Em seguida, não esqueça de testar o cheque especial, adicionando o seguinte código ao final do método principal da classe Main:

```

1 for(int i = 1; i <= 30; i++) {
2     c1.chequeEspecial(0.5);
3     System.out.println("Saldo apos " + i + " dia: " + c1.saldo);
4 }

```

A saída final do programa será:

```

*** EXTRATO DA CONTA ***
Conta: 1234-5
Titular: 123.456.789-00
Saldo disponivel para saque: 300.0
*** EXTRATO DA CONTA ***
Conta: 2345-6
Titular: 234.567.890-00
Saldo disponivel para saque: 350.0
Saque na conta 1234-5 realizado com sucesso. Novo saldo: -50.0
Saque na conta 1234-5 realizado com sucesso. Novo saldo: -150.0
Transferencia de 100.0 da conta 1234-5 para a conta 2345-6
foi realizado com sucesso.
ERRO: Saque na conta 1234-5 nao foi realizado. Valor disponivel: 50.0
ERRO: Saque na conta 1234-5 nao foi realizado. Valor disponivel: 150.0
ERRO: Nao foi possivel transferir 200.0 da conta 1234-5 para a conta 2345-6.
Valor disponivel: 150.0
Saldo apos 1 dia: -50.25
Saldo apos 2 dia: -50.50125
Saldo apos 3 dia: -50.75375625

```

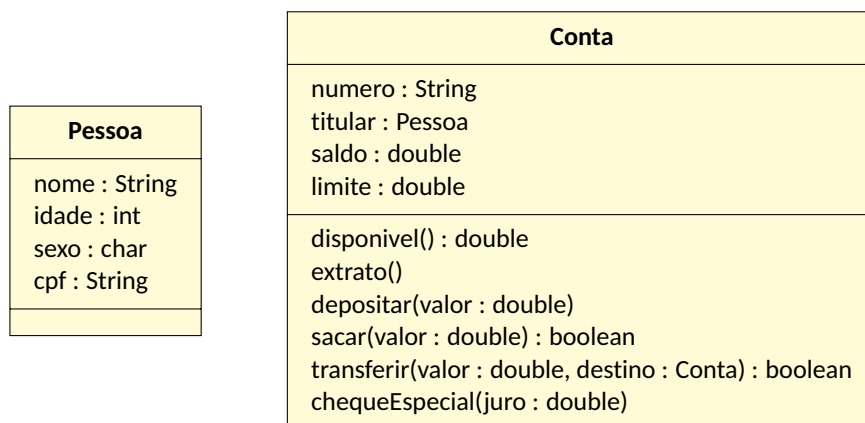
Saldo apos 4 dia: -51.007525031250005
Saldo apos 5 dia: -51.26256265640625
Saldo apos 6 dia: -51.51887546968828
Saldo apos 7 dia: -51.77646984703672
Saldo apos 8 dia: -52.03535219627191
Saldo apos 9 dia: -52.29552895725327
Saldo apos 10 dia: -52.557006602039536
Saldo apos 11 dia: -52.81979163504973
Saldo apos 12 dia: -53.08389059322498
Saldo apos 13 dia: -53.34931004619111
Saldo apos 14 dia: -53.61605659642206
Saldo apos 15 dia: -53.884136879404174
Saldo apos 16 dia: -54.15355756380119
Saldo apos 17 dia: -54.4243253516202
Saldo apos 18 dia: -54.6964469783783
Saldo apos 19 dia: -54.96992921327019
Saldo apos 20 dia: -55.24477885933654
Saldo apos 21 dia: -55.521002753633226
Saldo apos 22 dia: -55.79860776740139
Saldo apos 23 dia: -56.0776008062384
Saldo apos 24 dia: -56.357988810269596
Saldo apos 25 dia: -56.63977875432094
Saldo apos 26 dia: -56.92297764809255
Saldo apos 27 dia: -57.20759253633301
Saldo apos 28 dia: -57.49363049901468
Saldo apos 29 dia: -57.78109865150975
Saldo apos 30 dia: -58.0700041447673

Aula 15

Construtores - Prática

15.1 Retomando o projeto Banco

Mais uma vez, daremos continuidade ao projeto iniciado nas aulas passadas. Para começar, certifique-se de que suas classes Pessoa e Conta estão de acordo com os modelos a seguir:



Caso não tenha conseguido desenvolver o método **chequeEspecial** no último laboratório, segue um exemplo de como deve ser o código (implementado dentro da classe Conta):

```
1 void chequeEspecial(double juro) {  
2     if(this.saldo < 0) {  
3         this.saldo = this.saldo + (juro*this.saldo)/100;  
4     }  
5 }
```

15.2 Criando os construtores

Agora, vamos definir os construtores das classes Pessoa e Conta.

Primeiro, abra a classe Pessoa. O construtor deve ser inserido entre os atributos e os métodos da classe. Para esta classe, vamos definir que os valores de todos os atributos devem ser passados como parâmetro já na instânciação do objeto. Ou seja, não é possível criar uma nova Pessoa no sistema sem que o usuário indique seu nome, cpf, sexo e idade:

```
1 public class Pessoa {
2     String nome;
3     int idade;
4     char sexo;
5     String cpf;
6
7     public Pessoa(String nome, int idade, char sexo, String cpf) {
8         this.nome = nome;
9         this.idade = idade;
10        this.sexo = sexo;
11        this.cpf = cpf;
12    }
13
14    /* Metodos da classe Pessoa... */
15
16 }
```

Já a Conta vai ter dois construtores: o primeiro vai receber apenas dois parâmetros: o número da conta (String) e seu titular (objeto da classe Pessoa). Para os outros dois atributos (saldo e limite), definiremos um valor padrão: o saldo de toda conta começa zerado, enquanto o limite é sempre definido como 200.00 por padrão. O segundo recebe também o limite.

```
1 public class Conta {
2     String numero;
3     Pessoa titular;
4     double saldo, limite;
5
6     public Conta(String numero, Pessoa titular) {
7         this.numero = numero;
8         this.titular = titular;
9         this.saldo = 0;
10        this.limite = 200;
11    }
12
13    public Conta(String numero, Pessoa titular, double limite) {
14        this(numero, titular);
15        this.limite = limite;
16    }
17
18    /* Demais metodos... */
```

Pronto! Os dois construtores foram criados.

15.3 Testando os construtores

Repare que, agora, a classe com o método principal provavelmente estará cheia de erros. Isso acontecer porque não podemos mais utilizar o construtor padrão (aquele que não recebe nenhum parâmetro). Portanto, atualizar seu método **main** para que fique assim:

```
1 public static void main(String[] args) {
```

```

2  Pessoa p1 = new Pessoa("Maria", 18, 'F', "123.456.789-00");
3  Pessoa p2 = new Pessoa("Joao", 22, 'M', "234.567.890-00");
4
5  Conta c1 = new Conta("1234-5", p1);
6  Conta c2 = new Conta("2345-6", p2, 0);
7
8  c1.sacar(30);
9  c2.sacar(100);
10 c2.depositar(50);
11 c1.transferir(200, c2);
12
13 System.out.println(c2.saldo);
14 }

```

Repare que, com os construtores, o código principal fica bem mais enxuto e evita que o usuário crie objetos incorretos (por exemplo, uma conta que não pertence a nenhuma pessoa). Execute o programa, e verifique se as mensagens dos construtores estão sendo impressas.

15.4 Exercícios da Aula 3

Agora, vamos aos demais exercícios da Aula 3:

1. Implemente no Java no seu projeto uma nova classe chamada **Data**. A classe deve conter três atributos: dia, mês e ano (todos números inteiros).

```

1 public class Data {
2     int dia, mes, ano;
3 }

```

2. Implemente um construtor para a classe, que vai receber os três atributos como parâmetro.

```

1 public Data(int dia, int mes, int ano) {
2     this.dia = dia;
3     this.mes = mes;
4     this.ano = ano;
5 }

```

3. Implemente e teste um método na classe **Data** chamado **imprimir** que imprima a data no formato D/M/A. Exemplo: "23/10/2018".

```

1 void imprimir() {
2     System.out.println(this.dia + "/" + this.mes + "/" + this.ano)
3     ;
4 }

```

4. Altere a classe **Pessoa**, para que o objeto possua um atributo **dtNasc** com sua data de nascimento, ao invés da idade.

```

1 public class Pessoa {
2     String nome;
3     Data dtNasc;

```

```
4     char sexo;
5     String cpf;
6
7     public Pessoa(String nome, int dia, int mes, int ano, char
        sexo, String cpf) {
8         this.nome = nome;
9         this.dtNasc = new Data(dia, mes, ano);
10        this.sexo = sexo;
11        this.cpf = cpf;
12    }
13 }
```

5. Faça um método na classe Pessoa que receba a data de hoje como parâmetro, e calcule e retorne a idade desta pessoa.

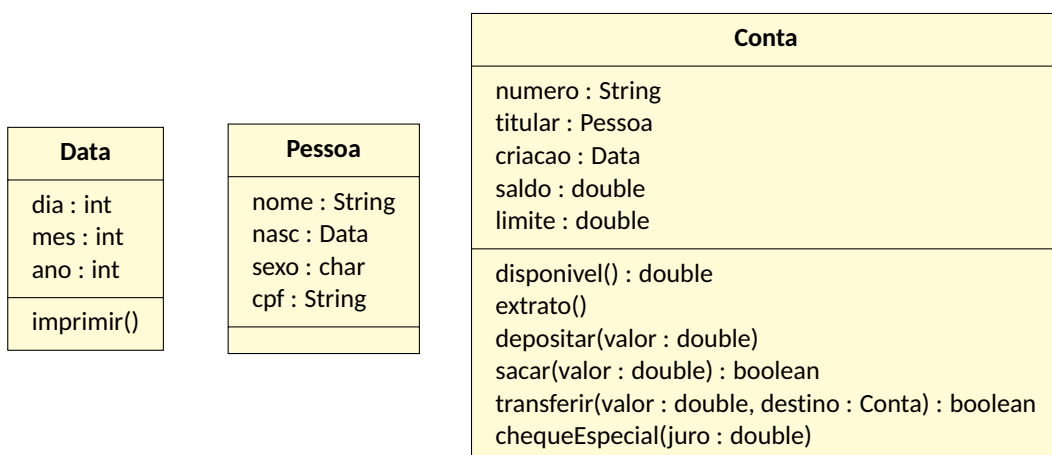
```
1 int idade(Data dataAtual) {
2     int x = dataAtual.ano - this.dtNasc.ano;
3
4     if (dataAtual.mes < this.dtNasc.mes) return x - 1;
5     if (dataAtual.mes > this.dtNasc.mes) return x;
6
7     if (dataAtual.dia < this.dtNasc.dia) return x - 1;
8     return x;
9 }
```

Aula 16

Herança - Prática

16.1 Retomando o projeto Banco

Mais uma vez, daremos continuidade ao projeto iniciado nas aulas passadas. Desta vez, vamos alterar as classes do projeto para utilizar heranças. Atualmente, seu projeto reflete as classes do modelo a seguir:



16.2 Adicionando herança às classes

Agora, utilizaremos herança para criar três novas classes:

1. **Gerente:** Subclasse de Pessoa. Um gerente é uma pessoa e também possui um número de matrícula e uma senha.

```
1 public class Gerente extends Pessoa {  
2     String matricula;  
3     String senha;  
4 }
```

2. **ContaCorrente:** Subclasse de Conta. Apenas uma conta corrente possui limite, além do método chequeEspecial.

```
1 public class ContaCorrente extends Conta {
```

```

2     double limite;
3 }

```

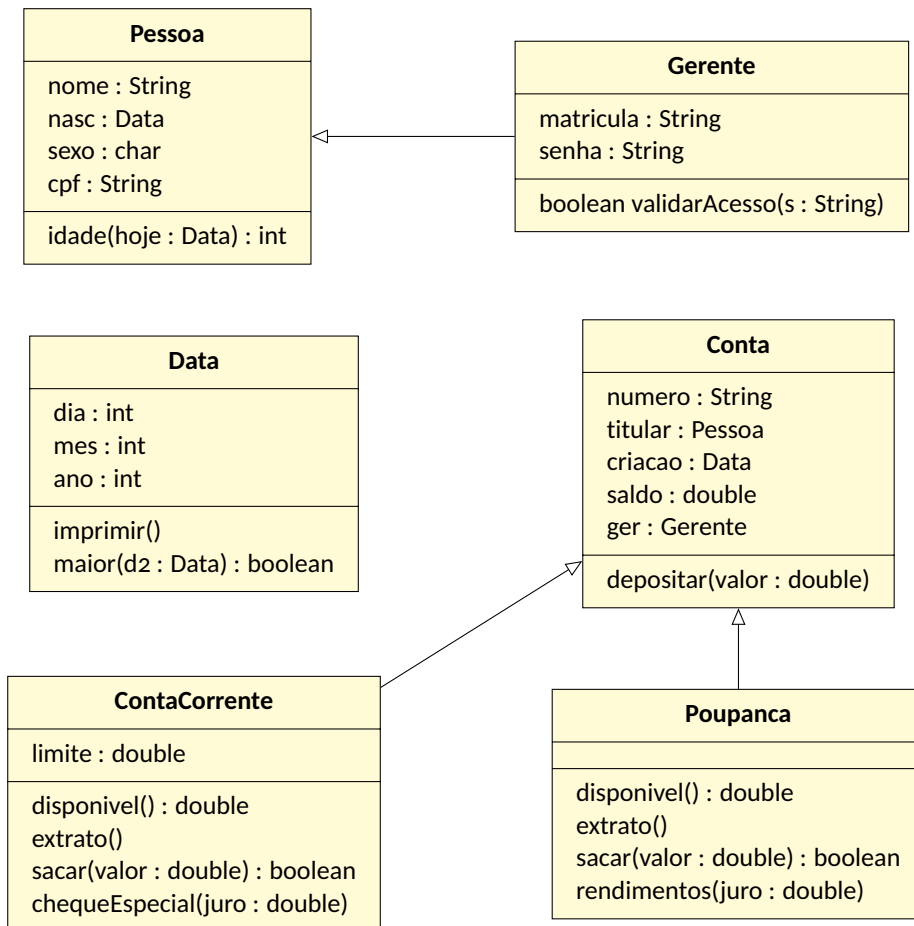
3. **Poupanca**: Subclasse de Conta. Apenas uma poupança possui rendimentos.

```

1 public class Poupanca extends Conta{
2 }

```

Portanto, certifique-se que suas classes a partir de agora reflitam **exatamente** o diagrama a seguir:



Os construtores das classes devem receber os seguintes parâmetros:

- Data: dia, mês e ano.
 - Pessoa: nome, data de nascimento, sexo e cpf.
 - Gerente: nome, data de nascimento, sexo, cpf, número de matrícula e senha.
- ```

1 Gerente(String nome, int dia, int mes, int ano, char sexo, String
 cpf, String matricula, String senha) {
2 super(nome, dia, mes, ano, sexo, cpf);
3 this.matricula = matricula;
4 this.senha = senha;
5 }

```



- Conta: número, titular, gerente e data de criação. O saldo inicia zerado.

```

1 public class Conta {
2 String numero;
3 Pessoa titular;
4 double saldo;
5 Data criacao;
6 Gerente ger;
7
8 public Conta(String numero, Pessoa titular, Gerente ger, int
 dia, int mes, int ano) {
9 this.numero = numero;
10 this.titular = titular;
11 this.ger = ger;
12 this.criacao = new Data(dia, mes, ano);
13 this.saldo = 0;
14 }
15 }

```

- ContaCorrente: número, titular, gerente e data de criação. O limite a princípio é 200.0.

```

1 ContaCorrente(String numero, Pessoa titular, Gerente ger, int dia,
 int mes, int ano, double limite) {
2 super(numero, titular, ger, dia, mes, ano);
3 this.limite = limite;
4 }

```

- Poupanca: número, titular, gerente e data de criação.

```

1 Poupanca(String numero, Pessoa titular, Gerente ger, int dia, int
 mes, int ano) {
2 super(numero, titular, ger, dia, mes, ano);
3 }

```

A seguir, uma breve explicação sobre alguns métodos a serem implementados:

- O método **idade** da classe Pessoa deve receber a data de hoje como parâmetro e retornar a idade da pessoa.

```

1 int idade(Data dataAtual) {
2 int x = dataAtual.ano - this.dtNasc.ano;
3
4 if (dataAtual.mes < this.dtNasc.mes) return x - 1;
5 if (dataAtual.mes > this.dtNasc.mes) return x;
6
7 if (dataAtual.dia < this.dtNasc.dia) return x - 1;
8 return x;
9 }

```

- O método **validarAcesso** da classe Gerente deve receber uma senha como parâmetro e verificar se ela está correta.

```
1 boolean validarAcesso(String senha) {
2 return (this.senha.equals(senha));
3 }
```

- O método **maior** da classe Data deve receber outra data d2 como parâmetro de verificar se a data **this** é maior que a data d2 (ou seja, se ela aconteceu após a data d2).

```
1 boolean maior(Data data) {
2 if (this.ano > data.ano) return true;
3 if (this.ano < data.ano) return false;
4 if (this.mes > data.mes) return true;
5 if (this.mes < data.mes) return false;
6 return this.dia > data.dia;
7 }
```

- O método **rendimentos** da classe Poupanca recebe uma taxa de juros como parâmetro e adiciona essa porcentagem ao saldo da poupança.

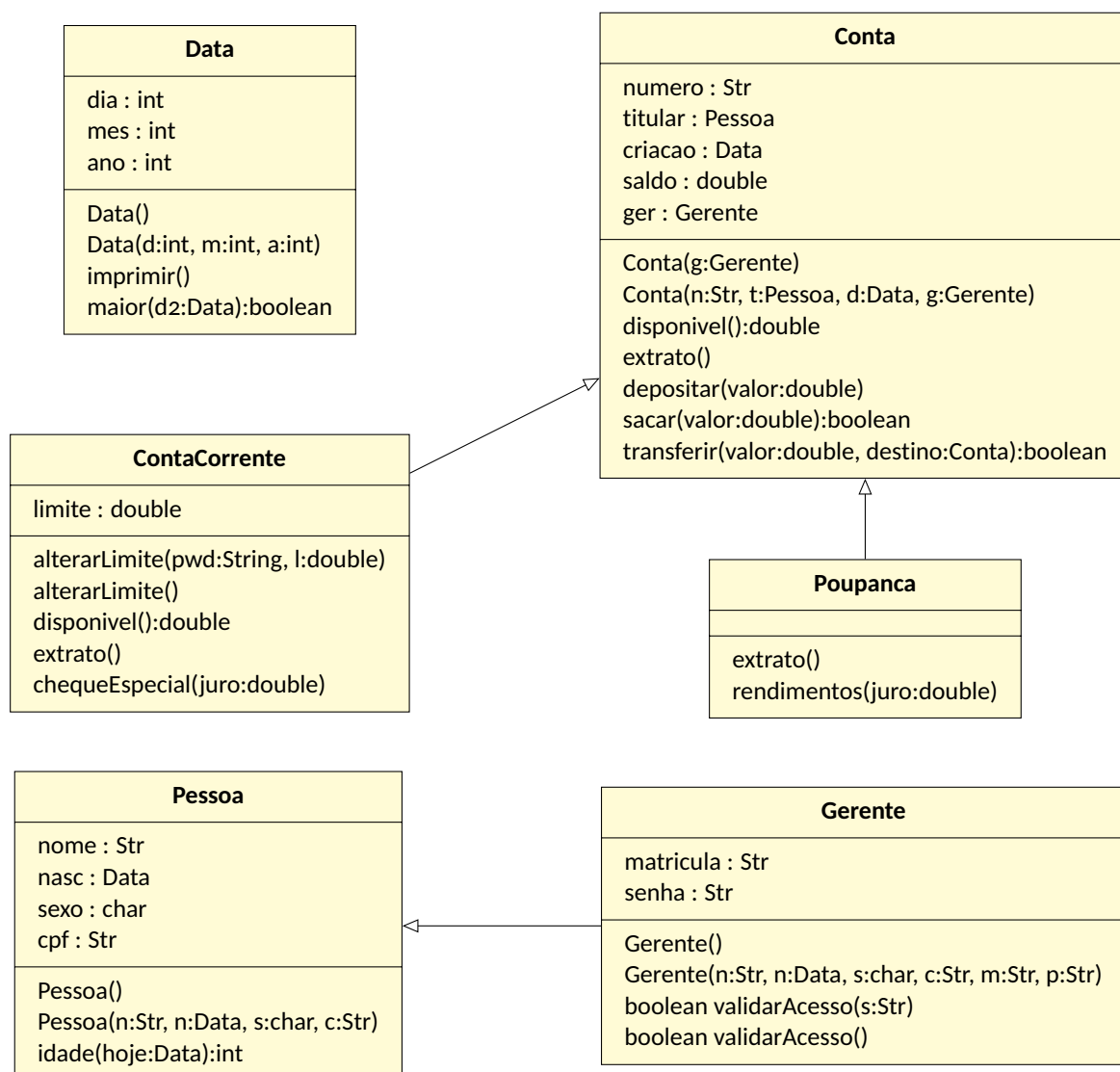
```
1 void rendimentos(double juro) {
2 this.saldo += (this.saldo * juro / 100);
3 }
```

## Aula 17

# Reescrita de métodos, Sobrecarga de Métodos, E/S - Prática

O objetivo desta aula prática é incorporar os conceitos de reescrita de métodos, sobrecarga de métodos e entrada de dados ao projeto do Sistema Bancário.

O novo sistema deve refletir o diagrama a seguir:



**Observações:**

1. Os construtores sem parâmetros das classes Pessoa e Data devem solicitar que o usuário informe **todos** os seus atributos.

```

1 Pessoa() {
2 Scanner s = new Scanner(System.in);
3
4 System.out.println("Digite o nome: ");
5 this.nome = s.nextLine();
6
7 System.out.println("Digite o cpf: ");
8 this.nome = s.nextLine();
9
10 System.out.println("Data de nascimento: ");
11 this.dtNasc = new Data();
12
13 System.out.println("Digite o sexo (M/F): ");
14 this.sexo = s.next().charAt(0);
15 }

1 Data() {
2 Scanner s = new Scanner(System.in);
3
4 System.out.println("Digite o dia: ");
5 this.dia = s.nextInt();
6
7 System.out.println("Digite o mes: ");
8 this.mes = s.nextInt();
9
10 System.out.println("Digite o ano: ");
11 this.ano = s.nextInt();
12 }

```

2. O construtor sem parâmetros da classe Gerente deve solicitar que o usuário informe todos os seus atributos, exceto a senha (que deve assumir um valor padrão – exemplo: “123456”).

```

1 Gerente() {
2 super();
3
4 Scanner s = new Scanner(System.in);
5
6 System.out.println("Matricula: ");
7 this.matricula = s.nextLine();
8
9 System.out.println("Senha: ");
10 this.senha = s.nextLine();
11 }

```

3. Na classe Gerente, o método **validarAcesso()** precisa comparar duas Strings. Em Java, para verificar se duas Strings *s1* e *s2* são iguais, usamos *s1.equals(s2)*. Não é correto utilizar *s1 == s2*. O método **validaraAcesso()** da classe Gerente que não recebe a senha como parâmetro deve solicitar que o usuário digite-a.

```

1 boolean validarAcesso(String senha) {
2 return (this.senha.equals(senha));
3 }
4
5 boolean validarAcesso() {
6 Scanner s = new Scanner(System.in);
7 String senha = s.nextLine();
8 return (this.validarAcesso(senha));
9 }

```

4. O construtor da classe Conta que recebe apenas o Gerente como parâmetro deve solicitar que o usuário informe os outros atributos e criar um novo objeto para armazenar o titular da conta. Apenas o saldo não deve ser digitado, pois deve começar sempre zerado. Não é possível criar uma Conta sem um Gerente associado a ela.

```

1 Conta(Gerente ger) {
2 Scanner s = new Scanner(System.in);
3
4 System.out.println("Digite o numero da conta: ");
5 this.numero = s.nextLine();
6
7 System.out.println("Titular da conta:");
8 this.titular = new Pessoa();
9
10 System.out.println("Digite o saldo: ");
11 this.saldo = s.nextDouble();
12
13 System.out.println("Data de criacao: ");
14 this.criacao = new Data();
15
16 this.ger = ger;
17 }

```

```

1 ContaCorrente(Gerente ger) {
2 super(ger);
3 Scanner s = new Scanner(System.in);
4 this.limite = s.nextDouble();
5 }

```

```

1 Poupanca(Gerente ger) {
2 super(ger);
3 }

```

5. Com a reescrita, os métodos **disponivel()**, **extrato()**, **sacar()** e **transferir()** ficam apenas na superclasse Conta, evitando códigos duplicados. O que fará com que os comportamentos deles sejam diferentes numa ContaCorrente é o fato de reescrevermos o método **disponivel()** nesta subclasse.
6. O método **alterarLimite()** de uma ContaCorrente só deve executar a ação após validar o acesso do gerente, utilizando a senha recebida como parâmetro. O método homônimo que não recebe parâmetros deve fazer a leitura tanto da senha como do novo limite.

```

1 void alterarLimite(String senha, double limite) {

```

```

2 if (this.ger.validarAcesso(senha)) {
3 this.limite = limite;
4 System.out.println("Senha alterada com sucesso.");
5 }
6 else {
7 System.out.println("ERRO: Senha invalida.");
8 }
9 }
10
11 void alterarLimite() {
12 Scanner s = new Scanner(System.in);
13
14 System.out.println("Digite a senha: ");
15 String senha = s.nextLine();
16
17 System.out.println("Digite o novo limite: ");
18 double limite = s.nextDouble();
19
20 this.alterarLimite(senha, limite);
21 }

```

7. Repare que o método **extrato()** deverá ser reescrito tanto na classe *ContaCorrente* quanto na *Poupanca*. A ideia é a seguinte: o método da suuperclasse imprime as informações necessárias sobre titular, número da conta e saldo disponível. Os métodos reescrito nas subclasses imprimem um cabeçalho indicando o tipo de conta (se é um extrato de conta corrente ou de uma poupança), e em seguida executa o método da superclasse para imprimir as demais informações. Note que **não devemos repetir o código** que imprime titular, número e saldo da conta. Para que um método reescrito execute o método homônimo da superclasse, utilizamos a palavra reservada **super**. Por exemplo, para que o método **extrato()** na *ContaCorrente* execute o método homônimo que foi reescrito, utilizamos o comando **super.extrato()**. Com isso, os extratos serão diferenciados da seguinte forma:

```

*** EXTRATO DE CONTA-CORRENTE ***
Titular: Joao da Silva (CPF: 123.456.789-00)
Número da Conta: 1234-5
Saldo disponível para saque: R$ 250.00

*** EXTRATO DE POUPANCA ***
Titular: Maria Joaquina (CPF: 234.567.890-00)
Número da Conta: 2345-6
Saldo disponível para saque: R$ 50.00

```

```

1 void extrato() {
2 System.out.println("*** EXTRATO DE CONTA CORRENTE ***");
3 super.extrato();
4 }

```

8. Por fim, seu método principal deve criar quatro objetos da classe *Pessoa*, dois da classe *Gerente*, dois da classe *ContaCorrente* e dois da classe *Poupanca*. Execute saques, depósitos, transferências e extratos em todas as contas, e verifique se o resultado está correto.

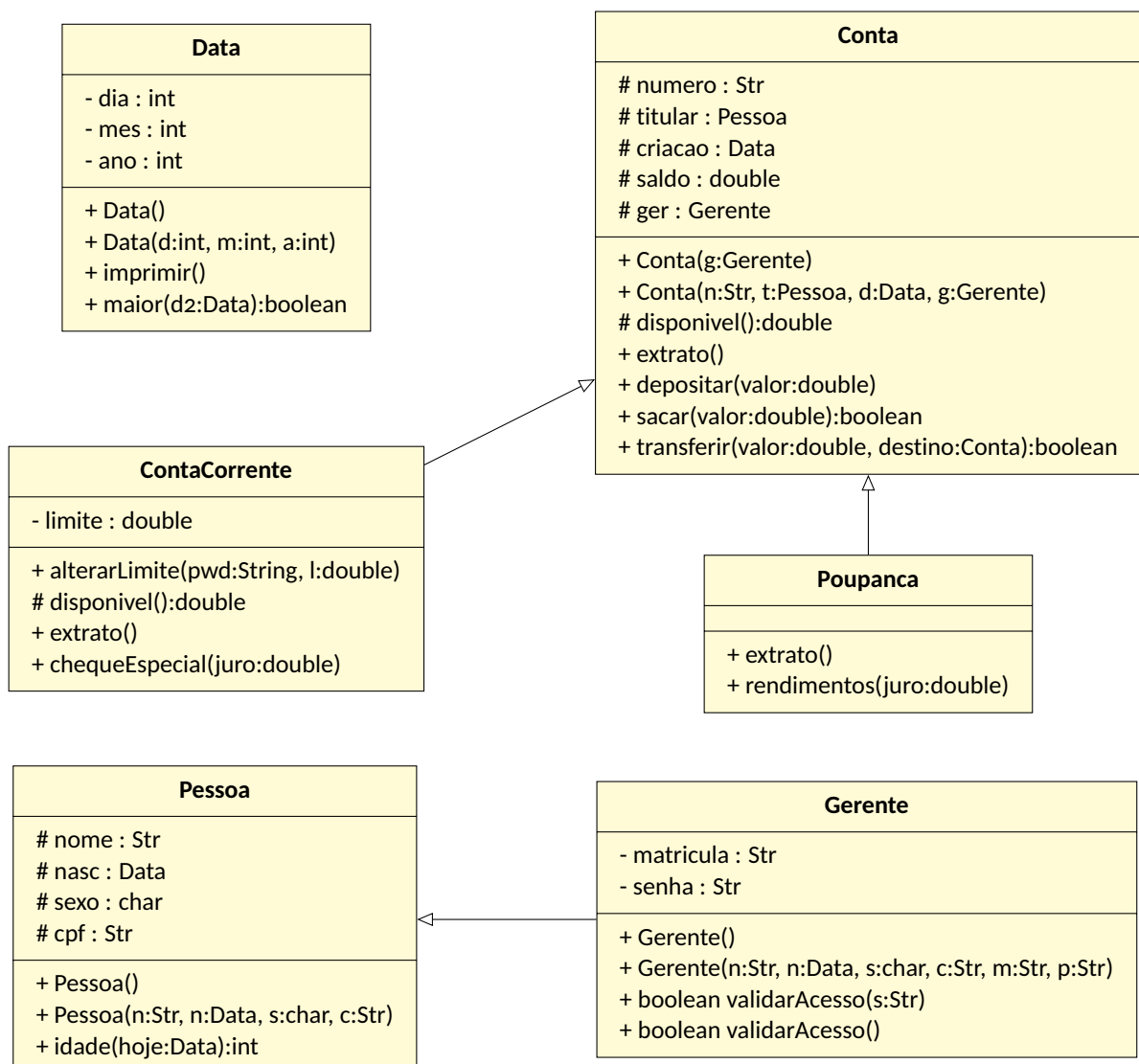
```
1 public static void main(String[] args) {
2 Scanner s = new Scanner(System.in);
3
4 System.out.println("Criando dois gerentes:");
5 Gerente g1 = new Gerente();
6 Gerente g2 = new Gerente();
7
8 System.out.println("Criando duas contas correntes:");
9 ContaCorrente c1 = new ContaCorrente(g1);
10 ContaCorrente c2 = new ContaCorrente(g2);
11
12 System.out.println("Criando duas poupanças:");
13 Poupanca p1 = new Poupanca(g1);
14 Poupanca p2 = new Poupanca(g2);
15
16 c1.sacar(30);
17 p1.sacar(100);
18 p1.depositar(50);
19 c1.transferir(200, p1);
20 c2.transferir(50, p2);
21
22 c1.extrato();
23 c2.extrato();
24 p1.extrato();
25 p2.extrato();
26 }
```

# Aula 18

## Encapsulamento - Prática

### 18.1 Retomando o projeto Banco

Nesta aula prática, vamos alterar pela última vez as classes Data, Pessoa, Gerente, Conta, ContaCorrente e Poupanca, encapsulando seus dados. Os modificadores de acessos devem estar de acordo com o diagrama a seguir:





```

1 public class Pessoa {
2 protected String nome;
3 protected Data dtNasc;
4 protected char sexo;
5 protected String cpf;
6
7 // Construtores e Metodos da classe...
8 }

1 public class Gerente extends Pessoa {
2 private String matricula;
3 private String senha;
4
5 // Construtores e Metodos da classe...
6 }

1 public class Conta {
2 protected String numero;
3 protected Pessoa titular;
4 protected double saldo;
5 protected Data criacao;
6 protected Gerente ger;
7
8 // Construtores e Metodos da classe...
9 }

1 public class ContaCorrente extends Conta {
2 private double limite;
3
4 // Construtores e Metodos da classe...
5 }

```

Entretanto, alguns erros podem surgir no código após encapsular os dados. O método **idade(Data hoje)** da classe **Pessoa** recebe a data de hoje como parâmetro e tenta acessar o atributo **hoje.dia**, por exemplo. Isso não será possível, já que os atributos de **Data** agora serão privados. Por isso, é necessário inserir um método para permitir na classe **Data** que retorne este valor:

```

1 import java.util.Scanner;
2
3 public class Data {
4 private int dia;
5 private int mes;
6 private int ano;
7
8 // Construtores...
9
10 public void imprimir() {
11 System.out.println(this.getDia() + "/" + this.getMes() + "/"
12 + this.getAno());
13 }
14
15 public boolean maior(Data data) {

```

```

15 if (this.getAno() > data.getAno()) return true;
16 if (this.getAno() < data.getAno()) return false;
17 if (this.getMes() > data.getMes()) return true;
18 if (this.getMes() < data.getMes()) return false;
19 return this.getDia() > data.getDia();
20 }
21
22 public int getDia() {
23 return dia;
24 }
25
26 public void setDia(int dia) {
27 this.dia = dia;
28 }
29
30 // Demais getters e setters
31 }

```

Assim, quando o método **idade(Data hoje)** precisar acessar o dia de hoje, ele o fará através do comando **hoje.getDia()** ao invés de **hoje.dia**.

Você pode usar a ajuda do IntelliJ para encapsular os dados. Deixe o cursor do teclado na área dos atributos da classe, aperte o botão direito do mouse e escolha as opções **Refactor** e depois **Encapsulate Fields**.

Depois, escolha os atributos que quer encapsular, se deseja criar getters e setters pra eles, o modificador de acesso dos atributos (no exemplo, private) e o modificador de acesso dos getters e setters que forem criados (no exemplo, public).

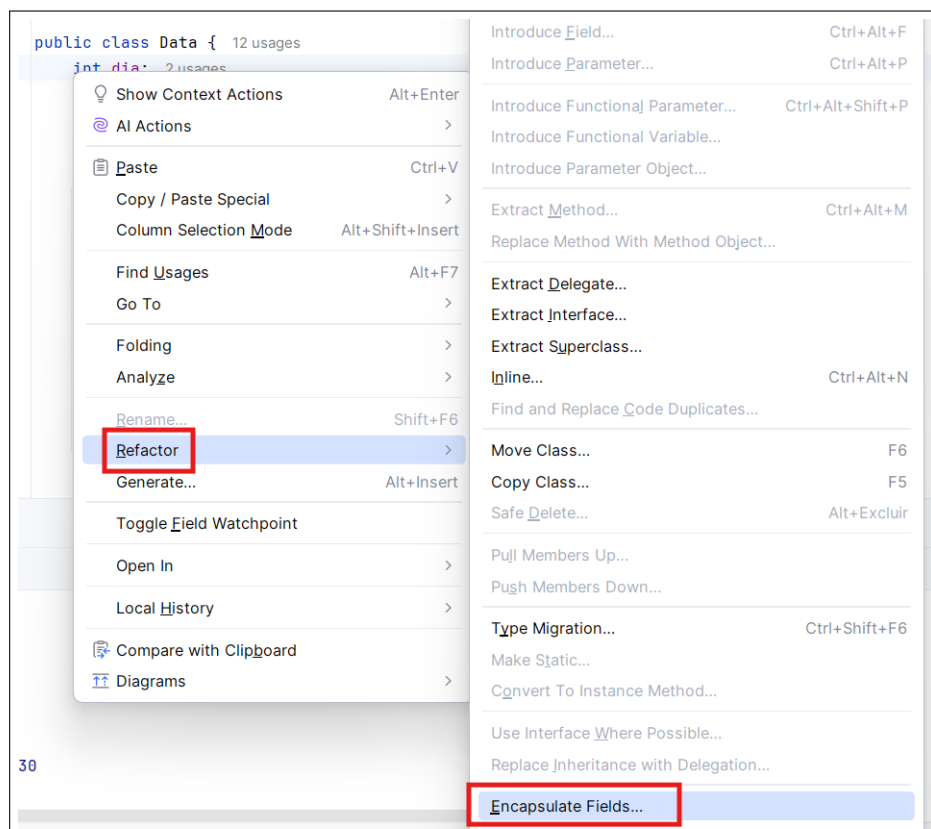


Figura 18.1: Encapsulando os dados.

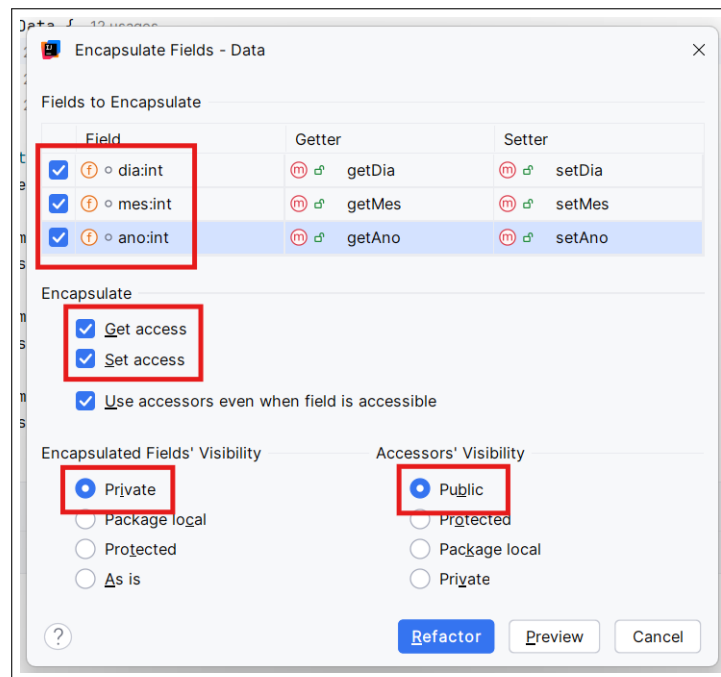


Figura 18.2: Escolhendo os modificadores de acesso.

# Aula 19

## Array e ArrayList - Prática

Neste capítulo, vamos adicionar um menu interativo ao sistema, e armazenar clientes, gerentes e contas em ArrayLists. Primeiro, vamos criar uma nova classe Sistema, onde os ArrayLists serão armazenados. Em seguida, criaremos uma classe Entrada, que cuidará de toda a interação com o usuário.

### 19.1 Classe Sistema

A classe Sistema irá armazenar os clientes, gerentes, contas correntes e poupanças. Haverá três ArrayLists: um para clientes, um para gerentes e apenas um para contas (que irá conter tanto as contas correntes como as poupanças).

```
1 public class Sistema {
2 private ArrayList<Conta> contas = new ArrayList<>();
3 private ArrayList<Gerente> gerentes = new ArrayList<>();
4 private ArrayList<Pessoa> clientes = new ArrayList<>();
5
6 public Sistema() {
7 this.gerentes = new ArrayList<>();
8 this.clientes = new ArrayList<>();
9 this.contas = new ArrayList<>();
10 }
11 }
```

Como os atributos estão privados, a classe precisa de métodos para inserir novas informações em cada um dos ArrayLists, e para retornar cada um deles.

```
1 public void cadastrarConta(Conta conta) {
2 this.contas.add(conta);
3 }
4
5 public void cadastrarGerente(Gerente gerente) {
6 this.gerentes.add(gerente);
7 }
8
9 public void cadastrarCliente(Pessoa cliente) {
10 this.clientes.add(cliente);
11 }
```

```
12
13 public ArrayList<Conta> getContas() {
14 return contas;
15 }
16
17 public ArrayList<Gerente> getGerentes() {
18 return gerentes;
19 }
20
21 public ArrayList<Pessoa> getClientes() {
22 return clientes;
23 }
```

## 19.2 Classe Entrada

A classe Entradda vai ser responsável por cuidar de toda a interação do sistema com o usuário. Todos os seus métodos são estáticos, ou seja, não é preciso instanciar a classe para que seus métodos sejam executados. Eles são criados utilizando a palavra reservada `static`. A classe tem métodos para fazer a leitura de dados de forma padronizada.

```
1 public class Entrada {
2
3 public static String lerLinha(String msg, Scanner input) {
4 // Imprime uma mensagem ao usuario, le uma linha do teclado
5 // e retorna esta linha
6 System.out.print(msg);
7 String linha = input.nextLine();
8 return linha;
9 }
10
11 public static int lerInteiro(String msg, Scanner input) {
12 // Imprime uma mensagem ao usuario, le uma linha contendo um
13 // inteiro e retorna este inteiro
14 String linha = Entrada.lerLinha(msg, input);
15 return Integer.parseInt(linha);
16 }
17
18 private static double lerDouble(String msg, Scanner input) {
19 // Imprime uma mensagem ao usuario, le uma linha contendo um
20 // double e retorna este double
21 String linha = Entrada.lerLinha(msg, input);
22 return Double.parseDouble(linha);
23 }
24 }
```

Os métodos recebem como parâmetro uma String a ser exibida na tela e um objeto da classe Scanner, e retornam o dado que foi lido do teclado. O objeto da classe Scanner será criado apenas uma vez, no main, para que seja utilizada uma entrada de dados única para todo o sistema.

## 19.3 Menu interativo no projeto do Banco

A classe Entrada também vai ter métodos para a leitura dos menus do sistema. São três menus: menu inicial, menu com opções de cadastro de dados e menu com opções de movimentações financeiras:

Menu principal:

```

1 public static int menu(Scanner input) {
2 // Imprime o menu principal, le a opcao escolhida pelo usuario e
 retorna a opcao selecionada.
3
4 String msg = "*****\n" +
5 "Escolha uma opcao:\n" +
6 "1) Cadastro\n" +
7 "2) Movimentacoes financeiras\n" +
8 "0) Sair\n";
9
10 int op = Entrada.lerInteiro(msg, input);
11
12 while (op < 0 || op > 2) {
13 System.out.println("Opcao invalida. Tente novamente: ");
14 op = Entrada.lerInteiro(msg, input);
15 }
16
17 return op;
18 }
```

Menu com as opções para cadastro de clientes, gerentes, contas correntes e poupanças:

```

1 public static void menu_cadastro(Sistema s, Scanner input) {
2 String msg = "*****\n" +
3 "1) Adicionar pessoa\n" +
4 "2) Adicionar gerente\n" +
5 "3) Criar conta corrente\n" +
6 "4) Criar poupanca\n";
7
8 int op = Entrada.lerInteiro(msg, input);
9
10 switch(op) {
11 case 1:
12 Entrada.cadCliente(s, input);
13 break;
14 case 2:
15 Entrada.cadGerente(s, input);
16 break;
17 case 3:
18 Entrada.cadContaCorrente(s, input);
19 break;
20 case 4:
21 Entrada.cadPoupanca(s, input);
22 break;
23 default:
```

```

24 System.out.println("Opcao Invalida.");
25 }
26
27 }

```

Menu com as opções de movimentações financeiras (extrato, depósitos, saques e transferências):

```

1 public static void menu_movimentacoes(Sistema s, Scanner input) {
2 String msg = "*****\n" +
3 "1) Verificar extrato\n" +
4 "2) Depositar\n" +
5 "3) Sacar\n" +
6 "4) Transferir\n";
7
8 int op = Entrada.lerInteiro(msg, input);
9
10 switch(op) {
11 case 1:
12 Entrada.extrato(s, input);
13 break;
14 case 2:
15 Entrada.depositar(s, input);
16 break;
17 case 3:
18 Entrada.sacar(s, input);
19 break;
20 case 4:
21 Entrada.transferir(s, input);
22 break;
23 default:
24 System.out.println("Opcao Invalida.");
25 }
26
27 }

```

Cada opção dos menus de cadastro e de movimentações financeiras utilizam métodos auxiliares implementados na própria classe Entrada, para centralizar a interação do sistema com o usuário.

## 19.4 Métodos para cadastros

Primeiro, há um método para cadastro de clientes e outro para cadastro de gerentes:

```

1 public static void cadCliente(Sistema s, Scanner input) {
2 String nome = Entrada.lerLinha("Digite o nome do cliente: ",
3 input);
4 String cpf = Entrada.lerLinha("Digite o cpf do cliente: ", input
5);
6 int dia = Entrada.lerInteiro("Digite o dia do nascimento do
7 cliente: ", input);

```

```

5 int mes = Entrada.lerInteiro("Digite o mes do nascimento do
 cliente: ", input);
6 int ano = Entrada.lerInteiro("Digite o ano do nascimento do
 cliente: ", input);
7 char sexo = Entrada.lerLinha("Digite o sexo do cliente: ", input
).charAt(0);
8
9 s.cadastrarCliente(new Pessoa(nome, dia, mes, ano, sexo, cpf));
10 }

1 public static void cadGerente(Sistema s, Scanner input) {
2 String nome = Entrada.lerLinha("Digite o nome do cliente: ",
 input);
3 String cpf = Entrada.lerLinha("Digite o cpf do cliente: ", input
);
4 int dia = Entrada.lerInteiro("Digite o dia do nascimento do
 cliente: ", input);
5 int mes = Entrada.lerInteiro("Digite o mes do nascimento do
 cliente: ", input);
6 int ano = Entrada.lerInteiro("Digite o ano do nascimento do
 cliente: ", input);
7 char sexo = Entrada.lerLinha("Digite o sexo do cliente: ", input
).charAt(0);
8 String mat = Entrada.lerLinha("Digite a matricula do cliente: ",
 input);
9 String senha = Entrada.lerLinha("Digite a senha do cliente: ",
 input);
10
11 s.cadastrarGerente(new Gerente(nome, dia, mes, ano, sexo, cpf,
 mat, senha));
12 }

```

Repare que toda a entrada de dados está sendo feita na classe Entrada, então não são mais necessário os construtores que havíamos criado que não recebiam parâmetros e faziam a leitura de dados do teclado. Só é necessário agora deixar nas classes os construtores que recebem os parâmetros já lidos, pois esta tarefa será concentrada na classe Entrada.

Para o cadastro das contas, há métodos auxiliares para selecionar um clientes e um gerente dentre os que já foram cadastrados:

```

1 public static Pessoa escolherCliente(ArrayList<Pessoa> clientes,
 Scanner input) {
2 System.out.println("Clientes cadastrados: ");
3 int index = 0;
4 for (Pessoa cliente : clientes) {
5 System.out.println(index + ") " + cliente.getCpf() + " - " +
 cliente.getNome());
6 index++;
7 }
8 index = Entrada.lerInteiro("Escolha o cliente pelo numero: ",
 input);
9

```



```

10 while (index < 0 || index >= clientes.size()) {
11 System.out.println("Cliente invalido. Tente novamente: ");
12 index = Entrada.lerInteiro("Escolha o cliente pelo numero: ",
13 input);
14 }
15 return clientes.get(index);
16 }

1 public static Gerente escolherGerente(ArrayList<Gerente> gerentes,
2 Scanner input) {
3 System.out.println("Gerentes cadastrados: ");
4 int index = 0;
5 for (Gerente g : gerentes) {
6 System.out.println(index + ") " + g.getCpf() + " - " + g.
7 getNome());
8 index++;
9 }
10 index = Entrada.lerInteiro("Escolha o gerente pelo numero: ",
11 input);
12
13 while (index < 0 || index >= gerentes.size()) {
14 System.out.println("Gerente invalido. Tente novamente: ");
15 index = Entrada.lerInteiro("Escolha o gerente pelo numero: "
16 , input);
17 }
18
19 return gerentes.get(index);
20 }

```

Estes métodos são necessário para que as contas correntes e poupanças sejam cadastradas:

```

1 public static void cadContaCorrente(Sistema s, Scanner input) {
2 ArrayList<Pessoa> clientes = s.getClientes();
3 ArrayList<Gerente> gerentes = s.getGerentes();
4 if (clientes.isEmpty() || gerentes.isEmpty()) {
5 System.out.println("E preciso haver clientes e gerentes
6 cadastrados no sistema para criar uma conta corrente.");
7 }
8 else {
9 String numero = Entrada.lerLinha("Digite o numero da conta
10 corrente: ", input);
11 Pessoa cliente = Entrada.escolherCliente(clientes, input);
12 Gerente gerente = Entrada.escolherGerente(gerentes, input);
13 int dia = Entrada.lerInteiro("Digite o dia de criacao da
14 conta: ", input);
15 int mes = Entrada.lerInteiro("Digite o mes de criacao da
16 conta: ", input);
17 int ano = Entrada.lerInteiro("Digite o ano de criacao da
18 conta: ", input);
19 double limite = Entrada.lerDouble("Digite o limite da conta:
20 ", input);

```

```

15
16 s.cadastrarConta(new ContaCorrente(numero, cliente, gerente,
17 dia, mes, ano, limite));
18 }

1 public static void cadPoupanca(Sistema s, Scanner input) {
2 ArrayList<Pessoa> clientes = s.getClientes();
3 ArrayList<Gerente> gerentes = s.getGerentes();
4 if (clientes.isEmpty() || gerentes.isEmpty()) {
5 System.out.println("E preciso haver clientes e gerentes
6 cadastrados no sistema para criar uma poupanca.");
7 }
8 else {
9 String numero = Entrada.lerLinha("Digite o numero da
10 poupanca: ", input);
11 Pessoa cliente = Entrada.escolherCliente(clientes, input);
12 Gerente gerente = Entrada.escolherGerente(gerentes, input);
13 int dia = Entrada.lerInteiro("Digite o dia de criacao da
14 poupanca: ", input);
15 int mes = Entrada.lerInteiro("Digite o mes de criacao da
16 poupanca: ", input);
17 int ano = Entrada.lerInteiro("Digite o ano de criacao da
18 poupanca: ", input);

19 s.cadastrarConta(new Poupanca(numero, cliente, gerente, dia,
20 mes, ano));
21 }
22 }

```

## 19.5 Métodos para movimentações financeiras

É preciso um método auxiliar para escolher a conta em que a movimentação financeira será realizada:

```

1 public static Conta escolherConta(ArrayList<Conta> contas, Scanner
2 input) {
3 System.out.println("Contas cadastradas: ");
4 int index = 0;
5 for (Conta c : contas) {
6 System.out.println(index + ") " + c.getNumero() + " (Titular
7 : " + c.getTitular().getNome() + ")");
8 index++;
9 }
10 index = Entrada.lerInteiro("Escolha a conta pelo indice: ",
11 input);

12 while (index < 0 || index >= contas.size()) {
13 System.out.println("Conta invalido. Tente novamente: ");
14 }
15 }

```

```
12 index = Entrada.lerInteiro("Escolha a conta pelo indice: ",
13 input);
14 }
15 return contas.get(index);
16 }
```

Agora, por fim, um método para cada opção de movimentação financeira:

```
1 public static void extrato(Sistema s, Scanner input) {
2 ArrayList<Conta> contas = s.getContas();
3
4 if (contas.isEmpty()) {
5 System.out.println("E preciso haver contas cadastradas no
6 sistema para exibir um extrato.");
7 }
8 else {
9 Conta c = Entrada.escolherConta(contas, input);
10 c.extrato();
11 }
12 }
```

```
1 public static void depositar(Sistema s, Scanner input) {
2 ArrayList<Conta> contas = s.getContas();
3
4 if (contas.isEmpty()) {
5 System.out.println("E preciso haver contas cadastradas no
6 sistema para fazer um deposito.");
7 }
8 else {
9 Conta c = Entrada.escolherConta(contas, input);
10 double valor = Entrada.lerDouble("Digite o valor a ser
11 depositado: ", input);
12 c.depositar(valor);
13 }
14 }
```

```
1 public static void sacar(Sistema s, Scanner input) {
2 ArrayList<Conta> contas = s.getContas();
3
4 if (contas.isEmpty()) {
5 System.out.println("E preciso haver contas cadastradas no
6 sistema para fazer um saque.");
7 }
8 else {
9 Conta c = Entrada.escolherConta(contas, input);
10 double valor = Entrada.lerDouble("Digite o valor a ser
11 sacado: ", input);
12 c.sacar(valor);
13 }
14 }
```

```
1 public static void transferir(Sistema s, Scanner input) {
2 ArrayList<Conta> contas = s.getContas();
3
4 if (contas.isEmpty()) {
5 System.out.println("E preciso haver contas cadastradas no
6 sistema para fazer uma transferencia.");
7 }
8 else {
9 Conta c1 = Entrada.escolherConta(contas, input);
10 Conta c2 = Entrada.escolherConta(contas, input);
11 double valor = Entrada.lerDouble("Digite o valor a ser
12 transferido: ", input);
13 c1.transferir(valor, c2);
14 }
15 }
```

## 19.6 Testando o menu

Para que o menu seja executado, ele será chamado no método **main**:

```
1 public static void main(String[] args) {
2 Sistema banco = new Sistema();
3 Scanner input = new Scanner(System.in);
4
5 System.out.println("Bem vindo ao Banco!\n");
6
7 int op = Entrada.menu(input);
8
9 while (op != 0) {
10 switch (op) {
11 case 1:
12 Entrada.menu_cadastro(banco, input);
13 break;
14 case 2:
15 Entrada.menu_movimentacoes(banco, input);
16 break;
17 default:
18 System.out.println("Opcao Invalida.");
19 }
20 op = Entrada.menu(input);
21 }
22
23 System.out.println("Bye!");
24 }
```

Para verificar se a aplicação está executando corretamente, execute os seguintes passos:

- Faça o cadastro dois clientes (Maria e João).

- Faça o cadastro de dois gerentes (Paula e José).
- Cadastre uma conta corrente para Maria, gerenciada por José.
- Cadastre uma poupança para João, gerenciada por Paula.
- Deposite 30 reais na conta de João.
- Saque 70 reais da poupança de Maria.
- Transfira 50 reais da conta de João para a poupança de Maria.
- Exiba o extrato das duas contas.

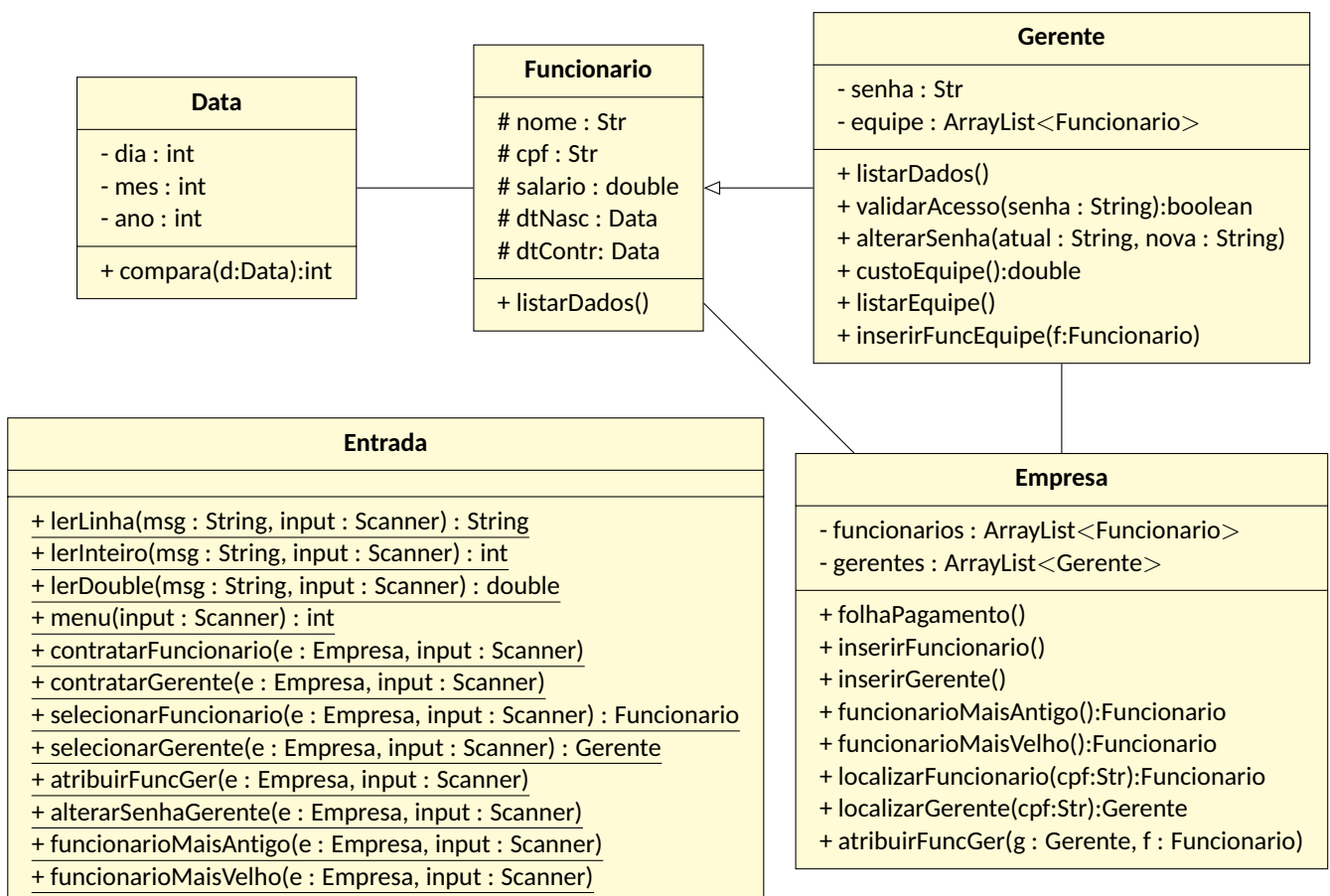
# Aula 20

## Revisão - Prática

Neste laboratório, vamos corrigir o exercício de revisão dos conceitos iniciais de POO: **Empresa de RH**. Certifique-se de ler **todo** o código e entender **todos** os métodos.

### 20.1 Por onde começar?

Temos cinco classes para implementar no sistema, mostradas no diagrama abaixo:



A primeira decisão é definir a ordem em que elas serão implementadas. Note que a classe Empresa depende das classes Funcionario e Gerente e não deveria ser implementada antes delas. Além disso,

Gerente depende de Funcionario, que por sua vez possui um atributo da classe Data. Já a classe Entrada depende de todas as Empresa, e deve ser a última a ser implementada.

## 20.2 Classe Data

Começamos então pela classe Data:

```
1 public class Data {
2 private int dia;
3 private int mes;
4 private int ano;
5
6 public Data(int dia, int mes, int ano) {
7 this.setDia(dia);
8 this.setMes(mes);
9 this.setAno(ano);
10 }
11
12 public int compara(Data d2) {
13 if (this.getAno() > d2.getAno()) return 1;
14 if (this.getAno() < d2.getAno()) return -1;
15
16 if (this.getMes() > d2.getMes()) return 1;
17 if (this.getMes() < d2.getMes()) return -1;
18
19 if (this.getDia() > d2.getDia()) return 1;
20 if (this.getDia() < d2.getDia()) return -1;
21
22 return 0;
23 }
24
25 // Metodos getters e setters
26 }
```

## 20.3 Classe Funcionario

Em seguida, podemos definir a classe Funcionario:

```
1 public class Funcionario {
2 protected String nome;
3 protected String cpf;
4 protected double salario;
5 protected Data dtNasc;
6 protected Data dtContr;
7
8 public Funcionario(String nome, String cpf, double salario, Data
9 dtNasc, Data dtContr) {
10 this.setNome(nome);
11 }
12 }
```

```

10 this.setCpf(cpf);
11 this.setSalario(salario);
12 this.setDtNasc(dtNasc);
13 this.setDtContr(dtContr);
14 }
15
16 public void listarDados() {
17 System.out.println(this.getNome() + " - CPF: " + this.getCpf
18 () + " - R$ " + this.getSalario());
19 }
20
21 // Metodos getters e setters
22 }

```

## 20.4 Classe Gerente

Agora, a classe Gerente:

```

1 import java.util.ArrayList;
2
3 public class Gerente extends Funcionario{
4 private String senha;
5 private ArrayList<Funcionario> equipe;
6
7 public Gerente(String nome, String cpf, double salario, Data
8 dtNasc, Data dtContr, String senha) {
9 super(nome, cpf, salario, dtNasc, dtContr);
10 this.senha = senha;
11
12 this.equipe = new ArrayList<>();
13 }
14
15 public void listarDados() {
16 System.out.print("GERENTE: ");
17 super.listarDados();
18 }
19
20 public boolean validarAcesso(String senha) {
21 return this.senha.equals(senha);
22 }
23
24 public void alterarSenha(String senhaAtual, String novaSenha) {
25 if (this.validarAcesso(senhaAtual)) {
26 this.senha = novaSenha;
27 System.out.println("Senha alterada com sucesso!");
28 }
29 else {
30 System.out.println("Senha atual incorreta!");
31 }
32 }
33 }

```



```

31 }
32
33 public double custoEquipe() {
34 double total = this.getSalario();
35 for (Funcionario f : equipe) {
36 total += f.getSalario();
37 }
38 return total;
39 }
40
41 public void listarEquipe() {
42 this.listarDados();
43 System.out.println("Funcionarios da equipe:");
44 for (Funcionario f : equipe) {
45 f.listarDados();
46 }
47 }
48
49 public void inserirFuncionarioEquipe(Funcionario f) {
50 this.equipe.add(f);
51 }
52 }

```

## 20.5 Classe Empresa

Agora, a classe Empresa:

```

1 import java.util.ArrayList;
2
3 public class Empresa {
4 ArrayList<Funcionario> funcionarios;
5 ArrayList<Gerente> gerentes;
6
7 public Empresa() {
8 this.funcionarios = new ArrayList<>();
9 this.gerentes = new ArrayList<>();
10 }
11
12 public boolean listarFuncionarios() {
13 if (funcionarios.isEmpty()) {
14 System.out.println("Empresa sem funcionarios cadastrados
15 .");
16 return false;
17 }
18 else {
19 for (Funcionario f : funcionarios) {
20 f.listarDados();
21 }
22 return true;
23 }
24 }
25 }

```

```
22 }
23 }
24
25 public boolean listarGerentes() {
26 if (gerentes.isEmpty()) {
27 System.out.println("Empresa sem gerentes cadastrados.");
28 return false;
29 }
30 else {
31 for (Gerente g : gerentes) {
32 g.listarDados();
33 }
34 return true;
35 }
36 }
37
38 public void folhaPagamento() {
39 if (gerentes.isEmpty() && funcionarios.isEmpty()) {
40 System.out.println("Empresa sem funcionarios ou gerentes
41 cadastrados.");
42 }
43 else {
44 double total = 0;
45
46 for (Gerente g : gerentes) {
47 g.listarDados();
48 System.out.println("Custo da equipe: R$ " + g.
49 custoEquipe());
50 total += g.getSalario();
51 }
52
53 for (Funcionario f : funcionarios) {
54 f.listarDados();
55 total += f.getSalario();
56 }
57
58 System.out.println("Custo total da empresa: R$ " + total
59);
60 }
61 }
62
63 public void inserirFuncionario(Funcionario f) {
64 this.funcionarios.add(f);
65 }
66
67 public void inserirGerente(Gerente g) {
68 this.gerentes.add(g);
69 }
70
71 public Funcionario funcionarioMaisAntigo() {
```

```
69 Funcionario f = null;
70
71 for (Funcionario func : funcionarios) {
72 if (f == null || f.getDtContr().compara(func.getDtContr
73 ())) > 0) {
74 f = func;
75 }
76 }
77
78 for (Gerente ger : gerentes) {
79 if (f == null || f.getDtContr().compara(ger.getDtContr()
80) > 0) {
81 f = ger;
82 }
83 }
84
85 return f;
86 }
87
88 public Funcionario funcionarioMaisVelho() {
89 Funcionario f = null;
90
91 for (Funcionario func : funcionarios) {
92 if (f == null || f.getDtNasc().compara(func.getDtNasc())
93 > 0) {
94 f = func;
95 }
96 }
97
98 for (Gerente ger : gerentes) {
99 if (f == null || f.getDtNasc().compara(ger.getDtNasc())
100 > 0) {
101 f = ger;
102 }
103 }
104
105 return f;
106 }
107
108 public Funcionario localizarFuncionario(String cpf) {
109 for (Funcionario f : funcionarios) {
110 if (f.getCpf().equals(cpf)) {
111 return f;
112 }
113 }
114
115 return null;
116 }
117
118 public Gerente localizarGerente(String cpf) {
```

```
115 for (Gerente g : gerentes) {
116 if (g.getCpf().equals(cpf)) {
117 return g;
118 }
119 }
120
121 return null;
122 }
123
124 public void atribuirFuncGerente(Gerente g, Funcionario f) {
125 if (g == null || f == null) {
126 System.out.println("Erro: um dos cpfs e invalido.");
127 }
128 else {
129 g.inserirFuncionarioEquipe(f);
130 }
131 }
132
133 }
```

## 20.6 Classe Entrada

Agora, a classe Entrada:

```
1 import java.util.Scanner;
2
3 public class Entrada {
4
5 public static String lerLinha(String msg, Scanner input) {
6 // Imprime uma mensagem ao usuario, le uma linha do teclado
6 // e retorna esta linha
7 System.out.print(msg);
8 String linha = input.nextLine();
9 return linha;
10 }
11
12 public static int lerInteiro(String msg, Scanner input) {
13 // Imprime uma mensagem ao usuario, le uma linha contendo um
13 // inteiro e retorna este inteiro
14 String linha = Entrada.lerLinha(msg, input);
15 return Integer.parseInt(linha);
16 }
17
18 private static double lerDouble(String msg, Scanner input) {
19 // Imprime uma mensagem ao usuario, le uma linha contendo um
19 // double e retorna este double
20 String linha = Entrada.lerLinha(msg, input);
21 return Double.parseDouble(linha);
22 }
23 }
```

```
23
24 public static int menu(Scanner input) {
25 // Imprime o menu principal, le a opcao escolhida pelo
 usuario e retorna a opcao selecionada.
26
27 String msg = "*****\n" +
28 "Escolha uma opcao:\n" +
29 "1) Contratar funcionario\n" +
30 "2) Contratar gerente\n" +
31 "3) Atribuir funcionario a gerente\n" +
32 "4) Alterar senha de gerente\n" +
33 "5) Funcionario mais antigo\n" +
34 "6) Funcionario mais velho\n" +
35 "7) Folha de pagamento mensal\n" +
36 "0) Sair\n";
37
38 int op = Entrada.lerInteiro(msg, input);
39
40 while (op < 0 || op > 8) {
41 System.out.println("Opcao invalida. Tente novamente: ");
42 op = Entrada.lerInteiro(msg, input);
43 }
44
45 return op;
46 }
47
48 public static void contratarFuncionario(Empresa e, Scanner input
49) {
50 String nome = Entrada.lerLinha("Digite o nome do funcionario
 : ", input);
51 String cpf = Entrada.lerLinha("Digite o CPF do funcionario:
 ", input);
52 double salario = Entrada.lerDouble("Digite o salario do
 funcionario: ", input);
53 int dia1 = Entrada.lerInteiro("Digite o dia de nascimento do
 funcionario: ", input);
54 int mes1 = Entrada.lerInteiro("Digite o mes de nascimento do
 funcionario: ", input);
55 int ano1 = Entrada.lerInteiro("Digite o ano de nascimento do
 funcionario: ", input);
56 int dia2 = Entrada.lerInteiro("Digite o dia de contratacao
 do funcionario: ", input);
57 int mes2 = Entrada.lerInteiro("Digite o mes de contratacao
 do funcionario: ", input);
58 int ano2 = Entrada.lerInteiro("Digite o ano de contratacao
 do funcionario: ", input);
59
60 e.inserirFuncionario(new Funcionario(nome, cpf, salario, new
 Data(dia1, mes1, ano1), new Data(dia2, mes2, ano2)));
61 }
```

```
61
62 public static void contratarGerente(Empresa e, Scanner input) {
63 String nome = Entrada.lerLinha("Digite o nome do gerente: ",
64 input);
65 String cpf = Entrada.lerLinha("Digite o CPF do gerente: ",
66 input);
67 double salario = Entrada.lerDouble("Digite o salario do
68 gerente: ", input);
69 int dia1 = Entrada.lerInteiro("Digite o dia de nascimento do
70 gerente: ", input);
71 int mes1 = Entrada.lerInteiro("Digite o mes de nascimento do
72 gerente: ", input);
73 int ano1 = Entrada.lerInteiro("Digite o ano de nascimento do
74 gerente: ", input);
75 int dia2 = Entrada.lerInteiro("Digite o dia de contratacao
76 do gerente: ", input);
77 int mes2 = Entrada.lerInteiro("Digite o mes de contratacao
78 do gerente: ", input);
79 int ano2 = Entrada.lerInteiro("Digite o ano de contratacao
80 do gerente: ", input);
81 String senha = Entrada.lerLinha("Digite a senha do gerente:
82 ", input);
83
84 e.inserirGerente(new Gerente(nome, cpf, salario, new Data(
85 dia1, mes1, ano1), new Data(dia2, mes2, ano2), senha));
86 }
87
88 public static Funcionario selecionarFuncionario(Empresa e,
89 Scanner input) {
90 System.out.println("Funcionarios cadastrados:");
91 if (e.listarFuncionarios()) {
92 String cpf = Entrada.lerLinha("Digite o CPF do
93 funcionario: ", input);
94 return e.localizarFuncionario(cpf);
95 }
96 else {
97 return null;
98 }
99 }
100
101 public static Gerente selecionarGerente(Empresa e, Scanner input
102) {
103 System.out.println("Gerentes cadastrados:");
104 if (e.listarGerentes()) {
105 String cpf = Entrada.lerLinha("Digite o CPF do gerente:
106 ", input);
107 return e.localizarGerente(cpf);
108 }
109 else {
110 return null;
111 }
112 }
```

```
96 }
97 }
98
99 public static void atribuirFuncGerente(Empresa e, Scanner input)
100 {
101 Funcionario f = Entrada.selecionarFuncionario(e, input);
102 Gerente g = Entrada.selecionarGerente(e, input);
103
104 if (f != null && g != null) {
105 e.atribuirFuncGerente(g, f);
106 }
107
108 public static void alterarSenhaGerente(Empresa e, Scanner input)
109 {
110 Gerente g = Entrada.selecionarGerente(e, input);
111
112 if (g != null) {
113 String senhaAtual = Entrada.lerLinha("Digite a senha
114 atual: ", input);
115 String novaSenha = Entrada.lerLinha("Digite a nova senha
116 : ", input);
117 g.alterarSenha(senhaAtual, novaSenha);
118 }
119 }
120
121 public static void funcionarioMaisAntigo(Empresa e, Scanner
122 input) {
123 Funcionario f = e.funcionarioMaisAntigo();
124
125 if (f == null) {
126 System.out.println("Erro: Funcionario nao encontrado.");
127 }
128 else {
129 f.listarDados();
130 }
131 }
132
133 public static void funcionarioMaisVelho(Empresa e, Scanner input
134) {
135 Funcionario f = e.funcionarioMaisVelho();
136
137 if (f == null) {
138 System.out.println("Erro: Funcionario nao encontrado.");
139 }
140 else {
141 f.listarDados();
142 }
143 }
144 }
```

140 }

## 20.7 Classe principal

Por fim, a classe principal cria um único objeto da classe Scanner que servirá de entrada para todo o sistema e exibe o menu do sistema para o usuário.

```
1 public static void main(String[] args) {
2 Scanner input = new Scanner(System.in);
3 Empresa e = new Empresa();
4
5 int op = Entrada.menu(input);
6
7 while (op != 0) {
8 switch (op) {
9 case 1:
10 Entrada.contratarFuncionario(e, input);
11 break;
12 case 2:
13 Entrada.contratarGerente(e, input);
14 break;
15 case 3:
16 Entrada.atribuirFuncGerente(e, input);
17 break;
18 case 4:
19 Entrada.alterarSenhaGerente(e, input);
20 break;
21 case 5:
22 Entrada.funcionarioMaisAntigo(e, input);
23 break;
24 case 6:
25 Entrada.funcionarioMaisVelho(e, input);
26 break;
27 case 7:
28 e.folhaPagamento();
29 }
30 op = Entrada.menu(input);
31 }
32
33 System.out.println("Bye!");
34 input.close();
35 }
```

Certifique-se de ler **todo** o código e entender **todos** os métodos! Isso vai ser importante no trabalho, que será passado na próxima aula.



## **Parte III**

### **Extra**

# Apêndice A

## Orientação a Objetos em Python

Neste capítulo, vamos aprender a utilizar Orientação a Objetos em Python.

### A.1 Classes, objetos e construtores

Em Python, a classe de um objeto e o tipo de um objeto são sinônimos. Cada objeto do Python tem uma classe (tipo) que é derivada diretamente ou indiretamente da classe **object**. A classe (tipo) de um objeto determina o que ele é e como pode ele ser manipulado. Uma classe encapsula dados, operações e semântica, ou seja, é um agrupamento de valores e suas operações. O usuário de uma classe manipula objetos instanciados dessa classe somente através dos métodos fornecidos por essa classe.

Ao criarmos um número inteiro, por exemplo, estamos criando um objeto da classe **int**. Podemos saber a classe de cada objeto através do comando `type`:

```
1 x = 50
2 l = [1, 2, 3, 4, 5, 6]
3
4 print(type(x))
5 print(type(l))
```

O resultado do código acima será:

```
<class 'int'>
<class 'list'>
```

A classe **list**, por exemplo, possui métodos como **append** (adiciona um elemento ao final da lista), **remove** (remove um elemento da lista), **reverse** (inverte a ordem dos elementos da lista) e **sort** (coloca os elementos da lista em ordem crescente):

```
1 l = [5, 2, 3, 7, 1, 4, 8, 9, 10, 6]
2 l.remove(5)
3 l.append(11)
4 l.sort()
5 l.reverse()
6 print(l)
```

O resultado do código acima será:

```
[11, 10, 9, 8, 7, 6, 4, 3, 2, 1]
```

Para criar nossas próprias classes, utilizamos o comando `class`:

```
1 class Pessoa:
2 def __init__(self):
3 self.nome = input("Digite o nome da pessoa: ")
4 self.cpf = input("Digite o cpf da pessoa: ")
5 self.idade = int(input("Digite a idade da pessoa: "))
```

O `__init__` é um método responsável pela inicialização dos objetos da classe (semelhante ao **construtor** do Java). Ele sempre deve ter este mesmo nome, e seu primeiro parâmetro (por padrão, chamamos de `self`) se refere ao próprio objeto que está sendo acessado no momento (semelhante ao `this` do Java). Ao contrário do Python, o `this` não precisa ser incluído na lista de parâmetros dos métodos de uma classe em Java.

Para criar um objeto da classe, o parâmetro `self` não precisa ser passado:

```
1 p1 = Pessoa()
```

Caso a classe possua um atributo cujo valor inicial é fixo, ele pode ser explicitado fora do `__init__`:

```
1 class Conta:
2 saldo = 0
3
4 def __init__(self, p):
5 self.numero = input("Digite o numero da conta: ")
6 self.titular = p
```

Neste caso, o valor atributo **saldo** sempre inicia zerado. Note que agora o `__init__` recebe um parâmetro adicional, que será um objeto da classe **Pessoa** com o titular da conta. Novamente, o `self` não precisa ser passado como parâmetro ao criar os objetos:

```
1 c1 = Conta(p1)
```

## A.2 Métodos

Classes possuem métodos, que definem o comportamento de seus objetos. Em Python, o método é definido de forma semelhante a uma função, mas deve ser indentado dentro da classe e seu primeiro parâmetro sempre será o próprio objeto (`self`):

```
1 class Conta:
2 saldo = 0
3
4 def __init__(self, p):
5 self.numero = input("Digite o numero da conta: ")
6 self.titular = p
7
```

```
8 def disponivel(self):
9 return self.saldo
10
11 def depositar(self, valor):
12 self.saldo += valor
13 print("Deposito realizado com sucesso.")
14
15 def sacar(self, valor):
16 if valor <= self.disponivel():
17 self.saldo -= valor
18 print("Saque realizado com sucesso.")
19 return True
20 else:
21 print("Valor indisponivel para saque.")
22 return False
23
24 p1 = Pessoa()
25
26 c1 = Conta(p1)
27
28 c1.depositar(300)
29 c1.sacar(150)
30 c1.sacar(180)
31
32 print("Valor disponivel: R$", c1.disponivel())
```

O resultado do código acima será:

```
Deposito realizado com sucesso.
Saque realizado com sucesso.
Valor indisponivel para saque.
Valor disponivel na conta: R$ 150
```

Lembre-se sempre que:

- O `self` sempre deve ser o primeiro parâmetro na definição de um método.
- Dentro do método, os atributos da classe são sempre precedidos do `self`. Por exemplo, `self.saldo`.
- O `self` não deve ser passado como parâmetro quando chamarmos o método.

## A.3 Herança

Python também permite a criação de classes derivadas, estendendo as classes já existentes. A classe nova é chamada de classe derivada e a classe existente de quem é derivada é chamada de classe base. Uma classe derivada herda todos os atributos de sua classe base. Isto é, a classe derivada contém todos os atributos da classe contidos na classe base e a classe derivada suporta todas as operações fornecidas pela classe base.

```
1 class ContaCorrente(Conta):
2 limite = 200
3
4 def disponivel(self):
5 return self.saldo + self.limite
6
7 cc1 = ContaCorrente(p1)
8
9 cc1.depositar(300)
10 cc1.sacar(150)
11 cc1.sacar(180)
12
13 print("Valor disponivel: R$", c1.disponivel())
```

Assim a conta corrente **cc1** é também uma **Conta**, tem os atributos **saldo**, **titular** e **numero**, e além deles possui também o atributo **limite**. Além disso, o método **disponivel** foi reescrito para incluir o valor do limite da conta corrente.

O resultado do código acima será:

```
Deposito realizado com sucesso.
Saque realizado com sucesso.
Saque realizado com sucesso.
Valor disponivel na conta: R$ 170
```

## A.4 Encapsulamento e modificadores de acesso

O Python não utiliza o termo **private**, que é um modificador de acesso e também chamado de modificador de visibilidade, para impedir o acesso aos atributos de uma classe. Ao invés disso, inserimos dois underscores (**\_\_**) na frente do atributo para adicionar esta característica. Por convenção, os programadores utilizam um único underscore na frente do atributo para indicar que ele é protegido (ou seja, só pode ser acessado na própria classe ou em suas classes derivadas):

```
1 class Conta:
2 _saldo = 0
3
4 def __init__(self, p):
5 self._numero = input("Digite o numero da conta: ")
6 self._titular = p
7
8 def disponivel(self):
9 return self._saldo
10
11 def depositar(self, valor):
12 self._saldo += valor
13 print("Deposito realizado com sucesso.")
14
15 def sacar(self, valor):
```

```
16 if valor <= self.disponivel():
17 self._saldo -= valor
18 print("Saque realizado com sucesso.")
19 return True
20 else:
21 print("Valor indisponível para saque.")
22 return False
23
24 class ContaCorrente(Conta):
25 __limite = 200
26
27 def disponivel(self):
28 return self._saldo + self.__limite
```

Dessa forma, não é possível acessar o limite das contas a partir de um código fora da classe (nem usando `cc1.limite` e nem usando `cc1.__limite`).

## A.5 Exercício

1. Implemente os códigos mostrados neste tutorial, contendo as classes `Pessoa`, `Conta` e `ContaCorrente`.
2. Adicione um novo método para transferir dinheiro entre duas contas.
3. Crie uma segunda `ContaCorrente`, e tente transferir dinheiro entre elas.

# Apêndice B

## Pygame

Neste laboratório, vamos criar nossos primeiros jogos utilizando Orientação a Objetos em Python, com o auxílio do PyGame ([www.pygame.org](http://www.pygame.org)). PyGame é um módulo Python que fornece algumas funcionalidades para a programação gráfica, em especial jogos. Provê facilidades tais como *Sprites*, *Render Groups*, detecção de colisão básica (retângulos), etc. PyGame é multi plataforma, funcionando em Linux, Windows, Sun, MacOS, etc.

Se não tiver PyGame na sua máquina, abra o *prompt* de comando e instale com:

```
py -m pip install --user pygame
```

### B.1 Primeiro Exemplo

Vá à página da disciplina e baixe o arquivo **intro.zip** para o seu computador e descompacte-o. Em seguida, abra o arquivo **ball.py** no Geany ou no seu editor preferido.

Você irá encontrar o seguinte código:

```
1 import sys
2 import pygame
3
4 size = width, height = 1024,768
5 color = 0,0,0
6
7 def main():
8 pygame.init()
9 screen = pygame.display.set_mode(size)
10
11 while True:
12 for event in pygame.event.get():
13 if event.type == pygame.QUIT: sys.exit()
14
15 screen.fill(color)
16 pygame.display.flip()
17
18 main()
```

As linhas 4 e 5 definem variáveis para o tamanho e cor do fundo da janela. Como elas estão no início do código, fora de qualquer função, poderão no futuro ser acessadas em qualquer parte do código. A variável **size** começa com o valor (1024, 768) que indica o número de pixels na horizontal e na vertical da tela, respectivamente. Os valores também poderão ser acessados individualmente através das variáveis **width** (largura) e **height** (altura).

A variável **color** define a cor do fundo. Uma cor deve ser definida através de três valores: R (vermelho), G (verde) e B (azul). Cada valor varia entre 0 (ausência desta cor) e 255 (intensidade máxima desta cor). A combinação de intensidade destes valores geram 16 milhões de cores diferentes. A cor (0, 0, 0) correspondendo ao preto, (255, 255, 255) ao branco, (255, 0, 0) ao **vermelho**, (0, 255, 0) ao **verde**, (0, 0, 255) ao **azul**, (255, 255, 0) ao **amarelo**, (0, 255, 255) ao **ciano**, (255, 0, 255) ao **magenta**, etc.

A linha 8 inicializa os módulos do pygame e a linha 9 cria uma janela gráfica. A função `set_mode()` cria um objeto da classe **Surface** que representa os gráficos exibidos na tela. Qualquer coisa que fizermos neste objeto será visível na tela.

Em seguida, temos o loop principal do programa. O código `while True` significa que algo será executado, a princípio, eternamente (pois `True` sempre será verdadeiro). Entretanto, as linhas 12 e 13 fazem com que o programa possa terminar:

- A linha 12 percorre todos os eventos que podem ter sido gerados pelo usuário. Qualquer interação do usuário (fechar a janela, pressionar o botão do mouse, digitar algo no teclado, etc) será capturada como um evento diferente, e cada evento possui um tipo específico.
- Por enquanto, tratamos apenas o evento de fechamento da janela na linha 13: `pygame.QUIT`. Neste caso, inserimos o código `sys.exit()` para que o programa aborte, e só assim o loop principal será encerrado.

A linha 15 apaga tudo que havia sido desenhado na tela até então, e a tela é totalmente pintada de preto. Parece estranho para quem nunca trabalhou com animação, mas na verdade os objetos que você vê em um jogo não “se movimentam” efetivamente na tela. O que ocorre é que, a cada instante, pintamos a tela por cima do que existia anteriormente e depois redesenhamos os objetos em novas posições.

Por fim, a linha 16 atualiza a tela, desenhando tudo que foi pedido desde a atualização anterior. Execute o programa com o código que temos até agora, e você verá apenas uma tela preta por enquanto. Altere o valor da variável **color** e veja a tela sendo desenhada de outra cor.

## B.2 Adicionando um objeto em movimento

Agora, vamos alterar o código para adicionarmos um objeto em movimento pela tela. Para isso, insira as linhas a seguir antes do loop principal:

```
1 ball = pygame.image.load("images/bola1.gif")
2 ballrect = ball.get_rect()
3
4 speed = [1, 1]
```

Primeiro, estamos carregando uma imagem chamada “bola1.gif”, que está salva dentro da pasta “images”. Em seguida, criamos um “retângulo” invisível ao redor das image. É através deste retângulo que iremos definir a posição do objeto e verificar se ele colidiu com outro objeto ou com as bordas da janela. Por fim, definimos uma variável com a velocidade do objeto no eixo x e no eixo y. Os valores [1, 1] indicam



que o objeto vai se movimentar 1 pixel no eixo x e 1 no eixo y cada vez que a tela for redesenhada. Se a velocidade fosse  $[-2, 4]$ , por exemplo, o objeto iria se deslocar 2 pixels para a esquerda e 4 para a direita. Se fosse  $[3, -9]$ , ele iria se movimentar 3 pixels para a direita e 9 para cima.

Depois, altere seu loop principal para o trecho de código a seguir:

```
1 while True:
2 for event in pygame.event.get():
3 if event.type == pygame.QUIT: sys.exit()
4
5 ballrect.move_ip(speed)
6 if ballrect.left < 0 or ballrect.right > width:
7 speed[0] = -speed[0]
8 if ballrect.top < 0 or ballrect.bottom > height:
9 speed[1] = -speed[1]
10
11 screen.fill(color)
12 screen.blit(ball, ballrect)
13 pygame.display.flip()
```

A linha 5 faz com que o objeto efetivamente se movimente a cada iteração do loop. O parâmetro da função `move_ip` indica o quanto o objeto irá se movimentar nos eixos x e y, respectivamente. Por isso, criamos a variável **speed** antes do loop.

Em seguida, alteramos a velocidade do objeto quando ele encontra um dos cantos da tela:

- Se a posição do objeto no eixo x for 0 (ele se encontra à esquerda da tela) ou se for igual à largura da janela (ele está à direita da tela), vamos alterar o sentido da velocidade no eixo x (linhas 6 e 7 do código).
- Se a posição do objeto no eixo y for 0 (topo da tela) ou se for igual à largura da janela (parte mais baixa da tela), vamos alterar o sentido da velocidade no eixo y (linhas 8 e 9 do código).

Por fim, após pintar o fundo de preto, desenhamos o objeto na tela (linha 12 do trecho de código acima). Altere seu código, execute-o, e veja o objeto se movimentando na tela. Em seguida, altere a velocidade inicial para  $[2, 2]$  para ver como ele se movimenta mais rápido.

## B.3 Criando mais de um objeto

Agora, vamos continuar alterando o código anterior, para incluir mais de um objeto na tela. Primeiro, antes do loop principal, vamos criar dois objetos:

```
1 ball = pygame.image.load("images/bola1.gif")
2 ballrect = ball.get_rect()
3 speed = [1, 1]
4
5 ball2 = pygame.image.load("images/bola2.png")
6 ballrect2 = ball2.get_rect()
7 ballrect2.centerx = width/2
8 speed2 = [-1, 1]
```

Repare que tivemos que carregar duas imagens e definir duas velocidades. Tudo que fizemos para um objeto no restante do código, teremos que fazer também para o outro. Além disso, alteramos a posição inicial do segundo objeto no eixo x (linha 7) para o centro da tela, e sua velocidade (a princípio, ele irá começar descendo para o lado esquerdo da tela, enquanto o primeiro começa descendo para a direita).

Depois, dentro do loop principal, temos que duplicar o código que verifica se cada objeto alcançou as bordas da janela:

```

1 ballrect.move_ip(speed)
2 ballrect2.move_ip(speed2)
3 if ballrect.left < 0 or ballrect.right > width:
4 speed[0] = -speed[0]
5 if ballrect.top < 0 or ballrect.bottom > height:
6 speed[1] = -speed[1]
7 if ballrect2.left < 0 or ballrect2.right > width:
8 speed2[0] = -speed2[0]
9 if ballrect2.top < 0 or ballrect2.bottom > height:
10 speed2[1] = -speed2[1]
```

E, por fim, temos que lembrar de desenhar os dois objetos:

```

1 #screen.fill(color)
2 screen.blit(ball, ballrect)
3 screen.blit(ball2, ballrect2)
4 pygame.display.flip()
```

Repare que comentamos a linha que pinta a tela de preto. Execute o código e veja o que acontece quando deixamos de pintar o fundo da janela. Por fim, descomente esta linha e volte a pintar o fundo da janela para que os objetos se “movimentem” corretamente.

## B.4 Criando uma classe para os objetos

Na seção seguinte, vimos que foi necessário replicar várias linhas de códigos para lidar com dois objetos na tela, o que deixou o código um pouco poluído. Para melhorá-lo, agora iremos criar uma classe para representar o objeto **bola**. A classe deve ser inserida antes do main:

```

1 class Ball(pygame.sprite.Sprite):
2 def __init__(self, img, s, pos):
3 self.image = pygame.image.load(img)
4 self.speed = s
5 self.rect = self.image.get_rect()
6 self.rect.centerx = pos[0]
7 self.rect.centery = pos[1]
8
9 def update(self):
10 self.rect.move_ip(self.speed)
11
12 if self.rect.left < 0 or self.rect.right > width:
13 self.speed[0] = -self.speed[0]
14 if self.rect.top < 0 or self.rect.bottom > height:
15 self.speed[1] = -self.speed[1]
```

A classe **Ball** irá representar todos as bolas que estão passeando pela tela do nosso jogo. A classe herda de **Sprite**, que é uma classe do PyGame com vários métodos já implementados para lidar com objetos gráficos. O método `__init__` está recebendo como parâmetros a figura que deve ser carregada para cada objeto, a velocidade inicial e a posição inicial de cada um na janela. Em seguida, o método `update` define como iremos atualizar as propriedades dos objetos a cada vez que a tela for redesenhada.

Com isso, o `main` ficará menos poluído:

```
1 def main():
2 pygame.init()
3
4 screen = pygame.display.set_mode(size)
5
6 ball1 = Ball("images/bola1.gif", [1,1], [100,100])
7 ball2 = Ball("images/bola2.png", [-1,1], [width-100,100])
8
9 while True:
10 for event in pygame.event.get():
11 if event.type == pygame.QUIT: sys.exit()
12
13 ball1.update()
14 ball2.update()
15
16 screen.fill(color)
17 screen.blit(ball1.image, ball1.rect)
18 screen.blit(ball2.image, ball2.rect)
19 pygame.display.flip()
```

Repare que, agora, só precisamos criar os dois objetos (dizendo a figura, velocidade e posição inicial de cada um – linhas 6 e 7), atualizá-los (linhas 13 e 14) e redesenhá-los (linhas 17 e 18). Execute o código e verifique que tudo continua funcionando após a utilização da classe **Ball**.

## B.5 Lidando com vários objetos

Com a criação da classe **Ball**, o código da função principal ficou menos poluído. Entretanto, com a inclusão de vários objetos, poder ficar cansativo repetir atualizar e desenhar todos eles a cada iteração do loop. Quando temos vários objetos semelhantes (de uma mesma classe), podemos agrupá-los em uma lista. Primeiro, vamos alterar o método `__init__` da classe **Ball** para que cada bola se inicie em uma posição aleatória da tela:

```
1 class Ball(pygame.sprite.Sprite):
2 def __init__(self, img, speed):
3 self.image = pygame.image.load(img)
4 self.speed = speed
5 self.rect = self.image.get_rect()
6 self.rect.centerx = randint(150, width-150)
7 self.rect.centery = randint(150, height-150)
```

Para que o código acima funcione, certifique-se de importar o método `randint` do módulo `random` (que gera um número inteiro aleatório dentro de um intervalo) no início do código:

```
1 from random import randint
```

Com isso, o código principal ficará assim:

```
1 def main():
2 pygame.init()
3 screen = pygame.display.set_mode(size)
4
5 balls = [Ball("images/bola1.gif", [1,1]),
6 Ball("images/bola2.png", [-1,1]),
7 Ball("images/bola3.png", [1,-1]),
8 Ball("images/bola4.png", [1,-1])]
9
10 while True:
11 for event in pygame.event.get():
12 if event.type == pygame.QUIT: sys.exit()
13
14 screen.fill((0, 0, randint(0,50)))
15
16 for ball in balls:
17 ball.update()
18 screen.blit(ball.image, ball.rect)
19
20 pygame.display.flip()
```

Repare que foram criadas 4 bolas, e cada uma delas foi inserida na lista *l*. Com isso, ao invés de repetir os códigos para atualizar e redesenhar cada uma delas, fazemos tudo isso através do loop nas linhas 16, 17 e 18. Só foi preciso definir a figura inicial de cada uma delas, e todo o resto é feito automaticamente por este loop.

Repare também que alteramos a cor de fundo na linha 14. Agora, o fundo será azul escuro em um tom aleatório. Execute o código novamente e confira o resultado.

## B.6 Exercício

1. Altere a velocidade inicial de cada objeto para números aleatórios entre  $-3$  e  $3$ , nos dois eixos. Execute o código mais de uma vez para conferir o resultado.
2. Comente a linha que pinta o fundo de cores aleatórias e veja a figura que será formada. Repare que os objetos são desenhados em ordem: o último a ser desenhado sempre aparecerá “por cima” do anterior.

## B.7 Segundo Exemplo: Ping

Vá à página da disciplina e baixe o arquivo **ping.zip** para o seu computador e [descompacte-o](#). Em seguida, abra o arquivo **ping.py** no Geany ou no seu editor preferido. Esta seção explica o código inicial. Entenda-o antes de alterá-lo. O código já possui duas classes:

1. **Plataforma:** A primeira classe é bem simples, herdada da classe `Sprite` nativa do PyGame. Um `sprite` é um elemento gráfico do jogo, e possui 2 propriedades importantes: **image** e **rect**. **Image** é a imagem em si, carregada do disco (os principais formatos são suportados, tais como JPG, GIF, PNG e BMP) e **rect** representa o retângulo virtual que contém a imagem (imagine um retângulo circunscrito à imagem). Alterando as propriedades **center**, **centerx**, **centery**, **top**, **bottom**, **left** e **right** de **rect**, podemos posicionar o `sprite` onde quisermos na tela.

```

1 class Plataforma(pygame.sprite.Sprite):
2 """Classe para a plataforma"""
3 def __init__(self, startpos, img):
4 pygame.sprite.Sprite.__init__(self)
5 self.image = pygame.image.load(img)
6 self.rect = self.image.get_rect()
7 self.rect.center = startpos

```

Repare que não criamos um método `update` para esta classe, pois a plataforma não se move automaticamente.

2. **Bola:** A segunda classe é similar à primeira. A maior diferença é no método `update`, que faz com que a bola “quique” pela tela (quando atingir os lados). Caso a bola atinja a parte inferior da tela, ela “morre” e é reinicializada no alto da tela com uma velocidade maior. O método **update** é executado a cada loop do jogo para mover o `sprite` pela tela, através da função `move_ip`, que aceita 2 parâmetros (*x* e *y*), indicando quantos pixels queremos mover o objeto no eixo *x* (horizontal) e *y* (vertical). Vale lembrar que, no nosso caso, a origem do eixo está no canto superior esquerdo da tela. Logo, ao executarmos `move_ip(1,2)`, estaremos movendo o `sprite` 1 pixel para a direita e 2 pixels para baixo.

```

1 class Bola(pygame.sprite.Sprite):
2 """Classe para a bola"""
3 def __init__(self, startpos):
4 pygame.sprite.Sprite.__init__(self)
5 self.image = pygame.image.load('images/wilson.gif')
6 self.rect = self.image.get_rect()
7 self.rect.center = startpos
8 self.speed = [2,2]
9
10 def update(self):
11 self.rect.move_ip(self.speed)
12 if self.rect.left < 0 or self.rect.right > width:
13 self.speed[0] = -self.speed[0]
14 if self.rect.bottom > height:
15 self.rect.center = (width/2, 50)
16 self.speed[0] = abs(self.speed[0]) + 1
17 self.speed[1] = self.speed[1] + 1

```

Nas linhas 12 e 13 do trecho de código acima, invertemos a direção horizontal se a bola atingir os lados da tela. Na linha 14, decidimos o que fazer quando a bola atingir o fundo da tela: ela é reiniciada no alto da tela e no centro horizontal (linha 15) e sua velocidade aumenta (linhas 16 e 17).

Vamos agora ao método principal:

```

1 def main():
2 pygame.init()

```

```

3 os.environ['SDL_VIDEO_CENTERED'] = '1'
4 screen = pygame.display.set_mode(size)
5 pygame.key.set_repeat(1,1)
6 pygame.display.set_caption('Ping!')
7 clock = pygame.time.Clock()
8
9 # Carregando a imagem de fundo
10 background = pygame.image.load("images/campo.gif")
11
12 # Criando um objeto para a bola
13 b = Bola((width/2, 50))
14
15 # Criando um objeto para a plataforma
16 p1 = Plataforma((width/2, height-5), 'images/red.gif')
17 plataformas = pygame.sprite.Group()
18 plataformas.add(p1)
19
20 # Selecionando a musica de fundo
21 pygame.mixer.init()
22 pygame.mixer.music.load("music/mario.mp3")
23 pygame.mixer.music.play(-1)

```

O comando na linha 3 do trecho de código acima define que a janela será aberta no centro do monitor. O código na linha 5 permite que o comando de uma determinada tecla seja repetido enquanto ela estiver pressionada. O seja, podemos manter as teclas ← e → pressionadas para que a plataforma continue se movimentando, sem ter que ficar apertando-as freneticamente. Na linha 6, definimos o título que será exibido na janela do programa. Por fim, a linha 7 inicializa um relógio para controlarmos melhor a velocidade do jogo.

Depois, carregamos uma imagem para ser utilizada como fundo de tela (linha 10), criamos um objeto para a bola e outro para a plataforma do jogo. O objetivo da plataforma é que o usuário rebata a bola, antes que ela alcance a parte de baixo da janela. Note que a posição inicial de um objeto é definido pelo centro de sua figura. Como a figura da plataforma possui 10 pixels de altura, sua posição inicial no eixo y será *height* – 5 (ou seja, a altura da janela menos a metade da altura da figura). Altere essa posição inicial para apenas (width/2, height) e veja que só metade da plataforma irá aparecer.

Nas linhas 17 e 18 criamos um grupo de Sprites (que, por enquanto, só possui a plataforma). Mais adiante, quando adicionarmos outra plataforma, você verá a utilidade deste grupo.

Por fim, nas linhas 21 a 23, carregamos a música que tocará durante o jogo. O parâmetro do método `play` indica quantas vezes a música irá repetir. Quando o valor é `-1`, a música repete indefinidamente.

Em seguida, temos o loop principal:

```

1 while abs(b.speed[0]) < 10:
2 clock.tick(120)
3
4 for event in pygame.event.get():
5 if event.type == pygame.QUIT:
6 print('Bye')
7 sys.exit()
8 elif event.type == pygame.KEYDOWN:
9 if event.key == pygame.K_ESCAPE or

```

```

10 event.key == pygame.K_q:
11 print ('Bye')
12 sys.exit()
13 moverPlataformas(event.key, p1)
14
15 # Atualizando os objetos
16 b.update()
17 plataformas.update()
18
19 # Desenhando os objetos
20 screen.blit(background, (0, 0))
21 screen.blit(b.image, b.rect)
22 plataformas.draw(screen)
23 pygame.display.update()

```

Na linha 1, definimos que o loop só termina quando a velocidade da bola atingir 10 unidades. Na linha 2, usamos o relógio para garantir que o jogo não irá redesenhar a janela numa frequência superior a 120 frames por segundo (comente esta linha e veja o que acontece).

Entre as linhas 4 e 13, tratamos dos eventos gerados pela interação com o usuário. Primeiro, o jogo é encerrado se o usuário fechar a janela (linhas 5 a 7). Depois, na linha 8, consideramos que o usuário pode ter digitado alguma tecla. Se ele digitar 'ESC' (linha 9) ou 'q' (linha 10), o jogo também é encerrado (linha 12). Na linha 13, chamamos uma função separada (para não poluir o código), que trata das demais teclas. Localize a função em seu código, e veja que a plataforma é movida 5 pixels para a esquerda se o usuário pressionar a tecla ← e 5 pixels para a direita se o usuário pressionar a tecla →.

Na linha 16, atualizamos a posição da bola. Na linha 17, atualizamos de uma vez só a posição de todos os objetos do grupo de Sprites que definimos anteriormente. Na verdade, esta linha é desnecessária (a posição das plataformas não será atualizada automática, e sim quando o usuário digitar alguma tecla). Deixamos esta linha aí apenas para você saber que é possível atualizar, de uma só vez, a posição de todos os Sprite dentro de um grupo. Nas linhas 20 a 23, desenhamos o fundo, depois a bola, depois todos os objetos do grupo de Sprites de uma só vez, e finalmente atualizamos a tela com tudo que queremos desenhar.

## B.8 Colisões

Com a bola e a plataforma já sendo desenhadas e movimentadas, é preciso que a bola seja rebatida verticalmente caso colida com a plataforma. Você deve inserir o código a seguir após a atualização dos objetos e antes que eles sejam redesenhados na tela:

```

1 # Verificando colisoes
2 colisoes = pygame.sprite.spritecollide(b, plataformas, False)
3
4 if len(colisoes) > 0:
5 b.speed[1] = -b.speed[1]

```

Na linha 2 do trecho de código acima, criamos uma lista com todos os objetos do grupo de plataformas que estão colidindo com a bola em um determinado momento. Por enquanto, só há uma plataforma no jogo, então a lista de colisões é vazia (caso não haja colisão) ou contém apenas o objeto *p1*. Dessa forma, na linha 4, verificamos se há algum elemento nesta lista. Se houver, mudamos o sentido da velocidade

vertical da bola na linha 5.

Note que é preciso inserir uma condição no método `update()` da classe **Bola** para que a bola seja rebatida verticalmente quando atingir o topo da tela.

## B.9 Ping Pong

Sua missão agora é transformar o jogo em um Ping Pong. Primeiro, você deve inserir outra plataforma na tela. Para isso:

- Você **não** deve criar outra classe, e sim um novo objeto.
- Esta nova plataforma deve ser posicionada no topo da janela.
- Repare que já há a figura de uma barra azul na pasta de imagens. Utilize esta figura para a nova plataforma.
- Utiliza as teclas 'z' e 'x' para movimentar a plataforma para esquerda e direita (veja a lista completa de teclas em <http://www.pygame.org/docs/ref/key.html>).
- Lembre-se de adicionar a nova plataforma ao grupo de Sprites (assim, ela será desenhada automaticamente).
- Por fim, você deverá alterar o método `update` da bola: se ela atingir o topo da tela, a bola é reiniciada da parte baixa da tela, subindo numa velocidade maior.

Repare que não precisamos alterar nada no tratamento das colisões. O método que utilizamos para detectar colisões já verifica colisões entre a bola e qualquer objeto do grupo de plataformas. No nosso jogo, sabemos que a bola nunca vai colidir com as duas plataformas ao mesmo tempo. Portanto, ou a lista de colisões é vazia (caso não haja colisão), ou contém apenas o objeto `p1`, ou contém apenas a nova plataforma. Portanto, ao verificamos se há algum elemento na lista de colisões, continuamos alterando o sentido da velocidade vertical da bola na linha 5.

A tela do jogo será semelhante a esta:



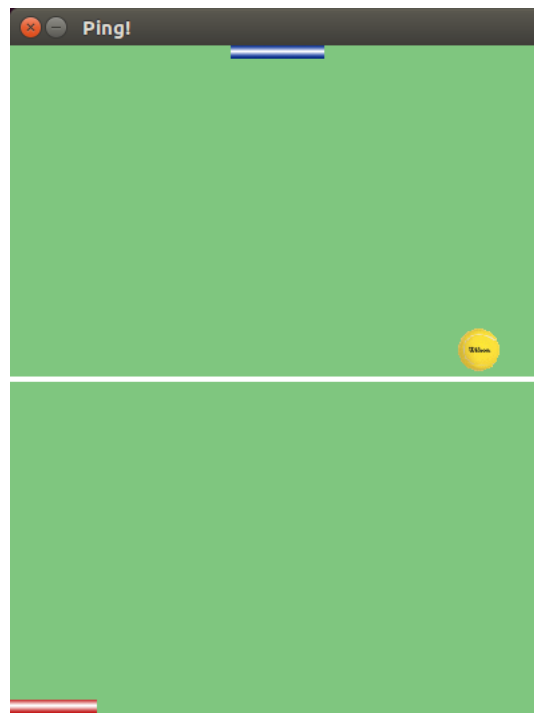


Figura B.1: Ping Pong.

## B.10 Exercício 2

Por fim, você deve encontrar uma forma de somar os pontos obtidos por cada jogador. Quando a bola atingir o fundo da tela sem ser rebatida, deve ser contado um ponto para o jogador azul. Quando a bola atingir o topo da tela sem ser rebatida, deve ser contado um ponto para o jogador vermelho. O loop do jogo deve ser modificado para que o jogo repita enquanto a soma das duas pontuações for menor que 10. Ao sair do loop, o programa deve imprimir uma mensagem no terminal com o placar final do jogo.